

College Fee Management & Receipt Generation System

Everything the team built, how it works, and how to explain it

Chalapathi Institute of Engineering and Technology, Guntur

System	SAP S/4HANA 1809 (SAP_BASIS 753), system CAL, client 110
Package	ZVYSHU · naming suffix _C60
Tools	SAP GUI for Java (macOS), Eclipse with ABAP Development Tools
Team	7 members, 7-day build
Status	Built and tested end to end on 06.10.2026
Document date	07.10.2026

Inside: architecture and payment flow, data model, security, every screen with a real screenshot, reports, OData service, test results, demo script, reviewer questions and answers, and the full source code of every program in the appendix.

Contents

1. Project at a glance	4
The problem	4
What the system does	4
2. Who uses it	5
Out of scope	5
3. How it works	5
3.1 Architecture	5
3.2 Payment lifecycle	6
4. System and technology	7
Technology used	7
5. Data model	8
Number ranges	8
Status values	8
Test data	8
6. Security and login	10
Security controls	10
7. Module walkthrough	12
7.1 Fee structure and fee demand	12
7.2 Registration and admin approval	12
7.3 Student payment submission	13
7.4 Counter payment	14
7.5 Verification	15
7.6 Receipt	15
7.7 Reports	15
7.8 Dashboard	17
7.9 Audit log	18
8. CDS view and OData service	20
8.1 The CDS view	20
8.2 The OData service	20
8.3 Why not RAP	21
9. Object catalogue	23
10. Business rules and validations	25
Payments	25
Login and roles	25

11. Testing	27
Module tests	27
End-to-end tests (06.10.2026)	27
12. Team and plan	29
Seven-day plan as executed	29
13. How to run and demo	29
Student	29
Accountant	29
Admin	30
Demo script (about 10 minutes)	30
14. Reviewer questions and answers	31
15. Limitations, open items and future work	32
Known limitations	32
Before the final demo	32
Future work	32
Appendix A. Glossary	33
Appendix B. Source code	34
B.1 ZCL_AUTH_CS60	34
B.2 ZCL_PAYMENT_C60	38
B.3 ZFEE_DEMAND_GEN_C60	43
B.4 ZSTU_FEE_VIEW_C60	46
B.5 ZSTU_PAYMENT_C60	48
B.6 ZACC_COUNTER_C60	51
B.7 ZACC_VERIFY_C60	53
B.8 ZDRIVER_FEE_RECEIPT_C60	56
B.9 ZCDS_FEE_PAY	60
B.10 ZRPT_PAY_REGISTER_C60	61
B.11 ZRPT_DEFAULTERS_C60	62
B.12 ZRPT_DAILY_COLL_C60	64
B.13 ZRPT_FEETYPE_COLL_C60	65
B.14 ZFEE_DASH_C60	67
B.15 ZRPT_AUDIT_C60	68
B.16 ZTEST_PAYMENT_C60	70

1. Project at a glance

The system records every fee payment of the college in SAP, has an accountant verify it against the bank statement, and then issues a numbered receipt that can be printed or saved as PDF. Students pay by UPI, bank transfer, demand draft, cheque or cash as they do today; there is no payment gateway, so SAP keeps the record and the accountant confirms the money.

8 custom database tables	2 classes that hold every business rule	15 application programs (screens, receipt, reports)
5 + 1 ALV reports plus a dashboard (ZFEEDASH)	1 + 1 CDS view plus a live OData web service	11 / 11 end-to-end tests passed on 06.10.2026

The problem

Fee payments were tracked by hand. Receipts were written manually, balances and defaulters were hard to see, and nobody could tell who had approved or changed what.

What the system does

- Students register, log in and submit their own payment details: amount, mode, bank and UTR or reference number.
- Accountants verify or reject each payment against the bank statement, or record cash, DD and cheque payments at the counter.
- A verified payment updates the fee balance and gets receipt number FR/yyyy/nnnnnn in one step.
- The receipt is a Smart Form with the college logo; it prints or downloads as PDF.
- Management sees collections, balances and defaulters in four reports and a dashboard.
- Every approval, verification, rejection and print is written to an audit log.
- The payment data is published as an OData service, so other systems can read it over the web.

Explain it in one minute

"Our project is a college fee management and receipt system in SAP ABAP. Students log in with their register number and enter the details of a payment they made through the bank. Nothing changes in their balance yet: the payment waits as SUBMITTED. An accountant checks it against the bank statement and approves or rejects it. On approval, in one database transaction, the balance is reduced, a receipt number is drawn from an SAP number range, a receipt record and an audit line are saved, and the receipt can be printed as a Smart Form or PDF. A rejection needs a reason, which the student sees. Reports, a dashboard and an OData service give management the totals and the defaulters. All rules sit in two classes, ZCL_AUTH_CS60 for login and ZCL_PAYMENT_C60 for payments, so every screen behaves the same way."

2. Who uses it

Three roles use the system. The role is stored per user in table ZUSERS_C60 (S = Student, A = Accountant, X = Admin) and is checked at login on every screen.

Role	Can do	Cannot do
Student	Register against the student master, log in, reset a forgotten password, view own fee demand and balance, submit a payment, see own payment history with status, receipt number and rejection reason	See another student's data, approve anything, change a payment after submitting
Accountant	Register (held PENDING until an admin approves), maintain the fee price list, generate fee demands, record counter payments (cash, DD, cheque), approve or reject payments, print receipts, run reports and the dashboard	Approve their own registration, create admins
Admin	Approve or reject accountant registrations, unlock locked users, review the audit log	Self-register: admins are created only by the seed program ZSEED_ADMIN_C60

Out of scope

- A live payment gateway. Money moves through the college bank account as today; SAP records and verifies it.
- Automatic bank-statement matching, late-fee calculation and e-mail delivery (planned, see section 15).
- A Fiori front end. The OData service in section 8 is the first step towards one.

3. How it works

3.1 Architecture

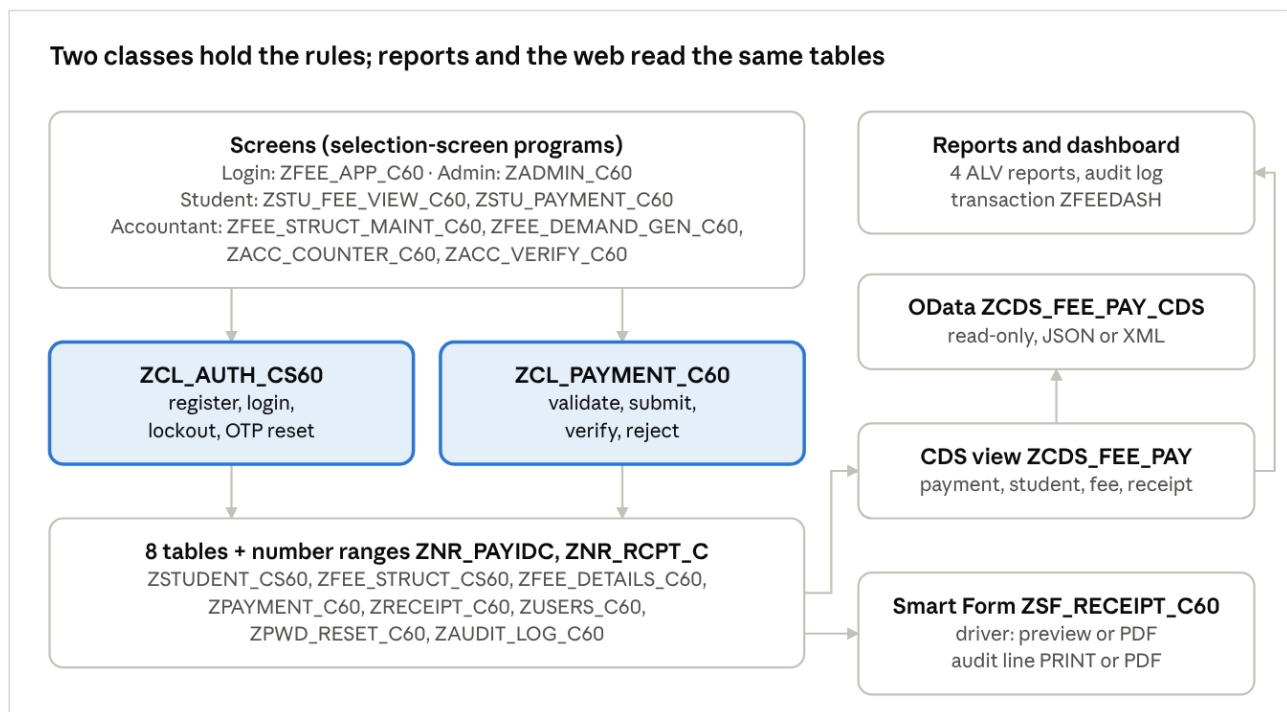


Figure 1. The layers: screens call two classes; the classes read and write the tables; reports, the CDS view, the OData service and the receipt only read.

Every login and payment screen is a thin selection-screen program. It collects input and calls a class method; the method checks the rules and writes the tables. Because the rules live in one place, a student screen and an accountant screen can never disagree about, for example, whether a reference number is a duplicate.

3.2 Payment lifecycle

A payment changes the balance only when an accountant verifies it

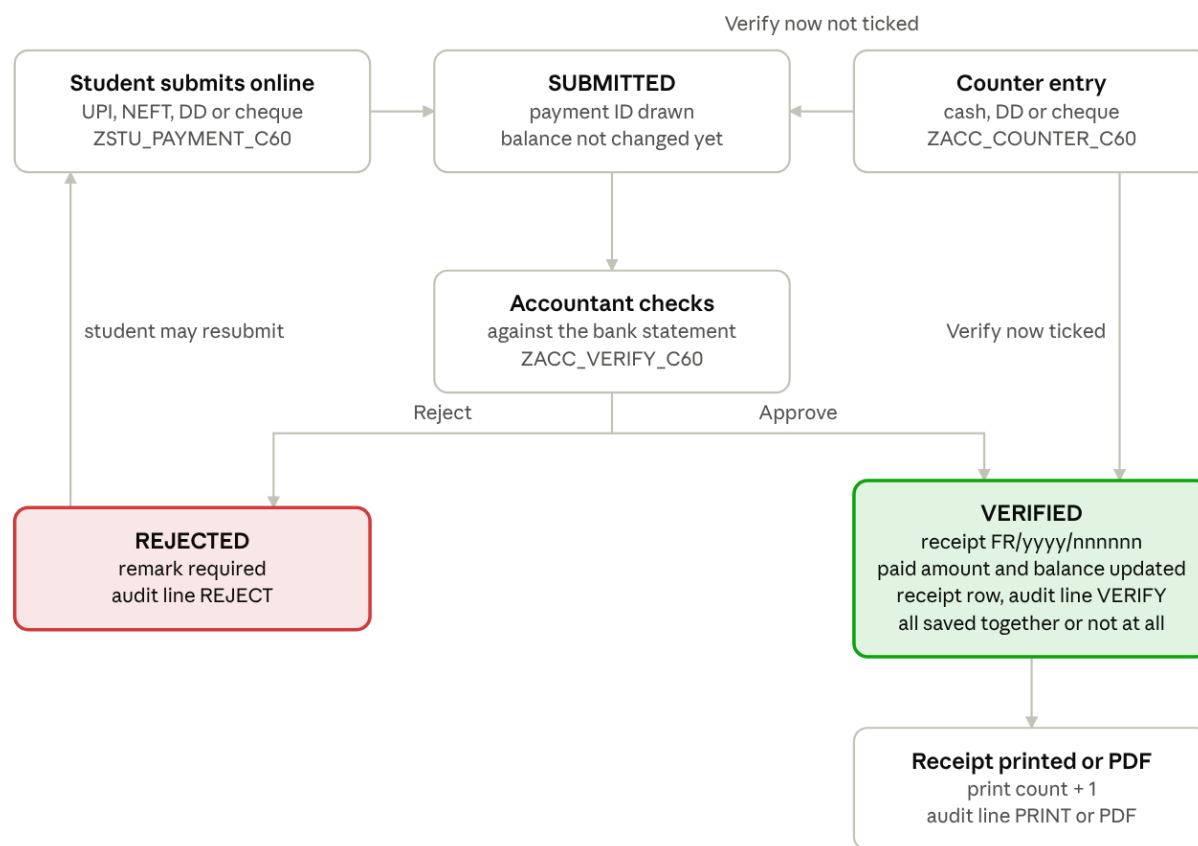


Figure 2. A payment changes money only when an accountant verifies it.

Status	Meaning	What changes in the database
SUBMITTED	Payment details entered, waiting for the accountant	New row in ZPAYMENT_C60 with a payment ID from number range ZNR_PAYIDC. The balance does not change.
VERIFIED	Accountant confirmed the money in the bank statement (or took cash at the counter)	Paid amount and balance updated in ZFEE_DETAILS_C60; receipt number FR/yyyy/nnnnnn from ZNR_RCPT_C; row in ZRECEIPT_C60; VERIFY line in ZAUDIT_LOG_C60. All in one unit of work.
REJECTED	The money was not found; a reason is required	Status, verifier, date and remark saved; REJECT audit line. The same reference may be submitted again.

Why "one unit of work" matters

Verification writes four things: the new balance, the payment status with its receipt number, the receipt row and the audit line. If any one of them fails, the method calls ROLLBACK WORK and none of them is saved. Only when all four succeed does it call COMMIT WORK. So the college can never end up with a reduced balance but no receipt, or a receipt without an audit trail.

4. System and technology

Everything runs in one SAP system, CAL, client 110, on S/4HANA 1809. The release was read from table CVERS (screenshot in section 8).

Item	Value
SAP product	S/4HANA 1809 (S4CORE 103)
ABAP platform	SAP_BASIS 753, SAP_ABA 75D
Gateway	SAP_GWFND 753, embedded in the same system (system alias LOCAL)
Client / package	110 / ZVYSHU
Naming rule	Suffix _C60 on all new objects (older objects keep _CS60 or _103)
Front ends	SAP GUI for Java on macOS; Eclipse with ABAP Development Tools (ADT) for CDS views
Web address (OData)	https://calcutta.cob.csuchico.edu:8037

Technology used

Area	What we used it for
ABAP Dictionary (SE11)	8 tables, domains, data elements, 2 foreign keys, currency reference fields
Number ranges (SNRO)	ZNR_PAYIDC for payment IDs, ZNR_RCPT_C for receipt numbers
Object-oriented ABAP	ZCL_AUTH_CS60 (login and security), ZCL_PAYMENT_C60 (payment rules), static methods returning a result structure
Selection-screen programs	All user screens: login blocks, masked passwords, radio buttons, push buttons, labels set in INITIALIZATION
ALV (CL_SALV_TABLE)	Lists with traffic lights, totals, subtotals, currency formatting, double-click events
Smart Forms	Receipt ZSF_RECEIPT_C60 with style ZSS_RECEIPT_C60 and the college logo; PDF via CONVERT_OTF
CDS view	ZCDS_FEE_PAY joins payment, student, fee demand and receipt
OData (SAP Gateway)	ZCDS_FEE_PAY published as service ZCDS_FEE_PAY_CDS
Transaction codes (SE93)	ZFEEDASH opens the fee dashboard
Hashing (CL_ABAP_MESSAGE_DIGEST)	Salted SHA-256 for passwords, security answers and OTPs

Why selection screens instead of dynpro screens? The graphical layout editor of the Screen Painter is not available in SAP GUI for Java on the Mac, so every screen is built as a selection screen. This also keeps each screen in one program that teammates can read top to bottom.

5. Data model

Eight custom tables hold all data. Fee demand and payment rows point to the student master through foreign keys on REG_NO, and every amount field references a CURRENCY field in its own table.

Table	Holds	Key (after MANDT)	Main fields
ZSTUDENT_CS60	Student master	REG_NO (CHAR 10)	NAME, COURSE, BRANCH, STUDYYEAR, PHONE, EMAIL
ZFEE_STRUCT_CS60	Fee price list	COURSE, BRANCH, STUDYYEAR, FEE_TYPE	AMOUNT, CURRENCY, DUE_DATE
ZFEE_DETAILS_C60	Fee demand per student	REG_NO, ACADEMIC_YEAR (CHAR 1), FEE_TYPE	TOTAL_FEE, CONCESSION, LATE_FEE, PAID_AMOUNT, BALANCE, CURRENCY, DUE_DATE, COURSE, BRANCH, STUDYYEAR
ZPAYMENT_C60	Every payment, whatever its status	PAYMENT_ID (NUMC 10), REG_NO	FEE_TYPE, ACADEMIC_YEAR, AMOUNT, CURRENCY, PAYMENT_MODE, TRANSACTION_ID, BANK_NAME, PAYMENT_DATE, STATUS, ENTERED_BY, VERIFIED_BY, VERIFIED_ON, REMARK, RECEIPT_NO (CHAR 16)
ZRECEIPT_C60	Issued receipts	RECEIPT_NO (CHAR 16)	PAY_ID, ISSUED_BY, ISSUED_ON, PRINT_COUNT (INT1), CANCELLED
ZUSERS_C60	Application users	USER_ID (CHAR 12)	ROLE, NAME, EMAIL, PHONE, PWD_HASH, SALT, SEC_QUESTION, SEC_ANSWER_HASH, STATUS, FAILED_ATTEMPTS, LAST_LOGIN, CREATED_ON
ZPWD_RESET_C60	Password-reset requests	USER_ID, REQ_ID (NUMC 6)	OTP_HASH, EXPIRES_AT, USED
ZAUDIT_LOG_C60	Audit trail	LOG_ID (NUMC 10)	USER_ID, ACTION, OBJECT_KEY, LOGGED_AT (UTC time stamp), REMARK

Balance rule

Every fee demand row keeps its own balance:

Balance = Total fee – Concession + Late fee – Paid amount

Number ranges

Object	Length	Interval	Used for
ZNR_PAYIDC	NUMC 10	01: 0000000001 to 9999999999, no rolling	PAYMENT_ID, drawn at submission
ZNR_RCPT_C	NUMC 6	01: 000001 to 999999, no rolling	Receipt number FR/yyyy/nnnnnn, drawn only at verification

Numbers are never reused. A number drawn by a test that was later cleaned up leaves a gap, which is normal for SAP number ranges.

Status values

- Payment (ZPAYMENT_C60-STATUS): SUBMITTED, VERIFIED, REJECTED.
- User (ZUSERS_C60-STATUS): PENDING, ACTIVE, LOCKED, REJECTED.
- Fee row (worked out on screen, not stored): PAID, PARTIAL, UNPAID, OVERDUE.

Test data

50 students, Y26CSE001 to Y26CSE025 and Y26ECE001 to Y26ECE025 (B.Tech, year 1), with two fee types each: 100 demand rows due 31.10.2026. Price list: CSE Tuition Fee 45,000 INR, ECE Tuition Fee 42,000 INR, Hostel Fee 30,000 INR for both. Seven older rows from the first version of the project (year 4 students) are still in ZFEE_DETAILS_C60. The register number format is Y + intake year + branch + serial, for example Y26CSE001.

SAP Dictionary: Display Table

Transparent Table: ZPAYMENT_C60 Active

Short Description: Student Fee Payment

Attributes Delivery and Maintenance Fields Input Help/Check Currency/Quantity Fields Indexes

	Field	Key	Initia...	Data element	Data Type	Length	Decim...	Short Description
☐	MANDT	☒	☒	MANDT	CLNT	3	0	Client
☐	PAYMENT_ID	☒	☒	ZDE_PAY_ID_C60	NUMC	10	0	payment id
☐	REG_NO	☒	☒	ZDE_REG_NO_C60	CHAR	10	0	Register Number
☐	TRANSACTION_ID	☐	☐	ZDE_TRANSACTION...	CHAR	30	0	Transaction ID
☐	CURRENCY	☐	☐	ZDE_CURRENCY_C60	CUKY	5	0	Currency
☐	AMOUNT	☐	☐	ZDEAMOUNT_C60	CURR	13	2	Amount
☐	PAYMENT_MODE	☐	☐	ZDEPAYMENT_MODE...	CHAR	15	0	Payment Mode
☐	PAYMENT_DATE	☐	☐	ZDEPAYMENT_DATE...	DATS	8	0	Payment Date

Figure 3. Table ZPAYMENT_C60 in SE11: PAYMENT_ID and REG_NO are the key; 17 fields in total.

The screenshot shows the SAP Dictionary: Display Table screen for table ZRECEIPT_C60. The 'Fields' tab is active, displaying a list of fields with their attributes. The 'Transparent Table' section shows the table name 'ZRECEIPT_C60' and the short description 'Receipt'. The 'Fields' tab shows a list of fields including MANDT, RECEIPT_NO, PAY_ID, ISSUED_BY, ISSUED_ON, PRINT_COUNT, and CANCELLED, each with checkboxes for 'Key' and 'Initi...' and their respective data elements and types.

Field	Key	Initi...	Data element	Data Type	Length	Decim...	Short Description
MANDT	☒	☒	MANDT	CLNT	3	0	Client
RECEIPT_NO	☒	☒	ZDE_RECEIPT_NO...	CHAR	16	0	Receipt Number
PAY_ID	☐	☐	ZDE_PAY_ID_C60	NUMC	10	0	payment id
ISSUED_BY	☐	☐	SYUNAME	CHAR	12	0	User Name
ISSUED_ON	☐	☐	SYDATUM	DATS	8	0	System Date
PRINT_COUNT	☐	☐	ZDE_PRINT_COUNT...	INT1	3	0	Print Count
CANCELLED	☐	☐	ZDE_CANCELLED_C...	CHAR	1	0	Cancelled

Figure 4. Table ZRECEIPT_C60: one row per receipt, with print count and cancelled flag.

6. Security and login

The app has its own login, built in class ZCL_AUTH_CS60, because students and accountants register themselves and do not get SAP user IDs. No password is stored or readable in plain text.

Method	What it does
REGISTER	Checks the password rule and duplicates; checks that a student's register number exists in ZSTUDENT_CS60; creates a random 32-character salt (UUID); stores the password and the security answer as hashes; sets students ACTIVE and accountants PENDING. Admins only with the seed flag.
LOGIN	Rejects PENDING, LOCKED and REJECTED users; compares the hash; on success resets FAILED_ATTEMPTS and stores LAST_LOGIN; on failure adds 1 and locks the user at 3. Returns the role.
REQUEST_RESET	Checks the security answer, then creates a 6-digit OTP valid for 10 minutes. Only the OTP's hash is stored; older open requests are cancelled.
RESET_PASSWORD	Checks the OTP (latest, unused, not expired), applies the password rule, saves the new hash, clears FAILED_ATTEMPTS, sets the user ACTIVE and marks the OTP used.
HASH (private)	SHA-256 of salt + text, using CL_ABAP_MESSAGE_DIGEST.

Security controls

Control	How it works
Password storage	Salted SHA-256 per user; the same password gives a different hash for every user
Password rule	At least 8 characters with at least one letter and one digit
Lockout	3 wrong passwords set the user to LOCKED; an admin unlock or a password reset clears it
No account guessing	An unknown user and a wrong password give the same message: "Invalid user ID or password"
Password reset	Security question plus a single-use OTP that expires after 10 minutes (shown on screen in the demo, as no mail server is set up)
Accountant onboarding	New accountants stay PENDING until an admin approves them in ZADMIN_C60
Role check on every screen	Each program logs the user in and stops if the role does not fit, for example "Only accountants can verify payments"
Own data only	A student's register number is taken from the login, never typed, so one student cannot open another student's fees or payments
Masked passwords	Password fields are hidden on every screen and cleared right after the check
Audit trail	Approvals, verifications, rejections, prints and PDFs are written to ZAUDIT_LOG_C60

The reports and the dashboard are management tools without an app login of their own; access to them is controlled by the SAP user and transaction authorization.

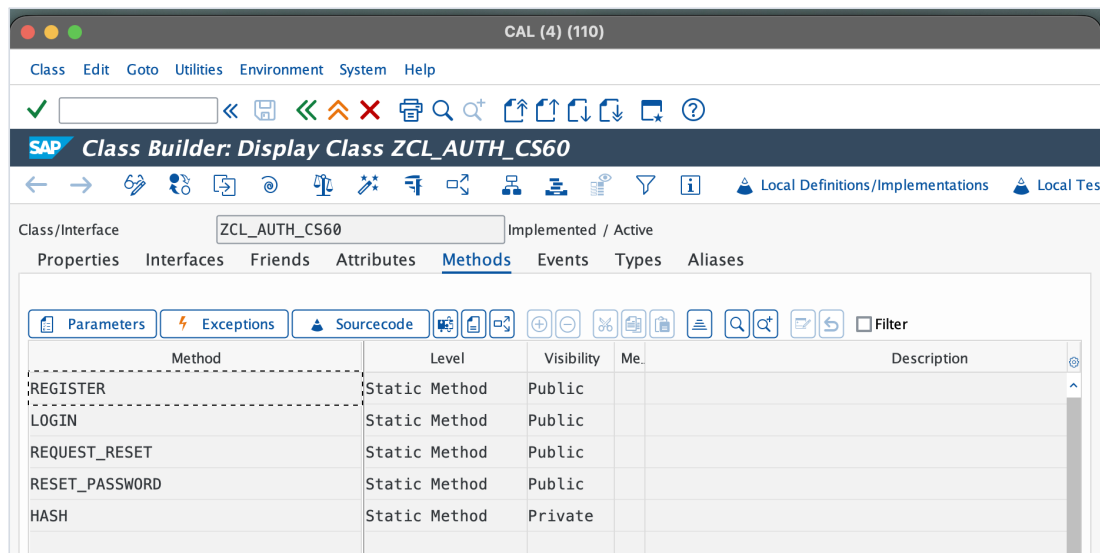


Figure 5. Class ZCL_AUTH_CS60 in the Class Builder: four public methods and the private HASH method.

Fld Name	Value	Value Unconverted	Technical Field Name
Client	110	110	MANDT
USER ID	ACC001	ACC001	USER_ID
Role (S/A/X)	A	A	ROLE
Name	Accountant 1	Accountant 1	NAME
EMAIL ID	acc1@college.edu	acc1@college.edu	EMAIL
Phone Number	9876511111	9876511111	PHONE
Hash	85FE02BC9, (hidden in this document)	85FE02BC9, (hidden in this document)	PWD_HASH
salt	6145A68C4 (hidden in this document)	6145A68C4 (hidden in this document)	SALT
Security question	City?	City?	SEC_QUESTION
Hash	DF1A7B4F2! (hidden in this document)	DF1A7B4F2 (hidden in this document)	SEC_ANSWER_HASH
User Status	ACTIVE	ACTIVE	STATUS
INT1	0	0	FAILED_ATTEMPTS
Time Stamp	20.261.006.064.927	20261006064927	LAST_LOGIN
Due Date	03.10.2026	20261003	CREATED_ON
Number of Entries	0	0	LINE_INDEX

Figure 6. A user record in ZUSERS_C60: only a hash and a salt are stored, never the password (values masked here).

7. Module walkthrough

This section follows the work in the order it happens in the college, with a real screenshot of each screen from the test system.

7.1 Fee structure and fee demand

The accountant keeps one price list and turns it into a fee demand for every student in one run.

- **ZFEE_STRUCT_MAINT_C60** maintains ZFEE_STRUCT_CS60: one amount, currency and due date per course, branch, year and fee type.
- **ZFEE_DEMAND_GEN_C60** creates one ZFEE_DETAILS_C60 row per student per matching fee type, with balance = total fee. Test run (ticked by default) only shows a preview. Existing rows are skipped and never overwritten, so the program can be run again safely. Rows are inserted one by one, so one duplicate does not stop the batch.
- **ZSTU_FEE_VIEW_C60** shows a student's fee rows with a traffic light (PAID, PARTIAL, UNPAID, OVERDUE) and totals in the header. Students see only their own rows; staff enter a register number.

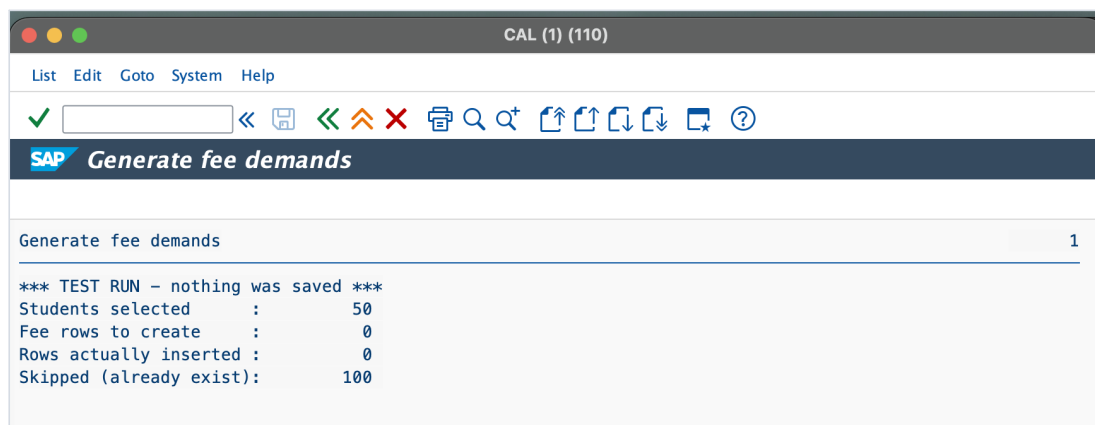


Figure 7. Demand generation in test mode: 50 students selected, 100 rows already exist, nothing saved.

Figure 8 shows the SAP CAL (2) (110) screen titled "Fee details for Y26CSE001 - CSE Student 1". The screen displays a table with the following data:

Fee Type	AcademicYr	Total Fee	Concession	Late Fee	Amount pd	Balance	Currency	Due Date	Status
Hostel Fee	1	30.000,00	0,00	0,00	0,00	30.000,00	INR	31.10.2026	UNPAID
Tuition Fee	1	45.000,00	0,00	0,00	3.000,00	42.000,00	INR	31.10.2026	PARTIAL

Figure 8. Student fee view for Y26CSE001: Hostel Fee unpaid, Tuition Fee partly paid.

7.2 Registration and admin approval

ZFEE_APP_C60 gives three screens: Login, Forgot Password and New Registration. A new accountant cannot log in until an admin approves the account in **ZADMIN_C60**, which lists users by status and can approve, reject or unlock them; every action is written to the audit log.

CAL (3) (110)

Program Edit Goto System Help

✓

<<



<>





















SAP Admin – user approval



Administrator login

Admin user ID

Admin password

Action

Target user ID

Figure 9. ZADMIN_C60: admin login, then the target user and the action.

CAL (3) (110)

List Edit Goto Settings System Help

✓ <<  <<           

SAP Users and status

USER ID	Role S/A/X	Name		INT1	
ADM001	X	Admin	admin@college.edu	ACTIVE	0 03.10.2026
ADMIN01	X	College Admin	admin@college.edu	ACTIVE	0 03.10.2026
Y26CSE001	S	CSE Student 1	y26cse1@college.edu	ACTIVE	0 05.10.2026
ACC001	A	Accountant 1	acc1@college.edu	PENDING	0 03.10.2026

Figure 10. User list in ZADMIN_C60: accountant ACC001 is still PENDING.

7.3 Student payment submission

ZSTU_PAYMENT_C60. The student logs in, chooses fee type, year, amount, UTR or reference, bank and mode (UPI, NEFT, DD or cheque), and presses F8. The reference is stored in capitals. Leaving the amount empty only shows the history: newest first, a green, yellow or red light, the receipt number of verified payments and the reason for rejected ones.

Program

Edit

Goto

System

Help

✓

<<

<<

>>

<

>

Student payment submission

Step 1 – Log in

User ID (Reg. No.)

Password

Step 2 – Payment details (leave Amount empty to only view your payments)

Fee type

Tuition Fee

Year of study

1

Amount paid

UTR / Reference no.

Bank name

Payment mode

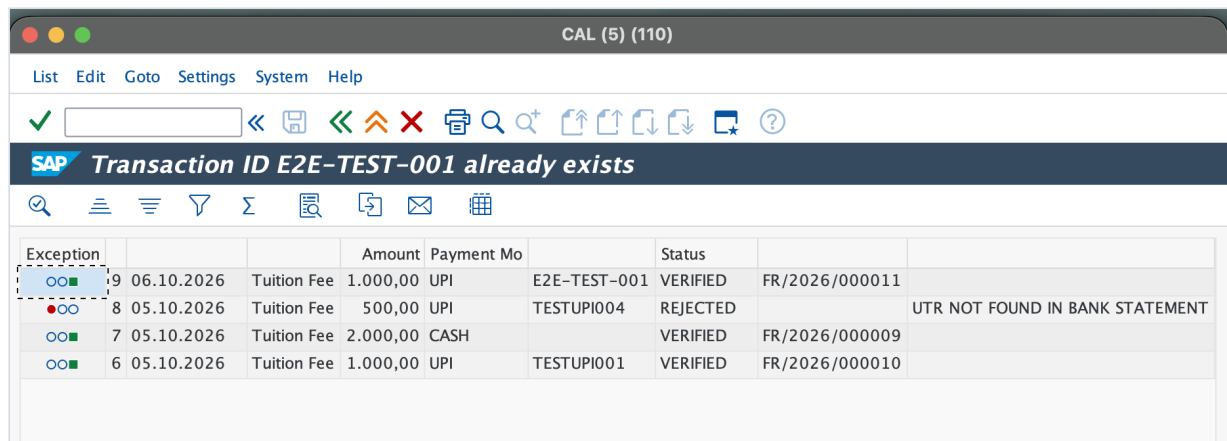
☒ UPI

☐ NEFT / bank transfer

☐ Demand Draft (DD)

☐ Cheque

Figure 11. Student payment screen: log in, payment details, payment mode.

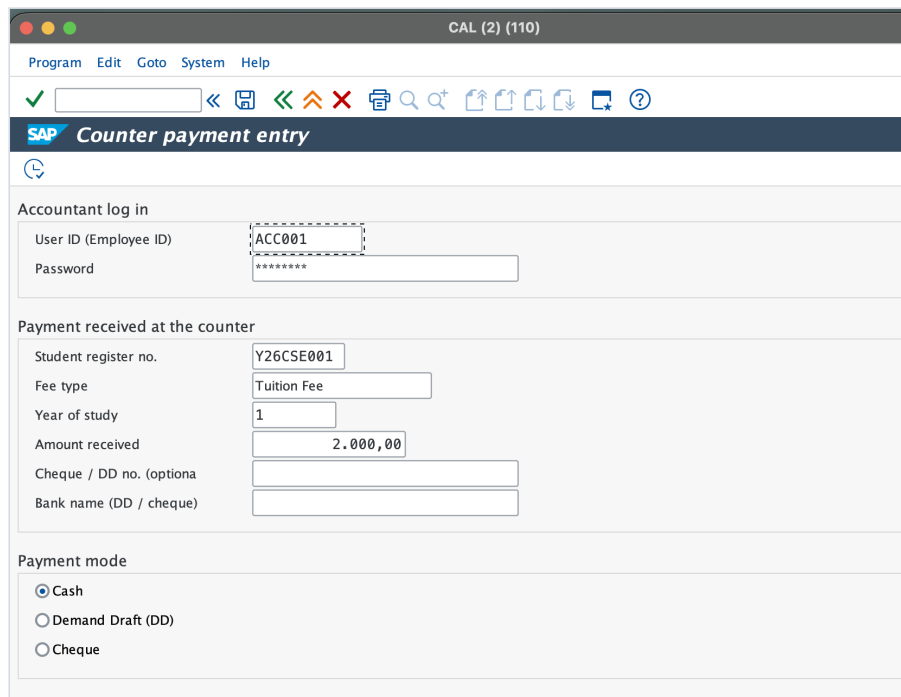


Exception			Amount	Payment Mo		Status	
00	9	06.10.2026	Tuition Fee	1.000,00 UPI	E2E-TEST-001	VERIFIED	FR/2026/000011
00	8	05.10.2026	Tuition Fee	500,00 UPI	TESTUPI004	REJECTED	UTR NOT FOUND IN BANK STATEMENT
00	7	05.10.2026	Tuition Fee	2.000,00 CASH		VERIFIED	FR/2026/000009
00	6	05.10.2026	Tuition Fee	1.000,00 UPI	TESTUPI001	VERIFIED	FR/2026/000010

Figure 12. Student history after a blocked duplicate: payment 9 verified with FR/2026/000011, payment 8 rejected with its reason.

7.4 Counter payment

ZACC_COUNTER_C60 is for money paid at the college counter: cash, DD or cheque. DD and cheque need a number and a bank. With "Verify now and issue the receipt" ticked (the default) the payment is verified at once, because the accountant already holds the money; otherwise it goes to the verification list.



Accountant log in

User ID (Employee ID): ACC001

Password: *****

Payment received at the counter

Student register no.: Y26CSE001

Fee type: Tuition Fee

Year of study: 1

Amount received: 2.000,00

Cheque / DD no. (optional):

Bank name (DD / cheque):

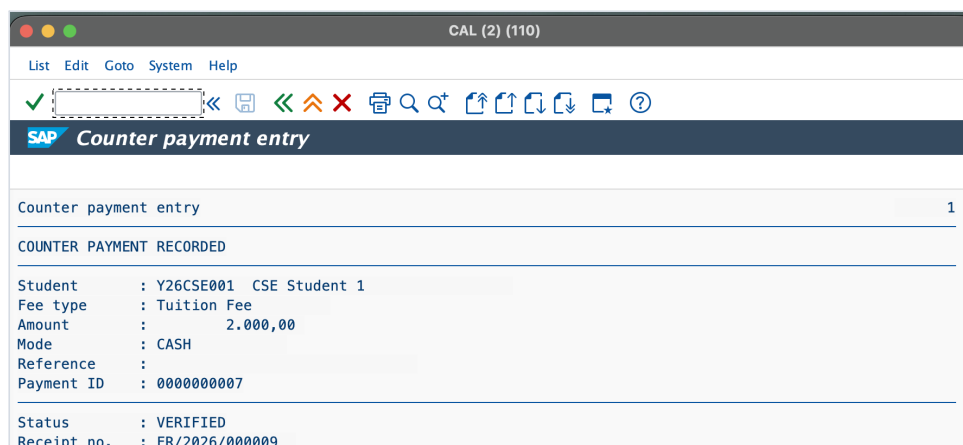
Payment mode

☒ Cash

☐ Demand Draft (DD)

☐ Cheque

Figure 13. Counter entry: accountant login, student register number, amount and mode.



Counter payment entry 1

COUNTER PAYMENT RECORDED

Student : Y26CSE001 CSE Student 1

Fee type : Tuition Fee

Amount : 2.000,00

Mode : CASH

Reference :

Payment ID : 0000000007

Status : VERIFIED

Receipt no. : FR/2026/000009

Figure 14. Result: cash payment 0000000007 verified at once, receipt FR/2026/000009.

7.5 Verification

ZACC_VERIFY_C60 is for accountants only. It lists SUBMITTED payments with student name, amount, mode, reference and bank. A double-click opens a popup that shows the payment and asks "Did you check this in the bank statement?" with Approve and Reject. Reject asks for the reason. The list refreshes after every action; a checkbox also shows verified and rejected payments.

	RegisterNo	Name	Amount	Payment Mo	USER ID	Status	
6	05.10.2026	Y26CSE001 CSE Student 1	Tuition Fee 1.000,00	UPI	TESTUPI001	VERIFIED	FR/2026/000010
7	05.10.2026	Y26CSE001 CSE Student 1	Tuition Fee 2.000,00	CASH	ACC001	VERIFIED	FR/2026/000009
8	05.10.2026	Y26CSE001 CSE Student 1	Tuition Fee 500,00	UPI	TESTUPI004	REJECTED	
9	06.10.2026	Y26CSE001 CSE Student 1	Tuition Fee 1.000,00	UPI	E2E-TEST-001	SUBMITTED	

Figure 15. Verification list with all payments shown: payment 9 waits as SUBMITTED.

7.6 Receipt

The receipt is the Smart Form **ZSF_RECEIPT_C60** (style ZSS_RECEIPT_C60, interface structure ZST_FEE_103), printed through the receipt driver program. Enter the receipt number and press F8 for a print preview, or tick "Save as PDF file" to get a Save dialog with the file name FR_2026_000011.pdf. Every print or PDF adds 1 to PRINT_COUNT and writes a PRINT or PDF audit line.

Window	Content
Logo and college	CIET logo, college name and address (Lam, Guntur, Andhra Pradesh)
Title	"FEE RECEIPT" banner
Billed to / receipt info	Student name, register number, course, branch, year; receipt number, date, paid on, status
Fee table	Fee type, total fee, concession, late fee, paid now, balance; then total fee, paid in this receipt, total paid so far, balance due
Payment information	Mode, reference number, bank, verified by; terms and conditions
Footer	Authorised signatory (Accounts Office), "Thank you for your payment"

Figure 16. Receipt FR/2026/000010 in print preview: paid now 1,000.00, total paid so far 3,000.00, balance due 42,000.00.

7.7 Reports

All reports are ALV lists with sorting, filtering, totals and export, and with proper field names on the selection screen.

Report	Shows	Filters
Payment Register (ZRPT_PAY_REGISTER_C60)	Every payment from ZCDS_FEE_PAY with student, receipt and status; total amount	Register no., payment date, status, fee type, mode
Fee Defaulters (ZRPT_DEFAULTERS_C60)	Fee rows still open after the due date: contact details, amounts, due date and days overdue, most overdue first; total balance	Register no., branch, year, overdue as on date, "Show not-yet-due balances"
Daily Collection (ZRPT_DAILY_COLL_C60)	VERIFIED payments per day and mode: count and amount, subtotal per day, grand totals	Payment date, payment mode
Fee-Type-wise Collection (ZRPT_FEETYPE_COLL_C60)	Per fee type and branch: students, demand, concession, collected, balance, % collected; subtotals and grand totals	Fee type, branch, year
Audit Log (ZRPT_AUDIT_C60)	Every audit line, newest first, date and time in local time	User ID, action, date

CAL (1) (110)

List Edit Goto Settings System Help

☒

SAP Payment Register

RegisterNo	Name				: Amount	Currency	Payment Mo	Status	AcademicYr	USER ID	USER ID			
8 Y26CSE001	CSE Student 1 CSE B.Tech 1	TESTUPI004			500,00	INR	UPI	05.10.2026	REJECTED	Tuition Fee 1	SBI Y26CSE001 ACC001	05.10.2026	UTR NOT FOUND IN BANK STATEMENT	
7 Y26CSE001	CSE Student 1 CSE B.Tech 1				2,000,00	INR	CASH	05.10.2026	VERIFIED	Tuition Fee 1	ACC001	ACC001	05.10.2026	FR/2026/
6 Y26CSE001	CSE Student 1 CSE B.Tech 1	TESTUPI001			1,000,00	INR	UPI	05.10.2026	VERIFIED	Tuition Fee 1	SBI Y26CSE001 ACC001	05.10.2026	FR/2026/	

Figure 17. Payment Register on 05.10.2026: three payments, total 3,500.00 (includes the rejected 500.00).

CAL (2) (110)												
List	Edit	Goto	Settings	System	Help							
✓		◀	📁	⏮	⏭	✖	🖨	🔍	🔍+	🔄	🔄	?
SAP Fee Defaulters												
🔍	≡	≡	🌐	Σ	Σ	📄	📄	✉	🗒			
Reg. No.	Student Name	Branch	Year	Phone	E-mail	Fee Type	Currency	Total Fee	Concession	Late Fee	Paid	Balance
masked	name masked	CSE	4	masked		TUITION	INR	50.000,00	0,00	0,00	18.000,00	32.00
masked	name masked	CSC	4	masked		TUITION	INR	50.000,00	0,00	0,00	15.000,00	35.00
masked	name masked	CSE	4	masked		TUITION	INR	50.000,00	0,00	0,00	20.000,00	30.00
masked	name masked	CSC	4	masked		TUITION	INR	50.000,00	0,00	0,00	10.000,00	40.00
masked	name masked	CSE	4	masked		TUITION	INR	50.000,00	0,00	0,00	17.000,00	33.00
masked	name masked	CSE	4	masked		TUITION	INR	50.000,00	0,00	0,00	25.000,00	25.00
masked	name masked	CSE	4	masked		TUITION	INR	50.000,00	0,00	0,00	42.000,00	8.00
Y26CSE001	CSE Student 1	CSE	1	9876500001	y26cse1@college.edu	Hostel Fee	INR	30.000,00	0,00	0,00	0,00	30.00
Y26CSE001	CSE Student 1	CSE	1	9876500001	y26cse1@college.edu	Tuition Fee	INR	45.000,00	0,00	0,00	3.000,00	42.00
Y26CSE002	CSE Student 2	CSE	1	9876500002	y26cse2@college.edu	Tuition Fee	INR	45.000,00	0,00	0,00	0,00	45.00
Y26CSE002	CSE Student 2	CSE	1	9876500002	y26cse2@college.edu	Hostel Fee	INR	30.000,00	0,00	0,00	0,00	30.00
Y26CSE003	CSE Student 3	CSE	1	9876500003	y26cse3@college.edu	Hostel Fee	INR	30.000,00	0,00	0,00	0,00	30.00
Y26CSE003	CSE Student 3	CSE	1	9876500003	y26cse3@college.edu	Tuition Fee	INR	45.000,00	0,00	0,00	0,00	45.00
Y26CSE004	CSE Student 4	CSE	1	9876500004	y26cse4@college.edu	Tuition Fee	INR	45.000,00	0,00	0,00	0,00	45.00
Y26CSE004	CSE Student 4	CSE	1	9876500004	y26cse4@college.edu	Hostel Fee	INR	30.000,00	0,00	0,00	0,00	30.00
Y26CSE005	CSE Student 5	CSE	1	9876500005	y26cse5@college.edu	Tuition Fee	INR	45.000,00	0,00	0,00	0,00	45.00
Y26CSE005	CSE Student 5	CSE	1	9876500005	y26cse5@college.edu	Hostel Fee	INR	30.000,00	0,00	0,00	0,00	30.00
Y26CSE006	CSE Student 6	CSE	1	9876500006	y26cse6@college.edu	Tuition Fee	INR	45.000,00	0,00	0,00	0,00	45.00

Figure 18. Fee Defaulters with "Show not-yet-due balances" ticked: contact details, balances and due dates.

Date	Payment Mo	Currency	No. of Pay	Amount Col
05.10.2026	CASH	INR	1	2.000,00
	UPI	INR	1	1.000,00
05.10.2026		INR	2	3.000,00
		INR	2	3.000,00

Figure 19. Daily Collection: 2,000.00 cash and 1,000.00 UPI on 05.10.2026.

Fee Type	Branch	Currency	No. of Stu	Total Demand	Concession	Collected	Balance	% Collecte
Hostel Fee	CSE	INR	25	750.000,00	0,00	0,00	750.000,00	0,00
Hostel Fee		INR	25	750.000,00	0,00	0,00	750.000,00	
HOSTEL FEE	ECE	INR	25	750.000,00	0,00	0,00	750.000,00	0,00
HOSTEL ...		INR	25	750.000,00	0,00	0,00	750.000,00	
TUITION	CSC	INR	3	150.000,00	0,00	45.000,00	105.000,00	30,00
	CSE	INR	4	200.000,00	0,00	102.000,00	98.000,00	51,00
TUITION		INR	7	350.000,00	0,00	147.000,00	203.000,00	
Tuition Fee	CSE	INR	25	1.125.000,00	0,00	3.000,00	1.122.000,00	0,27
	ECE	INR	25	1.050.000,00	0,00	0,00	1.050.000,00	0,00
Tuition F...		INR	50	2.175.000,00	0,00	3.000,00	2.172.000,00	
		INR	107	4.025.000,00	0,00	150.000,00	3.875.000,00	

Figure 20. Fee-Type-wise Collection: demand 4,025,000.00, collected 150,000.00, balance 3,875,000.00 before the end-to-end test.

7.8 Dashboard

Transaction **ZFEEDASH** opens ZFEE_DASH_C60: seven key figures and four buttons that open the reports. The figures are read again every time the screen appears, so they are current after returning from a report.

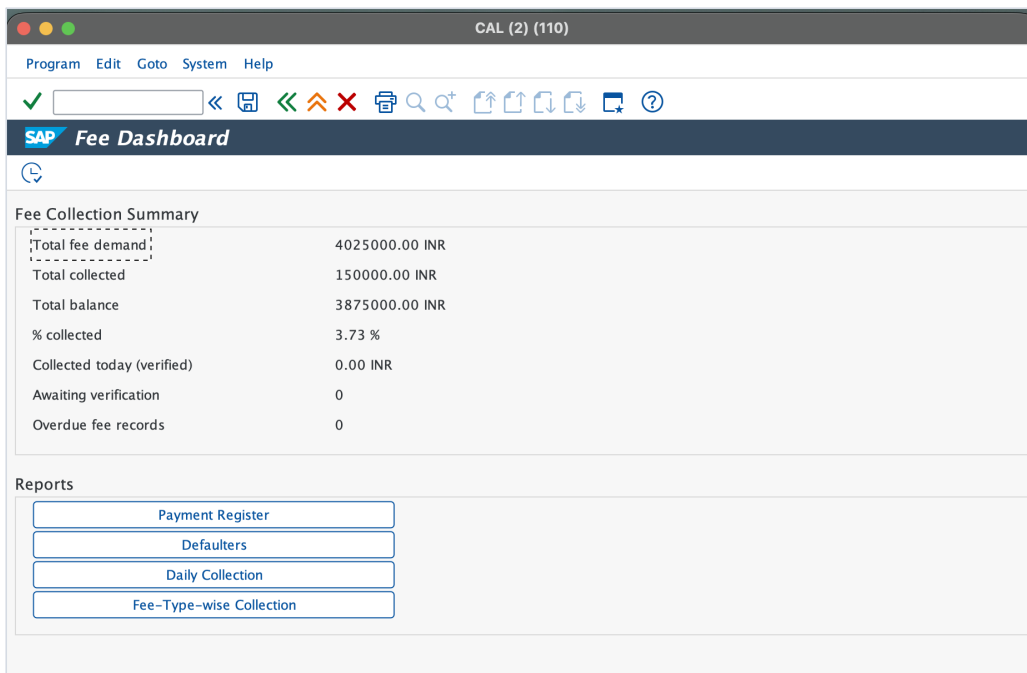


Figure 21. The dashboard before the end-to-end test (collected 150,000.00).

Figure	After the end-to-end test (06.10.2026)
Total fee demand	4,025,000.00 INR
Total collected	151,000.00 INR
Total balance	3,874,000.00 INR
% collected	3.75 %
Collected today (verified)	1,000.00 INR
Awaiting verification	0
Overdue fee records	0

7.9 Audit log

ZAUDIT_LOG_C60 records who did what, to which object and when. Each line gets the next LOG_ID, the user, a UTC time stamp and a remark; ZRPT_AUDIT_C60 shows it in local time.

Action	Written by	Object key
APPROVE / REJECT / UNLOCK (users)	ZADMIN_C60	User ID
VERIFY	ZCL_PAYMENT_C60=>VERIFY	Receipt number
REJECT (payment)	ZCL_PAYMENT_C60=>REJECT	Payment ID
PRINT / PDF	Receipt driver	Receipt number

Log No.	Date	Time	Done By	Action	Receipt /	Remark
8	06.10.2026	07:54:22	CIOE-027	PDF	FR/2026/000010	Receipt FR/2026/000010 printed
7	05.10.2026	23:49:54	ACC001	REJECT	0000000008	UTR NOT FOUND IN BANK STATEMENT
6	05.10.2026	23:37:44	ACC001	VERIFY	FR/2026/000010	Payment 0000000006 verified
5	05.10.2026	23:33:23	ACC001	VERIFY	FR/2026/000009	Payment 0000000007 verified
4	05.10.2026	23:32:25	ADM001	APPROVE	ACC001	By admin tool
3	05.10.2026	23:04:41	TESTACC	VERIFY	FR/2026/000008	Payment 0000000005 verified
2	05.10.2026	23:04:41	TESTACC	REJECT	0000000004	UTR not found in bank statement
1	05.10.2026	23:04:41	TESTACC	VERIFY	FR/2026/000007	Payment 0000000003 verified

Figure 22. Audit log: approval of ACC001 by the admin, verifications, a rejection with its reason and a PDF print.

8. CDS view and OData service

One CDS view, ZCDS_FEE_PAY, gives one row per payment with everything about it, and the same view is published on the web as OData service ZCDS_FEE_PAY_CDS. It was tested live in Chrome on 06.10.2026.

8.1 The CDS view

- SQL view ZCDS_FEE_C60. Joins ZPAYMENT_C60 with ZSTUDENT_CS60 (inner join), and ZFEE_DETAILS_C60 and ZRECEIPT_C60 (left outer joins), so payments without a receipt still appear.
- Keys payment_id and reg_no. Fields: student name, branch, course, year; amount and currency; mode, date, status, fee type, year, bank, entered by, verified by and on, remark, receipt number; receipt issue date, print count, cancelled flag; the fee row's total fee, concession, late fee, paid amount, balance and due date.
- The payment table stores the year as text up to 9 characters, the fee table as 1 character, so the join uses `substring(p.academic_year, 1, 1)`.
- Amount fields carry @Semantics currency annotations, so reports and OData show amounts with their currency.
- The Payment Register report reads directly from the view.

8.2 The OData service

Item	Value
How it was made	Annotation @OData.publish: true on ZCDS_FEE_PAY
Service	ZCDS_FEE_PAY_CDS, version 1, registered in /n/IWFND/MAINT_SERVICE with system alias LOCAL
ICF node	default_host / sap / opu / odata / sap / zcds_fee_pay_cds (active)
Entity set	ZCDS_FEE_PAY, read-only (create, update and delete are false)
Format	OData V2, XML or JSON (\$format=json)

The screenshot shows the SAP 'Add Selected Services' dialog. In the 'Filter' section, 'System Alias' is set to 'LOCAL', 'Technical Service Name' is 'ZCDS_FEE_PAY_CDS', and 'External Service Name' is empty. The 'Select Backend Services' table has the following data:

Type	Technical Service Name	Vers.	Service Description	External Service Name	Namespace
BEP	ZCDS_FEE_PAY_CDS	1	Student Fee Payment	ZCDS_FEE_PAY_CDS	

Figure 23. Registering the service in /n/IWFND/MAINT_SERVICE (Add Selected Services, system alias LOCAL).

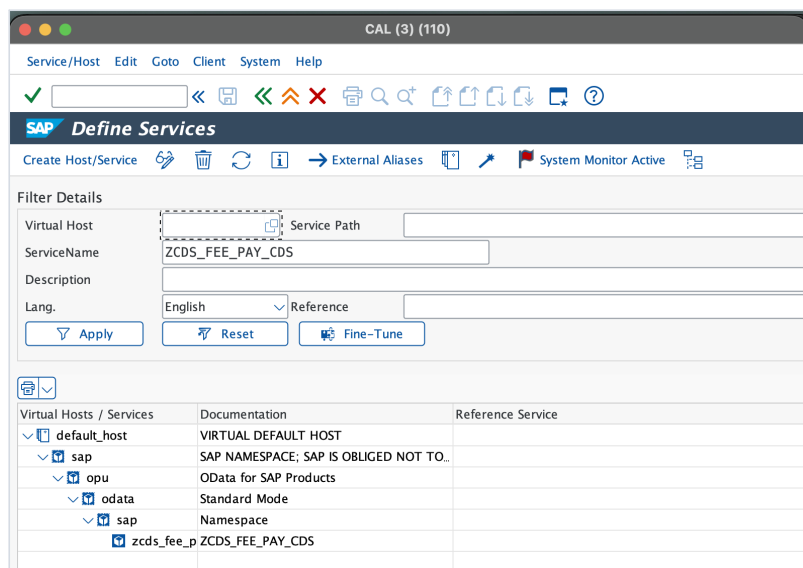


Figure 24. SICF: the service node `zcds_fee_pay_cds` under `/sap/opu/odata/sap` is active.

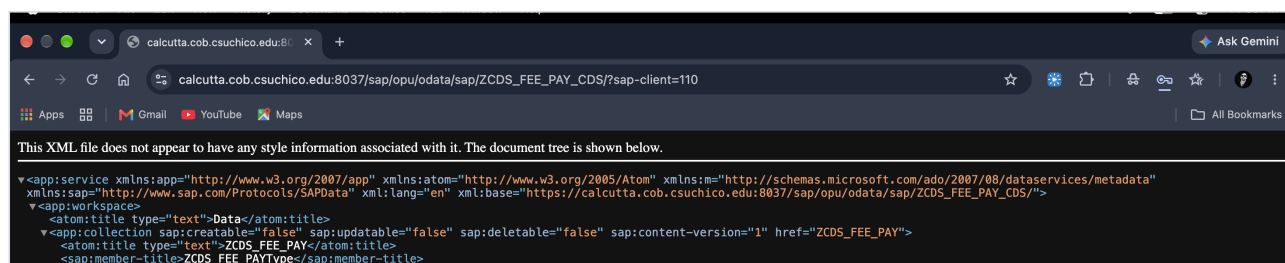


Figure 25. The service document in Chrome lists the collection `ZCDS_FEE_PAY`.

Test addresses (log in with the SAP user and SAP password)

What	Address
Service document	<code>https://calcutta.cob.csuchico.edu:8037/sap/opu/odata/sap/ZCDS_FEE_PAY_CDS/?sap-client=110</code>
First 5 payments (JSON)	<code>.../ZCDS_FEE_PAY_CDS/ZCDS_FEE_PAY?\$top=5&\$format=json&sap-client=110</code>
Verified payments only	<code>.../ZCDS_FEE_PAY?\$filter=status eq 'VERIFIED'&\$format=json&sap-client=110</code>
Number of payments	<code>.../ZCDS_FEE_PAY/\$count?sap-client=110</code>

The JSON test returned payments 6 to 10 of Y26CSE001 with live values, for example payment 9: E2E-TEST-001, VERIFIED, receipt FR/2026/000011, total fee 45,000.00, paid 4,000.00, balance 41,000.00.

8.3 Why not RAP

A RAP business object (a behavior definition with Approve and Reject actions) needs SAP_BASIS 754 or higher, which means S/4HANA 1909 or newer. This system is SAP_BASIS 753 (S/4HANA 1809): the Behavior Definition wizard opens in Eclipse, but the system returns no implementation types. The OData service is the supported way to expose the data on this release. Approve and Reject stay in ZACC_VERIFY_C60, where they are tested.

Table: CVERS
Displayed Fields: 5 of 5 Fixed Columns: 1 List Width 0250

COMPONENT	RELEASE	EXTRELEASE	COMP_TYPE	DESC_TEXT
☐ INSURANCE	803	0001	I	SAP Insurance
☐ IS-OIL	803	0001	I	IS-OIL
☐ IS-PRA	803	0001	I	SAP IS-PRA
☐ IS-PS-CA	803	0001	I	IS-PUBLIC SECTOR CONTRACT ACCOUNTING
☐ IS-UT	803	0001	I	SAP Utilities/Telecommunication
☐ MDG_APPL	803	0001	R	MDG Applications
☐ MDG_FND	803	0001	V	MDG Foundation
☐ MDG_LUX	803	0001	R	MDG Additional User Interface
☐ MSGPMCON	200	0000000003	C	msg.PM Connection
☐ PERSONAS	300	0000	C	PERSONAS - SAP GUI PERSONALISATION
☐ S4CORE	103	0001	R	S4CORE
☐ S4FND	103	0001	V	Foundation
☒ SAP_ABA	750	0001	S	Cross-Application Component
☒ SAP_BASIS	753	0001	S	SAP Basis Component
☐ SAP_BW	753	0001	W	SAP Business Warehouse
☐ SAP_GWFND	753	0001	S	SAP Gateway Foundation
☐ SAP_HR	608	0063	R	Human Resources
☐ SAP_HRCAE	608	0063	R	Subcomponent SAP_HRCAE of SAP_HR

Figure 26. Table CVERS: SAP_BASIS 753 and S4CORE 103 (S/4HANA 1809).

Known server limitation. The Gateway Client test tool (/n/IWFND/GW_CLIENT) cannot call this server itself ("Check the ICF nodes... at least one node is inactive" over HTTP, "HTTP Internal Error" over HTTPS), although every ICF node is active. Testing from the browser works.

9. Object catalogue

Everything the team created in package ZVYSHU, grouped by purpose.

Object	Type	Used by	Purpose
ZCL_AUTH_CS60	Class	All screens	Register, login, lockout, OTP reset, hashing
ZCL_PAYMENT_C60	Class	Payment screens, tests	VALIDATE, SUBMIT, VERIFY, REJECT
ZFEE_APP_C60	Program	Everyone	Login, Forgot Password and New Registration screens
ZADMIN_C60	Program	Admin	List users by status; approve, reject or unlock; audit log
ZFEE_STRUCT_MAINT_C60	Program	Accountant	Fee price list: list, add or update, delete
ZFEE_DEMAND_GEN_C60	Program	Accountant	Create fee demand rows from the price list, with test-run mode
ZSTU_FEE_VIEW_C60	Program	Student, staff	Fee rows with PAID / PARTIAL / UNPAID / OVERDUE lights and totals
ZSTU_PAYMENT_C60	Program	Student	Submit a payment and see own payment history
ZACC_COUNTER_C60	Program	Accountant	Record cash, DD or cheque at the counter, optionally verify at once
ZACC_VERIFY_C60	Program	Accountant	List of SUBMITTED payments; double-click to approve or reject
ZSF_RECEIPT_C60	Smart Form	Receipt driver	Receipt layout (style ZSS_RECEIPT_C60, structure ZST_FEE_103)
ZDRIVER_FEE_RECEIPT_C60	Program	Accountant	Print or save a receipt as PDF, count prints, write the audit log
ZCDS_FEE_PAY	CDS view	Reports, OData	One row per payment with student, fee demand and receipt
ZCDS_FEE_PAY_CDS	OData service	Browser, other systems	Read-only web access to ZCDS_FEE_PAY
ZRPT_PAY_REGISTER_C60	ALV report	Accountant, management	Payment register with filters and total
ZRPT_DEFAULTERS_C60	ALV report	Accountant, management	Balances past the due date, days late
ZRPT_DAILY_COLL_C60	ALV report	Accountant, management	Verified collections per day and mode
ZRPT_FEETYPE_COLL_C60	ALV report	Accountant, management	Demand, collected, balance and % per fee type and branch
ZFEE_DASH_C60 / ZFEEDASH	Program + transaction	Management	Key figures and buttons to the four reports
ZRPT_AUDIT_C60	ALV report	Admin	Audit log with user, action and date filters
ZNR_PAYIDC, ZNR_RCPT_C	Number ranges	ZCL_PAYMENT_C60	Payment IDs and receipt numbers
ZLOAD_TESTDATA_C60	Helper	Team	Loads the 50 test students
ZSEED_ADMIN_C60	Helper	Team	Creates the permanent admin (the only way to create one)

Object	Type	Used by	Purpose
ZTEST_AUTH_C60, ZTEST_AUTH2_C60	Test programs	Team	9 login cases and 8 password-reset cases
ZTEST_PAYMENT_C60	Test program	Team	16 checks on ZCL_PAYMENT_C60; cleans up after itself
ZRESET_PWD_C60	Helper (temporary)	Team	Resets a test password during testing; delete before the final demo

In the system the receipt driver is saved under the team's existing program name ending in 103; ZDRIVER_FEE_RECEIPT_C60 is the name of its source in Project K.

10. Business rules and validations

Every rule lives in one class method, so it applies the same way on every screen. The messages below are the exact texts the user sees.

Payments

Rule	Enforced in	Message
Amount must be positive	VALIDATE	Amount must be greater than zero
A fee row must exist for student, year and fee type	VALIDATE	Fee record not found for this student and fee type
No overpayment	VALIDATE	Amount 999999999.00 exceeds balance 41000.00
No duplicate reference (rejected payments do not count, so they can be resubmitted)	VALIDATE	Transaction ID E2E-TEST-001 already exists
Student must give a reference and a bank	ZSTU_PAYMENT_C60	Enter the UTR / reference number of your payment; Enter the bank name
DD and cheque need a number and a bank	ZACC_COUNTER_C60	Enter the DD / cheque number; Enter the bank name
Only SUBMITTED payments can be verified or rejected	VERIFY, REJECT	Payment is VERIFIED. Only SUBMITTED payments can be verified.
Balance is re-checked at approval (two pending payments may together exceed it)	VERIFY	Amount ... now exceeds balance ... Reject this payment.
A rejection needs a reason	REJECT	A remark is required to reject a payment
Balance changes only at verification, never at submission	SUBMIT, VERIFY	-
Receipt number only at verification, never reused	VERIFY, ZNR_RCPT_C	Payment verified. Receipt: FR/2026/000011

Login and roles

Rule	Message
Wrong user or password (same text for both)	Invalid user ID or password
Third wrong password	Too many attempts. Account locked
Locked user	Account locked. Contact the admin
Accountant not yet approved	Account awaiting admin approval
Password rule	Password needs 8+ characters with a letter and a digit
Student must be in the student master	Register number not found in student records
Admins cannot self-register	Admin accounts cannot be self-registered
Wrong security answer	User ID or security answer is incorrect
OTP older than 10 minutes / wrong OTP	OTP expired. Request a new one / Incorrect OTP
Wrong role on a screen	Only accountants can verify payments (and similar per screen)
Student asks for someone else's fees	Students can view only their own fees

Exception	Amount	Payment Mo	Status	
9 06.10.2026 Tuition Fee 1.000,00 UPI	E2E-TEST-001	VERIFIED	FR/2026/000011	
8 05.10.2026 Tuition Fee 500,00 UPI	TESTUPI004	REJECTED		UTR NOT FOUND IN BANK STATEMENT
7 05.10.2026 Tuition Fee 2.000,00 CASH		VERIFIED	FR/2026/000009	

Figure 27. The overpayment rule in action: 999,999,999.00 is blocked against a balance of 41,000.00.

11. Testing

All 11 end-to-end tests on 06.10.2026 passed, after module tests on every class and screen. Each layer was tested by a script before a screen was built on top of it.

Module tests

Suite	Covers	Result
ZTEST_AUTH_C60 (9 cases)	Registration valid and invalid, accountant blocked until approved, admin self-registration blocked, correct login, lockout after 3 wrong passwords, login while locked	Pass
ZTEST_AUTH2_C60 (8 cases)	Wrong security answer, OTP issued, wrong OTP, weak new password, successful reset, login with new password, old password rejected, OTP reuse blocked	Pass
ZFEE_APP_C60 and ZADMIN_C60 (10 cases)	The same flows through the real screens, including approve and unlock	Pass
ZFEE_DEMAND_GEN_C60 (3 runs)	Preview; real run created 100 rows; repeat run created 0 and skipped 100	Pass
ZSTU_FEE_VIEW_C60	Y26CSE001 shows its 2 fee rows with correct status and balance	Pass
ZTEST_PAYMENT_C60 (16 checks)	Submit, duplicate reference, overpayment, zero amount, verify with receipt and balance, receipt row, double verify blocked, reject without and with remark, verify after reject blocked, resubmit after reject, second receipt unique, partial payments add up, audit lines	Pass, 16 of 16

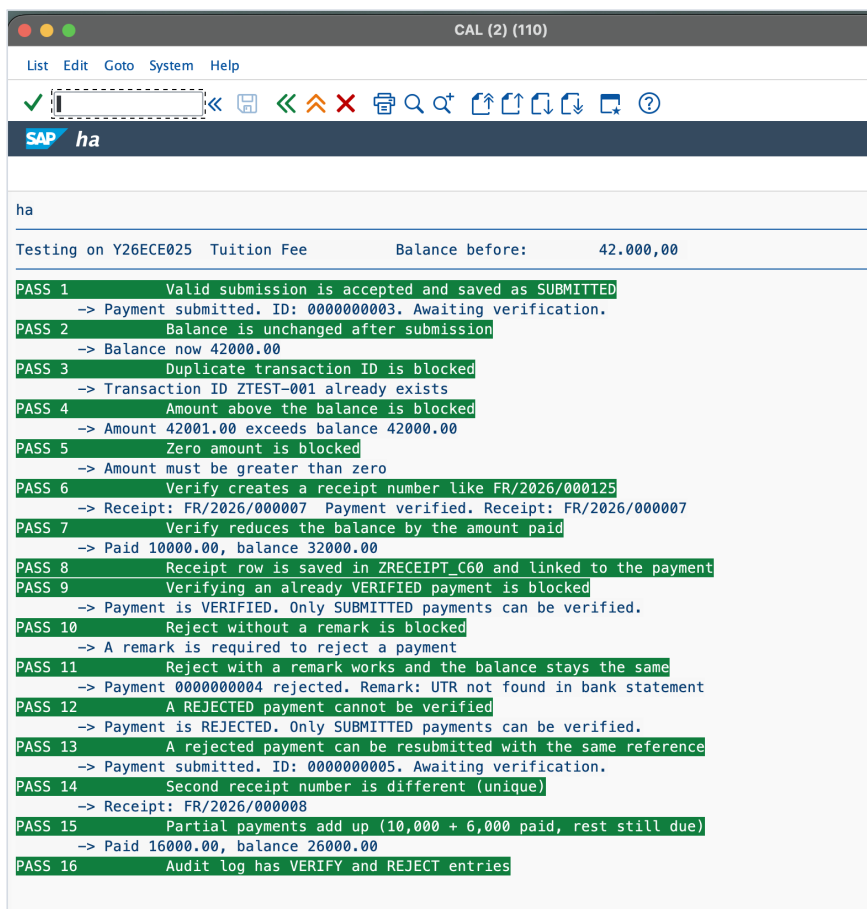


Figure 28. ZTEST_PAYMENT_C60: all 16 checks PASS on test student Y26ECE025.

End-to-end tests (06.10.2026)

Test student Y26CSE001, Tuition Fee year 1 (45,000.00 INR); test accountant ACC001.

No.	Test	Expected	Result
1	Student submits 1,000.00 by UPI, reference E2E-TEST-001	Saved as SUBMITTED	Pass: payment 0000000009
2	Accountant approves payment 9	VERIFIED, receipt issued, balance reduced	Pass: receipt FR/2026/000011
3	Dashboard after approval	Collected 151,000.00; balance 3,874,000.00; 3.75 %; today 1,000.00; awaiting 0	Pass
4	Print FR/2026/000011, then check the audit log	Paid so far 4,000.00, balance 41,000.00; VERIFY and PRINT lines	Pass: audit lines 9 and 10
5a	Amount 999,999,999	Blocked	Pass: "exceeds balance 41000.00"
5b	Reference E2E-TEST-001 again	Blocked	Pass: "already exists"
5c	Reject without remark, then with "Test rejection"	First blocked, then REJECTED	Pass: payment 0000000010 rejected
6a	Audit log after the rejection	REJECT line by ACC001	Pass: line 11
6b	Student's own view	Rejected payments red with reason, verified ones green with receipt	Pass
6c	Dashboard after the rejection	No change	Pass
7	OData service in Chrome	Service document and JSON rows	Pass: payments 6 to 10 returned

Audit times are stored in UTC and shown in the SAP user's time zone, so they can differ from the wall clock if that zone is not set to India.

12. Team and plan

Seven members, one owner per module. The database and the login came first, because everything else depends on them. Fill in the member names before submission.

#	Role	Owned	Name
M1	Team lead / integrator	Package, transport, naming rules, integration, dashboard, demo	
M2	Database	Tables, domains, data elements, foreign keys, number ranges, test data; later CDS view	
M3	Authentication logic	ZCL_AUTH_CS60: register, login, hashing, lockout, OTP reset; security tests	
M4	Authentication screens	ZFEE_APP_C60, ZADMIN_C60; later documentation and presentation	
M5	Fee management	ZFEE_STRUCT_MAINT_C60, ZFEE_DEMAND_GEN_C60, ZSTU_FEE_VIEW_C60	
M6	Payments	ZCL_PAYMENT_C60, student submission, counter entry, verification, audit log	
M7	Receipt and reports	Smart Form, receipt driver and PDF, CDS view, ALV reports	

Seven-day plan as executed

Day	Focus	Steps	Result
1	Foundation: package, tables, number ranges, test data	1 to 4	Done
2	Authentication: class, login screens, admin approval	5 to 7	Done
3	Fee management: price list, demand generation, student fee view	8 to 10	Done
4	Payments: class, student submission, counter entry, verification	11 to 14	Done
5	Receipt Smart Form and PDF, CDS view, four ALV reports	15 to 18	Done
6	Dashboard, audit log viewer, print and PDF logging	19 to 20	Done
7	End-to-end testing, OData service, documentation	21 to 23	Tests done; demo cleanup open

13. How to run and demo

Every program starts from SE38 (type /nSE38, the program name, then F8), except the dashboard, which has its own transaction. Programs with a login block ask for the app user ID and password first.

Student

1. Register or reset a password: run ZFEE_APP_C60.
2. See fees: run ZSTU_FEE_VIEW_C60 and log in with the register number.
3. Pay: run ZSTU_PAYMENT_C60, log in, fill fee type, year, amount, UTR / reference, bank and mode, press F8. The message shows the new payment ID.
4. Check status: run ZSTU_PAYMENT_C60 again with the amount empty. Green = verified (with receipt number), yellow = waiting, red = rejected (with reason).

Accountant

1. Once per term: maintain prices in ZFEE_STRUCT_MAINT_C60, run ZFEE_DEMAND_GEN_C60 in test mode, check the preview, untick Test run and run it again.

2. Counter payment: ZACC_COUNTER_C60, log in, enter the register number, amount and mode; keep "Verify now" ticked to issue the receipt at once.
3. Online payments: ZACC_VERIFY_C60, log in, double-click a payment, compare with the bank statement, choose Approve or Reject (reason required).
4. Receipt: run the receipt driver, enter the receipt number, F8 for preview or tick "Save as PDF file".
5. Management view: transaction /nZFEEDASH, then the buttons for the four reports.

Admin

1. Approve new accountants and unlock users: ZADMIN_C60.
2. Review actions: ZRPT_AUDIT_C60, filtered by user, action or date if needed.

Demo script (about 10 minutes)

1. Open ZFEEDASH and show the starting figures.
2. Student Y26CSE001 submits a UPI payment in ZSTU_PAYMENT_C60.
3. Show a blocked overpayment and a blocked duplicate reference.
4. Accountant ACC001 approves it in ZACC_VERIFY_C60; point out the receipt number.
5. Print the receipt and save it as PDF.
6. Back to ZFEEDASH: collected and today's figures have moved; open the Payment Register and Daily Collection.
7. Reject a second payment without a remark (blocked), then with one; show it red in the student's view.
8. Show the audit log lines for all of the above.
9. Open the OData JSON address in a browser to show the same data on the web.

Demo users: student Y26CSE001, accountant ACC001, admin ADM001. Passwords are kept by the team lead and are not written in this document.

14. Reviewer questions and answers

Questions a reviewer is likely to ask, with short answers any team member can give.

Q. Why is there no payment gateway?

Many colleges collect fees through their bank account, by UPI, NEFT, DD, cheque or cash. The system records those payments and makes an accountant confirm each one against the bank statement before a receipt exists. A gateway could later call the same VERIFY method automatically.

Q. How do you make sure a receipt number is never duplicated?

Receipt numbers come from the SAP number range object ZNR_RCPT_C (no rolling), drawn only inside VERIFY. Number ranges are locked by SAP, so two accountants approving at the same moment still get different numbers.

Q. What if the system fails halfway through a verification?

VERIFY writes the balance, the payment status, the receipt row and the audit line in one logical unit of work. Any failure calls ROLLBACK WORK, so either all four are saved or none is.

Q. Can a student see another student's data?

No. The register number used in every student query is taken from the login, never typed on the screen, and ZSTU_FEE_VIEW_C60 refuses a different register number for a student role.

Q. How are passwords stored?

As SHA-256 hashes of a per-user random 32-character salt plus the password. The same password gives a different hash for every user, and nobody can read a password back, not even an admin.

Q. How do you stop the same bank payment being claimed twice?

VALIDATE blocks a reference (UTR, DD or cheque number) that is already used by a payment that is not REJECTED, and the accountant still checks the bank statement before approving.

Q. What if two pending payments together exceed the balance?

Each passes VALIDATE on its own, so VERIFY checks the amount again against the balance as it is at approval time and refuses the second one.

Q. Why did you use a CDS view?

It defines the joined data model once (payment, student, fee demand, receipt) and runs it in the database. The same view feeds the Payment Register and the OData service.

Q. Why not RAP?

RAP business objects need SAP_BASIS 754 (S/4HANA 1909) or newer; our system is 753 (1809). We used the release-appropriate option, a CDS view published as an OData service.

Q. Why selection screens and not module pools?

The graphical Screen Painter is not available in SAP GUI for Java on the Mac. Selection screens still give login blocks, radio buttons, push buttons and hidden password fields.

Q. How is the balance calculated?

Balance = Total fee - Concession + Late fee - Paid amount, kept per fee row and changed only by VERIFY.

Q. What is in the audit log, and who uses it?

Each approval, rejection, unlock, verification, print and PDF, with user, action, object and time. The admin reviews it in ZRPT_AUDIT_C60.

15. Limitations, open items and future work

The system meets every must-have requirement; the points below are known gaps and next steps.

Known limitations

- The Payment Register total includes rejected payments. Filter Status = VERIFIED to see money actually collected.
- Remarks are stored in capitals, because the remark domain does not allow lowercase.
- Seven legacy fee rows from the first version have no due date, so they never count as overdue, and some use other spellings of fee types (TUITION, HOSTEL FEE).
- Reports and the dashboard rely on SAP authorization, not the app login.
- The OTP is shown on screen, as no mail server (SAPconnect) is configured.
- The receipt table has a CANCELLED flag, but there is no cancel-and-reissue program yet.
- Full RAP is not possible on SAP_BASIS 753 (section 8.3).

Before the final demo

- Delete the helper program ZRESET_PWD_C60.
- Align the legacy fee-type spellings and fill their due dates, so the Defaulters report shows real cases.
- Optional: short transaction codes for the reports (for example ZPAYREG, ZDEFAULT, ZDAYCOLL, ZFEETYPE, ZAUDIT).
- Rehearse the demo script in section 13.

Future work

- Bank-statement reconciliation: upload the bank file and match SUBMITTED payments by reference and amount.
- Automatic late fee after the due date, and concession handling.
- E-mail or SMS for the OTP and the receipt PDF.
- Receipt cancellation and reissue with a reversal entry.
- Formal AUTHORITY-CHECK objects in addition to the app's role checks.
- A Fiori app on top of the OData service, and a RAP business object with Approve and Reject actions once the system is on S/4HANA 1909 or newer.

Appendix A. Glossary

Term	Meaning in this project
ABAP	SAP's programming language; every program and class here is written in it
SE11 / SE38 / SE93	Transactions for tables (Dictionary), programs (ABAP Editor) and transaction codes
ALV	SAP List Viewer: the interactive grid used by all reports and lists
CDS view	Core Data Services: a database view defined in code (here ZCDS_FEE_PAY)
OData	A web standard for reading data over HTTP; our service returns payments as XML or JSON
SAP Gateway / SICF	The SAP layer that publishes OData services, and the transaction that switches web nodes on
Smart Form	SAP's form designer, used for the printed receipt
Number range (SNRO)	An SAP object that hands out unique sequential numbers
LUW (unit of work)	A group of database changes saved together by COMMIT WORK or undone together by ROLLBACK WORK
UTR	Unique Transaction Reference of a bank transfer or UPI payment
RAP	ABAP RESTful Application Programming Model, available from SAP_BASIS 754
Package / transport	Where objects are stored (ZVYSHU) and how they move between systems

Appendix B. Source code

The full source of the objects whose code is kept in Project K, exactly as activated in the system. Line numbers help when discussing a line in review.

Not reproduced here: ZFEE_APP_C60, ZADMIN_C60, ZFEE_STRUCT_MAINT_C60, the helper and seed programs, the Smart Form ZSF_RECEIPT_C60 and the dictionary objects. They are in package ZVYSHU in the system.

B.1 ZCL_AUTH_CS60

Class: login, registration, logout and password reset · 367 lines

```

1  CLASS zcl_auth_cs60 DEFINITION PUBLIC FINAL CREATE PUBLIC.
2  PUBLIC SECTION.
3      TYPES: BEGIN OF ty_result,
4              ok          TYPE abap_bool,
5              message TYPE string,
6              role        TYPE zusers_c60-role,
7          END OF ty_result.
8
9      TYPES: BEGIN OF ty_otp_result,
10             ok          TYPE abap_bool,
11             message TYPE string,
12             otp          TYPE c LENGTH 6,
13         END OF ty_otp_result.
14
15     CLASS-METHODS:
16         register IMPORTING iv_user_id TYPE zusers_c60-user_id
17                 iv_role      TYPE zusers_c60-role
18                 iv_name      TYPE zusers_c60-name
19                 iv_email     TYPE zusers_c60-email
20                 iv_phone     TYPE zusers_c60-phone
21                 iv_password  TYPE string
22                 iv_question  TYPE zusers_c60-sec_question
23                 iv_answer    TYPE string
24                 iv_seed      TYPE abap_bool DEFAULT abap_false
25             RETURNING VALUE(rs_result) TYPE ty_result,
26
27         login    IMPORTING iv_user_id TYPE zusers_c60-user_id
28                 iv_password TYPE string
29             RETURNING VALUE(rs_result) TYPE ty_result,
30
31         request_reset IMPORTING iv_user_id TYPE zusers_c60-user_id
32                 iv_answer TYPE string
33             RETURNING VALUE(rs_result) TYPE ty_otp_result,
34
35         reset_password IMPORTING iv_user_id TYPE zusers_c60-user_id
36                 iv_otp TYPE string
37                 iv_new_password TYPE string
38             RETURNING VALUE(rs_result) TYPE ty_result.
39
40     PRIVATE SECTION.
41     CLASS-METHODS hash IMPORTING iv_salt TYPE string
42                     iv_text TYPE string
43             RETURNING VALUE(rv_hash) TYPE string.
44 ENDCLASS.
45
46
47
48 CLASS ZCL_AUTH_CS60 IMPLEMENTATION.
49
50
51 * <SIGNATURE>-----+
52 * | Static Private Method ZCL_AUTH_CS60=>HASH
53 * +-----+
54 * | [--->] IV_SALT                TYPE      STRING
55 * | [--->] IV_TEXT                TYPE      STRING
56 * | [<-()] RV_HASH                TYPE      STRING
57 * +-----+</SIGNATURE>
58 METHOD hash.
59     DATA lv_hash TYPE string.
60     TRY.
61         cl_abap_message_digest=>calculate_hash_for_char(
62             EXPORTING if_algorithm = 'SHA256'
63             if_data      = |{ iv_salt }{ iv_text }|
64             IMPORTING ef_hashstring = lv_hash ).
65         CATCH cx_abap_message_digest.
66         CLEAR lv_hash.
67     ENDTRY.

```

```

68     rv_hash = lv_hash.
69 ENDMETHOD.
70
71
72 * <SIGNATURE>-----+
73 * | Static Public Method ZCL_AUTH_CS60=>LOGIN
74 * +-----+
75 * | [--->] IV_USER_ID                TYPE      ZUSERS_C60-USER_ID
76 * | [--->] IV_PASSWORD                TYPE      STRING
77 * | [<-()] RS_RESULT                  TYPE      TY_RESULT
78 * +-----</SIGNATURE>
79 METHOD login.
80     DATA lv_ts TYPE timestamp.
81
82     SELECT SINGLE * FROM zusers_c60
83         WHERE user_id = @iv_user_id
84         INTO @DATA(ls_user).
85     IF sy-subrc <> 0.
86         rs_result-message = 'Invalid user ID or password'.    " same text on purpose
87         RETURN.
88     ENDIF.
89
90     CASE ls_user-status.
91     WHEN 'PENDING'.
92         rs_result-message = 'Account awaiting admin approval'.
93         RETURN.
94     WHEN 'LOCKED'.
95         rs_result-message = 'Account locked. Contact the admin'.
96         RETURN.
97     WHEN 'REJECTED'.
98         rs_result-message = 'Registration was rejected'.
99         RETURN.
100    ENDCASE.
101
102    IF hash( iv_salt = |{ ls_user-salt }| iv_text = iv_password ) = ls_user-pwd_hash.
103        GET TIME STAMP FIELD lv_ts.
104        UPDATE zusers_c60
105            SET failed_attempts = 0,
106                last_login      = @lv_ts
107            WHERE user_id = @iv_user_id.
108        COMMIT WORK.
109        rs_result-ok           = abap_true.
110        rs_result-role         = ls_user-role.
111        rs_result-message = 'Login successful'.
112    ELSE.
113        DATA(lv_fail) = ls_user-failed_attempts + 1.
114        DATA(lv_status) = COND zusers_c60-status(
115            WHEN lv_fail >= 3 THEN 'LOCKED' ELSE ls_user-status ).
116        UPDATE zusers_c60
117            SET failed_attempts = @lv_fail,
118                status         = @lv_status
119            WHERE user_id = @iv_user_id.
120        COMMIT WORK.
121        rs_result-message = COND #( WHEN lv_fail >= 3
122            THEN 'Too many attempts. Account locked'
123            ELSE 'Invalid user ID or password' ).
124    ENDIF.
125 ENDMETHOD.
126
127
128 * <SIGNATURE>-----+
129 * | Static Public Method ZCL_AUTH_CS60=>REGISTER
130 * +-----+
131 * | [--->] IV_USER_ID                TYPE      ZUSERS_C60-USER_ID
132 * | [--->] IV_ROLE                    TYPE      ZUSERS_C60-ROLE
133 * | [--->] IV_NAME                    TYPE      ZUSERS_C60-NAME
134 * | [--->] IV_EMAIL                  TYPE      ZUSERS_C60-EMAIL
135 * | [--->] IV_PHONE                  TYPE      ZUSERS_C60-PHONE
136 * | [--->] IV_PASSWORD                TYPE      STRING
137 * | [--->] IV_QUESTION                TYPE      ZUSERS_C60-SEC_QUESTION
138 * | [--->] IV_ANSWER                  TYPE      STRING
139 * | [--->] IV_SEED                    TYPE      ABAP_BOOL (default =ABAP_FALSE)
140 * | [<-()] RS_RESULT                  TYPE      TY_RESULT
141 * +-----</SIGNATURE>
142 METHOD register.
143     " 0. Only the seed program may create admins
144     IF iv_role = 'X' AND iv_seed = abap_false.
145         rs_result-message = 'Admin accounts cannot be self-registered'.
146         RETURN.
147     ENDIF.
148     IF iv_role <> 'S' AND iv_role <> 'A' AND iv_role <> 'X'.
149         rs_result-message = 'Invalid role'.
150         RETURN.
151     ENDIF.

```

```

152
153 " 1. Required fields
154 IF iv_user_id IS INITIAL OR iv_name IS INITIAL OR iv_answer IS INITIAL.
155     rs_result-message = 'Please fill all required fields'.
156     RETURN.
157 ENDIF.
158
159 " 2. Password rule: 8+ characters, one letter, one digit
160 IF strlen( iv_password ) < 8
161     OR NOT matches( val = iv_password regex = '.*[A-Za-z].*' )
162     OR NOT matches( val = iv_password regex = '.*[0-9].*' ).
163     rs_result-message = 'Password needs 8+ characters with a letter and a digit'.
164     RETURN.
165 ENDIF.
166
167 " 3. Duplicate user
168 SELECT SINGLE user_id FROM zusers_c60
169     WHERE user_id = @iv_user_id
170     INTO @DATA(lv_dummy).
171 IF sy-subrc = 0.
172     rs_result-message = 'User already registered'.
173     RETURN.
174 ENDIF.
175
176 " 4. Students must exist in the student master
177 IF iv_role = 'S'.
178     SELECT SINGLE reg_no FROM zstudent_cs60
179         WHERE reg_no = @iv_user_id
180         INTO @DATA(lv_reg).
181     IF sy-subrc <> 0.
182         rs_result-message = 'Register number not found in student records'.
183         RETURN.
184     ENDIF.
185 ENDIF.
186
187 " 5. Build and store the user
188 DATA ls_user TYPE zusers_c60.
189 ls_user-user_id = iv_user_id.
190 ls_user-role    = iv_role.
191 ls_user-name    = iv_name.
192 ls_user-email   = iv_email.
193 ls_user-phone   = iv_phone.
194 TRY.
195     ls_user-salt = cl_system_uuid=>create_uuid_c32_static( ).
196     CATCH cx_uuid_error.
197     ls_user-salt = |{ sy-datum }|{ sy-zeit }|{ sy-uname }|.
198 ENDTRY.
199 ls_user-pwd_hash    = hash( iv_salt = |{ ls_user-salt }| iv_text = iv_password ).
200 ls_user-sec_question = iv_question.
201 ls_user-sec_answer_hash = hash( iv_salt = |{ ls_user-salt }|
202                                 iv_text = to_upper( iv_answer ) ).
203 ls_user-status      = COND #( WHEN iv_role = 'A' THEN 'PENDING' ELSE 'ACTIVE' ).
204 ls_user-created_on   = sy-datum.
205
206 INSERT zusers_c60 FROM @ls_user.
207 IF sy-subrc = 0.
208     COMMIT WORK.
209     rs_result-ok = abap_true.
210     rs_result-message = COND #( WHEN iv_role = 'A'
211                                 THEN 'Registered. Wait for admin approval before logging in'
212                                 ELSE 'Registered successfully. You can log in now' ).
213 ELSE.
214     ROLLBACK WORK.
215     rs_result-message = 'Registration failed'.
216 ENDIF.
217 ENDMETHOD.
218
219
220 * <SIGNATURE>-----+
221 * | Static Public Method ZCL_AUTH_CS60=>REQUEST_RESET
222 * +-----+
223 * | [--->] IV_USER_ID                TYPE          ZUSERS_C60-USER_ID
224 * | [--->] IV_ANSWER                 TYPE          STRING
225 * | [<-()] RS_RESULT                 TYPE          TY_OTP_RESULT
226 * +-----+</SIGNATURE>
227 METHOD request_reset.
228     DATA: lv_otp TYPE n LENGTH 6,
229           lv_now TYPE timestamp,
230           lv_exp TYPE timestamp,
231           lv_max TYPE n LENGTH 6,
232           lv_req TYPE n LENGTH 6,
233           ls_rst TYPE zpwd_reset_c60.
234
235     " 1. User and security answer must match

```

```

236 SELECT SINGLE * FROM zusers_c60
237 WHERE user_id = @iv_user_id
238 INTO @DATA(ls_user).
239 IF sy-subrc <> 0
240     OR hash( iv_salt = |{ ls_user-salt }| iv_text = to_upper( iv_answer ) )
241     <> ls_user-sec_answer_hash.
242     rs_result-message = 'User ID or security answer is incorrect'.
243     RETURN.
244 ENDIF.
245
246 IF ls_user-status = 'PENDING' OR ls_user-status = 'REJECTED'.
247     rs_result-message = 'This account is not active yet'.
248     RETURN.
249 ENDIF.
250
251 " 2. Generate a 6-digit OTP
252 TRY.
253     lv_otp = cl_abap_random_int=>create( seed = cl_abap_random=>seed( )
254                                         min  = 100000
255                                         max  = 999999 )->get_next( ).
256 CATCH cx_root.
257     rs_result-message = 'Could not generate OTP'.
258     RETURN.
259 ENDTRY.
260
261 " 3. Expiry = now + 10 minutes
262 GET TIME STAMP FIELD lv_now.
263 TRY.
264     lv_exp = cl_abap_tstmp=>add( tstmp = lv_now secs = 600 ).
265 CATCH cx_root.
266     rs_result-message = 'Could not set OTP expiry'.
267     RETURN.
268 ENDTRY.
269
270 " 4. Cancel older unused requests, then store the new one
271 UPDATE zpwd_reset_c60 SET used = 'X'
272 WHERE user_id = @iv_user_id AND used = @space.
273
274 SELECT MAX( req_id ) FROM zpwd_reset_c60
275 WHERE user_id = @iv_user_id
276 INTO @lv_max.
277 lv_req = lv_max + 1.
278
279 ls_rst-user_id   = iv_user_id.
280 ls_rst-req_id   = lv_req.
281 ls_rst-otp_hash = hash( iv_salt = |{ ls_user-salt }| iv_text = |{ lv_otp }| ).
282 ls_rst-expires_at = lv_exp.
283 ls_rst-used     = space.
284 INSERT zpwd_reset_c60 FROM @ls_rst.
285 IF sy-subrc <> 0.
286     ROLLBACK WORK.
287     rs_result-message = 'Could not create reset request'.
288     RETURN.
289 ENDIF.
290 COMMIT WORK.
291
292 " 5. Return the OTP (demo: shown on screen; production: email or SMS)
293 rs_result-ok      = abap_true.
294 rs_result-otp     = lv_otp.
295 rs_result-message = 'OTP generated. It is valid for 10 minutes'.
296 ENDMETHOD.
297
298
299 * <SIGNATURE>-----+
300 * | Static Public Method ZCL_AUTH_CS60=>RESET_PASSWORD
301 * +-----+
302 * | [--->] IV_USER_ID                TYPE      ZUSERS_C60-USER_ID
303 * | [--->] IV_OTP                    TYPE      STRING
304 * | [--->] IV_NEW_PASSWORD           TYPE      STRING
305 * | [-( )] RS_RESULT                 TYPE      TY_RESULT
306 * +-----+</SIGNATURE>
307 METHOD reset_password.
308     DATA lv_now TYPE timestamp.
309
310     " 1. Password rule
311     IF strlen( iv_new_password ) < 8
312         OR NOT matches( val = iv_new_password regex = '.*[A-Za-z].*' )
313         OR NOT matches( val = iv_new_password regex = '.*[0-9].*' ).
314         rs_result-message = 'Password needs 8+ characters with a letter and a digit'.
315         RETURN.
316     ENDIF.
317
318     " 2. User must exist
319     SELECT SINGLE * FROM zusers_c60

```

```

320     WHERE user_id = @iv_user_id
321     INTO @DATA(ls_user).
322 IF sy-subrc <> 0.
323     rs_result-message = 'Invalid request'.
324     RETURN.
325 ENDIF.
326
327 " 3. Latest unused reset request
328 SELECT * FROM zpwd_reset_c60
329     WHERE user_id = @iv_user_id AND used = @space
330     ORDER BY req_id DESCENDING
331     INTO TABLE @DATA(lt_rst)
332     UP TO 1 ROWS.
333 READ TABLE lt_rst INTO DATA(ls_rst) INDEX 1.
334 IF sy-subrc <> 0.
335     rs_result-message = 'No active reset request. Request a new OTP'.
336     RETURN.
337 ENDIF.
338
339 " 4. Expiry and OTP check
340 GET TIME STAMP FIELD lv_now.
341 IF lv_now > ls_rst-expires_at.
342     rs_result-message = 'OTP expired. Request a new one'.
343     RETURN.
344 ENDIF.
345
346 IF hash( iv_salt = |{ ls_user-salt }| iv_text = iv_otp ) <> ls_rst-otp_hash.
347     rs_result-message = 'Incorrect OTP'.
348     RETURN.
349 ENDIF.
350
351 " 5. Save the new password (same salt) and unlock the account
352 DATA(lv_newhash) = hash( iv_salt = |{ ls_user-salt }| iv_text = iv_new_password ).
353 UPDATE zusers_c60
354     SET pwd_hash      = @lv_newhash,
355         failed_attempts = 0,
356         status        = 'ACTIVE'
357     WHERE user_id = @iv_user_id.
358
359 UPDATE zpwd_reset_c60 SET used = 'X'
360     WHERE user_id = @iv_user_id AND req_id = @ls_rst-req_id.
361 COMMIT WORK.
362
363 rs_result-ok      = abap_true.
364 rs_result-role    = ls_user-role.
365 rs_result-message = 'Password changed. You can log in now'.
366 ENDMETHOD.
367 ENDClass.

```

B.2 ZCL_PAYMENT_C60

Class: payment rules (validate, submit, verify, reject) · 349 lines

```

1  CLASS zcl_payment_c60 DEFINITION PUBLIC FINAL CREATE PUBLIC.
2  PUBLIC SECTION.
3      TYPES: BEGIN OF ty_result,
4              ok                TYPE abap_bool,
5              message           TYPE string,
6              payment_id        TYPE n LENGTH 10,
7              receipt_no        TYPE c LENGTH 16,
8          END OF ty_result.
9
10     CLASS-METHODS:
11         validate IMPORTING iv_reg_no    TYPE zpayment_c60-reg_no
12                 iv_fee_type TYPE zpayment_c60-fee_type
13                 iv_acad_year TYPE zpayment_c60-academic_year
14                 iv_amount    TYPE zpayment_c60-amount
15                 iv_txn_id    TYPE zpayment_c60-transaction_id
16                 RETURNING VALUE(rs_result) TYPE ty_result,
17
18         submit    IMPORTING iv_reg_no    TYPE zpayment_c60-reg_no
19                 iv_fee_type TYPE zpayment_c60-fee_type
20                 iv_acad_year TYPE zpayment_c60-academic_year
21                 iv_amount    TYPE zpayment_c60-amount
22                 iv_txn_id    TYPE zpayment_c60-transaction_id
23                 iv_mode      TYPE zpayment_c60-payment_mode
24                 iv_bank      TYPE zpayment_c60-bank_name
25                 iv_entered   TYPE zpayment_c60-entered_by
26                 RETURNING VALUE(rs_result) TYPE ty_result,
27
28         verify    IMPORTING iv_payment_id TYPE zpayment_c60-payment_id
29                 iv_verifier TYPE zpayment_c60-verified_by

```

```

30         RETURNING VALUE(rs_result) TYPE ty_result,
31
32     reject    IMPORTING iv_payment_id TYPE zpayment_c60-payment_id
33                iv_verifier  TYPE zpayment_c60-verified_by
34                iv_remark    TYPE zpayment_c60-remark
35                RETURNING VALUE(rs_result) TYPE ty_result.
36 ENDCLASS.
37
38
39
40 CLASS zcl_payment_c60 IMPLEMENTATION.
41
42
43 * <SIGNATURE>-----+
44 * | Static Public Method ZCL_PAYMENT_C60=>REJECT
45 * +-----+
46 * | [--->] IV_PAYMENT_ID          TYPE          ZPAYMENT_C60-PAYMENT_ID
47 * | [--->] IV_VERIFIER            TYPE          ZPAYMENT_C60-VERIFIED_BY
48 * | [--->] IV_REMARK              TYPE          ZPAYMENT_C60-REMARK
49 * | [<-()] RS_RESULT              TYPE          TY_RESULT
50 * +-----+</SIGNATURE>
51 METHOD reject.
52     " A rejection must always say why
53     IF iv_remark IS INITIAL.
54         rs_result-message = 'A remark is required to reject a payment'.
55         RETURN.
56     ENDIF.
57
58     SELECT SINGLE status FROM zpayment_c60
59         WHERE payment_id = @iv_payment_id
60         INTO @DATA(lv_status).
61     IF sy-subrc <> 0.
62         rs_result-message = 'Payment not found'.
63         RETURN.
64     ENDIF.
65     IF lv_status <> 'SUBMITTED'.
66         rs_result-message =
67             |Payment is { lv_status }. Only SUBMITTED payments can be rejected.|.
68         RETURN.
69     ENDIF.
70
71     UPDATE zpayment_c60
72         SET status      = 'REJECTED',
73             verified_by = @iv_verifier,
74             verified_on = @sy-datum,
75             remark      = @iv_remark
76         WHERE payment_id = @iv_payment_id.
77     IF sy-subrc <> 0.
78         ROLLBACK WORK.
79         rs_result-message = 'Rejection failed'.
80         RETURN.
81     ENDIF.
82
83     DATA ls_log TYPE zaudit_log_c60.
84     SELECT MAX( log_id ) FROM zaudit_log_c60 INTO @DATA(lv_max).
85     ls_log-log_id      = lv_max + 1.
86     ls_log-user_id     = iv_verifier.
87     ls_log-action      = 'REJECT'.
88     ls_log-object_key  = |{ iv_payment_id }|.
89     GET TIME STAMP FIELD ls_log-logged_at.
90     ls_log-remark      = iv_remark.
91     INSERT zaudit_log_c60 FROM @ls_log.
92     IF sy-subrc <> 0.
93         ROLLBACK WORK.
94         rs_result-message = 'Rejection failed (audit log could not be written)'.
95         RETURN.
96     ENDIF.
97     COMMIT WORK.
98
99     rs_result-ok       = abap_true.
100    rs_result-message =
101        |Payment { iv_payment_id } rejected. Remark: { iv_remark }|.
102 ENDMETHOD.
103
104
105 * <SIGNATURE>-----+
106 * | Static Public Method ZCL_PAYMENT_C60=>SUBMIT
107 * +-----+
108 * | [--->] IV_REG_NO              TYPE          ZPAYMENT_C60-REG_NO
109 * | [--->] IV_FEE_TYPE            TYPE          ZPAYMENT_C60-FEE_TYPE
110 * | [--->] IV_ACAD_YEAR          TYPE          ZPAYMENT_C60-ACADEMIC_YEAR
111 * | [--->] IV_AMOUNT             TYPE          ZPAYMENT_C60-AMOUNT
112 * | [--->] IV_TXN_ID             TYPE          ZPAYMENT_C60-TRANSACTION_ID
113 * | [--->] IV_MODE               TYPE          ZPAYMENT_C60-PAYMENT_MODE

```

```

114 * | [--->] IV_BANK                                TYPE      ZPAYMENT_C60-BANK_NAME
115 * | [--->] IV_ENTERED                             TYPE      ZPAYMENT_C60-ENTERED_BY
116 * | [<-()] RS_RESULT                             TYPE      TY_RESULT
117 * +-----<-->-----</SIGNATURE>
118 METHOD submit.
119   DATA ls_pay      TYPE zpayment_c60.
120   DATA lv_pay_id TYPE n LENGTH 10.
121
122   DATA(ls_val) = validate( iv_reg_no   = iv_reg_no
123                           iv_fee_type  = iv_fee_type
124                           iv_acad_year = iv_acad_year
125                           iv_amount   = iv_amount
126                           iv_txn_id   = iv_txn_id ).
127   IF ls_val-ok = abap_false.
128     rs_result-message = ls_val-message.
129     RETURN.
130   ENDIF.
131
132   " Currency comes from the fee row, so reports show amounts correctly
133   SELECT SINGLE currency FROM zfee_details_c60
134     WHERE reg_no       = @iv_reg_no
135     AND academic_year = @iv_acad_year
136     AND fee_type       = @iv_fee_type
137     INTO @DATA(lv_currency).
138
139   CALL FUNCTION 'NUMBER_GET_NEXT'
140     EXPORTING nr_range_nr = '01'
141     object     = 'ZNR_PAYIDC'
142     IMPORTING number      = lv_pay_id
143     EXCEPTIONS OTHERS    = 1.
144   IF sy-subrc <> 0.
145     rs_result-message = 'Could not generate payment ID'.
146     RETURN.
147   ENDIF.
148
149   ls_pay-payment_id   = lv_pay_id.
150   ls_pay-reg_no       = iv_reg_no.
151   ls_pay-fee_type     = iv_fee_type.
152   ls_pay-academic_year = iv_acad_year.
153   ls_pay-transaction_id = iv_txn_id.
154   ls_pay-currency     = lv_currency.
155   ls_pay-amount       = iv_amount.
156   ls_pay-payment_mode = iv_mode.
157   ls_pay-bank_name    = iv_bank.
158   ls_pay-payment_date = sy-datum.
159   ls_pay-status       = 'SUBMITTED'.
160   ls_pay-entered_by   = iv_entered.
161
162   INSERT zpayment_c60 FROM @ls_pay.
163   IF sy-subrc <> 0.
164     ROLLBACK WORK.
165     rs_result-message = 'Payment submission failed'.
166     RETURN.
167   ENDIF.
168   COMMIT WORK.
169
170   rs_result-ok          = abap_true.
171   rs_result-payment_id = lv_pay_id.
172   rs_result-message     =
173     |Payment submitted. ID: { lv_pay_id }. Awaiting verification.|.
174   ENDMETHOD.
175
176
177 * <SIGNATURE>-----<-->-----+
178 * | Static Public Method ZCL_PAYMENT_C60=>VALIDATE
179 * +-----<-->-----+
180 * | [--->] IV_REG_NO                                TYPE      ZPAYMENT_C60-REG_NO
181 * | [--->] IV_FEE_TYPE                             TYPE      ZPAYMENT_C60-FEE_TYPE
182 * | [--->] IV_ACAD_YEAR                             TYPE      ZPAYMENT_C60-ACADEMIC_YEAR
183 * | [--->] IV_AMOUNT                                TYPE      ZPAYMENT_C60-AMOUNT
184 * | [--->] IV_TXN_ID                                TYPE      ZPAYMENT_C60-TRANSACTION_ID
185 * | [<-()] RS_RESULT                             TYPE      TY_RESULT
186 * +-----<-->-----</SIGNATURE>
187 METHOD validate.
188   IF iv_amount <= 0.
189     rs_result-message = 'Amount must be greater than zero'.
190     RETURN.
191   ENDIF.
192
193   SELECT SINGLE balance, currency
194     FROM zfee_details_c60
195     WHERE reg_no       = @iv_reg_no
196     AND academic_year = @iv_acad_year
197     AND fee_type       = @iv_fee_type

```

```

198     INTO @DATA(ls_fee).
199     IF sy-subrc <> 0.
200         rs_result-message = 'Fee record not found for this student and fee type'.
201         RETURN.
202     ENDIF.
203
204     IF iv_amount > ls_fee-balance.
205         rs_result-message =
206             |Amount { iv_amount } exceeds balance { ls_fee-balance }|.
207         RETURN.
208     ENDIF.
209
210     " Duplicate UTR check. A REJECTED payment does not count, so the
211     " student can resubmit the same reference after a rejection.
212     IF iv_txn_id IS NOT INITIAL.
213         SELECT SINGLE payment_id FROM zpayment_c60
214             WHERE transaction_id = @iv_txn_id
215             AND status <> 'REJECTED'
216             INTO @DATA(lv_dup).
217         IF sy-subrc = 0.
218             rs_result-message =
219                 |Transaction ID { iv_txn_id } already exists|.
220             RETURN.
221         ENDIF.
222     ENDIF.
223
224     rs_result-ok = abap_true.
225     rs_result-message = 'Validation passed'.
226     ENDMETHOD.
227
228
229 * <SIGNATURE>-----+
230 * | Static Public Method ZCL_PAYMENT_C60=>VERIFY
231 * +-----+
232 * | [--->] IV_PAYMENT_ID                TYPE          ZPAYMENT_C60-PAYMENT_ID
233 * | [--->] IV_VERIFIER                  TYPE          ZPAYMENT_C60-VERIFIED_BY
234 * | [<-()] RS_RESULT                    TYPE          TY_RESULT
235 * +-----+-----</SIGNATURE>
236 METHOD verify.
237     DATA: ls_pay TYPE zpayment_c60,
238           ls_rcpt TYPE zreceipt_c60,
239           lv_nr TYPE n LENGTH 6,
240           lv_rcpt TYPE c LENGTH 16,
241           lv_paid TYPE zfee_details_c60-paid_amount,
242           lv_bal TYPE zfee_details_c60-balance.
243
244     SELECT SINGLE * FROM zpayment_c60
245         WHERE payment_id = @iv_payment_id
246         INTO @ls_pay.
247     IF sy-subrc <> 0.
248         rs_result-message = 'Payment not found'.
249         RETURN.
250     ENDIF.
251     IF ls_pay-status <> 'SUBMITTED'.
252         rs_result-message =
253             |Payment is { ls_pay-status }. Only SUBMITTED payments can be verified|.
254         RETURN.
255     ENDIF.
256
257     " Read the fee row first. Check it BEFORE using a receipt number.
258     SELECT SINGLE paid_amount, balance FROM zfee_details_c60
259         WHERE reg_no = @ls_pay-reg_no
260             AND academic_year = @ls_pay-academic_year
261             AND fee_type = @ls_pay-fee_type
262         INTO @DATA(ls_fee).
263     IF sy-subrc <> 0.
264         rs_result-message = 'Fee record not found for this payment'.
265         RETURN.
266     ENDIF.
267
268     " Two payments can be SUBMITTED at the same time, each one smaller than
269     " the balance. Re-check against the balance as it is right now.
270     IF ls_pay-amount > ls_fee-balance.
271         rs_result-message =
272             |Amount { ls_pay-amount } now exceeds balance { ls_fee-balance }. Reject this payment|.
273         RETURN.
274     ENDIF.
275
276     CALL FUNCTION 'NUMBER_GET_NEXT'
277         EXPORTING nr_range_nr = '01'
278         object = 'ZNR_RCPT_C'
279         IMPORTING number = lv_nr
280         EXCEPTIONS OTHERS = 1.
281     IF sy-subrc <> 0.

```

```

282     rs_result-message = 'Could not generate receipt number'.
283     RETURN.
284 ENDIF.
285 lv_rcpt = |FR/{ sy-datum(4) }/{ lv_nr }|.
286
287 lv_paid = ls_fee-paid_amount + ls_pay-amount.
288 lv_bal  = ls_fee-balance      - ls_pay-amount.
289
290 " From here on everything is ONE unit: all saved, or all undone.
291 UPDATE zfee_details_c60
292     SET paid_amount = @lv_paid,
293         balance     = @lv_bal
294     WHERE reg_no     = @ls_pay-reg_no
295         AND academic_year = @ls_pay-academic_year
296         AND fee_type     = @ls_pay-fee_type.
297 IF sy-subrc <> 0.
298     ROLLBACK WORK.
299     rs_result-message = 'Balance update failed'.
300     RETURN.
301 ENDIF.
302
303 UPDATE zpayment_c60
304     SET status      = 'VERIFIED',
305         receipt_no   = @lv_rcpt,
306         verified_by  = @iv_verifier,
307         verified_on   = @sy-datum
308     WHERE payment_id = @iv_payment_id.
309 IF sy-subrc <> 0.
310     ROLLBACK WORK.
311     rs_result-message = 'Payment status update failed'.
312     RETURN.
313 ENDIF.
314
315 ls_rcpt-receipt_no = lv_rcpt.
316 ls_rcpt-pay_id     = iv_payment_id.
317 ls_rcpt-issued_on  = sy-datum.
318 ls_rcpt-issued_by  = iv_verifier.
319 ls_rcpt-print_count = 0.
320 ls_rcpt-cancelled   = space.
321 INSERT zreceipt_c60 FROM @ls_rcpt.
322 IF sy-subrc <> 0.
323     ROLLBACK WORK.
324     rs_result-message = 'Receipt could not be saved'.
325     RETURN.
326 ENDIF.
327
328 DATA ls_log TYPE zaudit_log_c60.
329 SELECT MAX( log_id ) FROM zaudit_log_c60 INTO @DATA(lv_max).
330 ls_log-log_id      = lv_max + 1.
331 ls_log-user_id     = iv_verifier.
332 ls_log-action      = 'VERIFY'.
333 ls_log-object_key  = lv_rcpt.
334 GET TIME STAMP FIELD ls_log-logged_at.
335 ls_log-remark      = |Payment { iv_payment_id } verified|.
336 INSERT zaudit_log_c60 FROM @ls_log.
337 IF sy-subrc <> 0.
338     ROLLBACK WORK.
339     rs_result-message = 'Audit log could not be written'.
340     RETURN.
341 ENDIF.
342
343 COMMIT WORK.
344
345 rs_result-ok       = abap_true.
346 rs_result-receipt_no = lv_rcpt.
347 rs_result-message  = |Payment verified. Receipt: { lv_rcpt }|.
348 ENDMETHOD.
349 ENDCLASS.

```

B.3 ZFEE_DEMAND_GEN_C60

Program: fee demand generation · 235 lines

```

1  *&-----*
2  *& Report ZFEE_DEMAND_GEN_C60
3  *&-----*
4  *& Step 9 - Fee demand generation (Accountant)
5  *&
6  *& Reads the fee price list (ZFEE_STRUCT_CS60) and creates one row in
7  *& ZFEE_DETAILS_C60 per student per fee type.
8  *&
9  *& - Test run (default) shows a preview and saves nothing.
10 *& - Rows that already exist are skipped and never overwritten, so the
11 *&   program can be run again safely (idempotent) and paid amounts are
12 *&   never touched.
13 *& - Rows are inserted one by one, so one duplicate never stops the
14 *&   whole batch.
15 *&-----*
16 REPORT zfee_demand_gen_c60.
17
18 *-----*
19 * Types
20 *-----*
21 TYPES: BEGIN OF ty_out,                " one line in the result ALV
22         status      TYPE c LENGTH 20,
23         reg_no       TYPE zstudent_cs60-reg_no,
24         name         TYPE zstudent_cs60-name,
25         course       TYPE zstudent_cs60-course,
26         branch       TYPE zstudent_cs60-branch,
27         studyyear    TYPE zstudent_cs60-studyyear,
28         fee_type     TYPE zfee_details_c60-fee_type,
29         total_fee    TYPE zfee_details_c60-total_fee,
30         currency     TYPE zfee_details_c60-currency,
31         due_date     TYPE zfee_details_c60-due_date,
32     END OF ty_out.
33
34 TYPES: BEGIN OF ty_key,                " primary key of a demand row
35         reg_no       TYPE zfee_details_c60-reg_no,
36         academic_year TYPE zfee_details_c60-academic_year,
37         fee_type     TYPE zfee_details_c60-fee_type,
38     END OF ty_key.
39
40 *-----*
41 * Data
42 *-----*
43 DATA: gs_stu_sel   TYPE zstudent_cs60,    " only used to type the
44         gs_fs_sel   TYPE zfee_struct_cs60,  " select-options below
45         gt_out       TYPE STANDARD TABLE OF ty_out,
46         gt_existing  TYPE SORTED TABLE OF ty_key
47                     WITH UNIQUE KEY reg_no academic_year fee_type,
48         gs_demand    TYPE zfee_details_c60,
49         gv_found     TYPE abap_bool,
50         gv_to_create TYPE i,
51         gv_created   TYPE i,
52         gv_skipped   TYPE i,
53         gv_failed    TYPE i,
54         gv_no_fs     TYPE i,
55         gv_header    TYPE string.
56
57 *-----*
58 * Selection screen
59 *-----*
60 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE TEXT-001. " Students
61     SELECT-OPTIONS: s_course FOR gs_stu_sel-course LOWER CASE,
62                     s_branch FOR gs_stu_sel-branch,
63                     s_year   FOR gs_stu_sel-studyyear,
64                     s_regno  FOR gs_stu_sel-reg_no.
65 SELECTION-SCREEN END OF BLOCK b1.
66
67 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE TEXT-002. " Fee types
68     SELECT-OPTIONS s_ftype FOR gs_fs_sel-fee_type LOWER CASE.
69 SELECTION-SCREEN END OF BLOCK b2.
70
71 SELECTION-SCREEN BEGIN OF BLOCK b3 WITH FRAME TITLE TEXT-003. " Run mode
72     PARAMETERS p_test AS CHECKBOX DEFAULT 'X'.
73 SELECTION-SCREEN END OF BLOCK b3.
74
75 *-----*
76 START-OF-SELECTION.
77 *-----*
78
79     "1. Students that match the filters

```

```

80 SELECT reg_no, name, course, branch, studyyear
81 FROM zstudent_cs60
82 WHERE course IN @s_course
83 AND branch IN @s_branch
84 AND studyyear IN @s_year
85 AND reg_no IN @s_regno
86 ORDER BY reg_no
87 INTO TABLE @DATA(lt_students).
88
89 IF lt_students IS INITIAL.
90 MESSAGE 'No students match the selection' TYPE 'S' DISPLAY LIKE 'E'.
91 RETURN.
92 ENDIF.
93
94 "2. Price-list rows that match the filters
95 SELECT course, branch, studyyear, fee_type, amount, currency, due_date
96 FROM zfee_struct_cs60
97 WHERE course IN @s_course
98 AND branch IN @s_branch
99 AND studyyear IN @s_year
100 AND fee_type IN @s_fotype
101 INTO TABLE @DATA(lt_struct).
102
103 IF lt_struct IS INITIAL.
104 MESSAGE 'No fee structure rows match the selection' TYPE 'S' DISPLAY LIKE 'E'.
105 RETURN.
106 ENDIF.
107
108 "3. Demand rows that already exist for these students
109 " (one database read for all students, not one read per row)
110 SELECT reg_no, academic_year, fee_type
111 FROM zfee_details_c60
112 FOR ALL ENTRIES IN @lt_students
113 WHERE reg_no = @lt_students-reg_no
114 INTO TABLE @gt_existing.
115
116 "4. One demand row per student per matching fee type
117 LOOP AT lt_students INTO DATA(ls_stu).
118
119 gv_found = abap_false.
120
121 LOOP AT lt_struct INTO DATA(ls_fs)
122 WHERE course = ls_stu-course
123 AND branch = ls_stu-branch
124 AND studyyear = ls_stu-studyyear.
125
126 gv_found = abap_true.
127
128 CLEAR gs_demand.
129 gs_demand-reg_no = ls_stu-reg_no.
130 gs_demand-academic_year = ls_stu-studyyear. " year of study, 1-4
131 gs_demand-fee_type = ls_fs-fee_type.
132 gs_demand-total_fee = ls_fs-amount.
133 gs_demand-concession = 0.
134 gs_demand-late_fee = 0.
135 gs_demand-paid_amount = 0.
136 " Balance = Total - Concession + Late fee - Paid
137 gs_demand-balance = gs_demand-total_fee - gs_demand-concession
138 + gs_demand-late_fee - gs_demand-paid_amount.
139 gs_demand-currency = ls_fs-currency.
140 gs_demand-due_date = ls_fs-due_date.
141 gs_demand-course = ls_stu-course. " Delete these 3 lines if
142 gs_demand-branch = ls_stu-branch. " your ZFEE_DETAILS_C60 has no
143 gs_demand-studyyear = ls_stu-studyyear. " COURSE/BRANCH/STUDYYEAR
144
145 APPEND VALUE #( reg_no = ls_stu-reg_no
146 name = ls_stu-name
147 course = ls_stu-course
148 branch = ls_stu-branch
149 studyyear = ls_stu-studyyear
150 fee_type = ls_fs-fee_type
151 total_fee = ls_fs-amount
152 currency = ls_fs-currency
153 due_date = ls_fs-due_date )
154 TO gt_out ASSIGNING FIELD-SYMBOL(<ls_out>).
155
156 " Already generated earlier? Skip it, never overwrite it.
157 IF line_exists( gt_existing[ reg_no = gs_demand-reg_no
158 academic_year = gs_demand-academic_year
159 fee_type = gs_demand-fee_type ] ).
160 <ls_out>-status = 'EXISTS - SKIPPED'.
161 gv_skipped = gv_skipped + 1.
162 CONTINUE.
163 ENDIF.

```

```

164
165     IF p_test = abap_true.
166       <ls_out>-status = 'WILL BE CREATED'.
167       gv_to_create = gv_to_create + 1.
168     ELSE.
169       " Row-by-row insert: a duplicate returns sy-subrc = 4 instead of
170       " dumping, so the rest of the batch is still saved.
171       INSERT zfee_details_c60 FROM @gs_demand.
172       IF sy-subrc = 0.
173         <ls_out>-status = 'CREATED'.
174         gv_created = gv_created + 1.
175       ELSE.
176         <ls_out>-status = 'FAILED - DUPLICATE'.
177         gv_failed = gv_failed + 1.
178       ENDIF.
179     ENDIF.
180
181   ENDLOOP.
182
183   " Student has no price-list row for their course/branch/year
184   IF gv_found = abap_false.
185     APPEND VALUE #( status      = 'NO FEE STRUCTURE'
186                   reg_no      = ls_stu-reg_no
187                   name         = ls_stu-name
188                   course       = ls_stu-course
189                   branch       = ls_stu-branch
190                   studyyear    = ls_stu-studyyear ) TO gt_out.
191     gv_no_fs = gv_no_fs + 1.
192   ENDIF.
193
194 ENDLOOP.
195
196 "5. Save (only in a real run)
197 IF p_test = abap_false.
198   COMMIT WORK.
199 ENDIF.
200
201 "6. Show the result
202 SORT gt_out BY reg_no fee_type.
203
204 IF p_test = abap_true.
205   gv_header = |TEST RUN - To create: { gv_to_create } Exist: { gv_skipped } | &&
206             |No fee structure: { gv_no_fs }|.
207 ELSE.
208   gv_header = |SAVED - Created: { gv_created } Skipped: { gv_skipped } | &&
209             |Failed: { gv_failed } No fee structure: { gv_no_fs }|.
210 ENDIF.
211
212 TRY.
213   cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv)
214                         CHANGING t_table      = gt_out ).
215   CATCH cx_salv_msg.
216     MESSAGE 'Result list could not be displayed' TYPE 'S' DISPLAY LIKE 'E'.
217   RETURN.
218 ENDTRY.
219
220 lo_alv->get_functions( )->set_all( abap_true ).
221 lo_alv->get_columns( )->set_optimize( abap_true ).
222 lo_alv->get_display_settings( )->set_striped_pattern( abap_true ).
223 lo_alv->get_display_settings( )->set_list_header( CONV #( gv_header ) ).
224
225 TRY.
226   lo_alv->get_columns( )->get_column( 'TOTAL_FEE' )->set_currency_column( 'CURRENCY' ).
227   lo_alv->get_columns( )->get_column( 'STATUS' )->set_long_text( 'Result' ).
228   lo_alv->get_columns( )->get_column( 'STATUS' )->set_medium_text( 'Result' ).
229   lo_alv->get_columns( )->get_column( 'STATUS' )->set_short_text( 'Result' ).
230   CATCH cx_salv_not_found cx_salv_data_error.
231     " Column names are fixed in TY_OUT, so this cannot normally happen
232 ENDTRY.
233
234 MESSAGE gv_header TYPE 'S'.
235 lo_alv->display( ).

```

B.4 ZSTU_FEE_VIEW_C60

Program: student fee view · 175 lines

```

1  *&-----*
2  *& Report ZSTU_FEE_VIEW_C60
3  *&-----*
4  *& Step 10 - Student fee view
5  *&
6  *& - The user must log in first (same ZCL_AUTH_CS60 login as the app).
7  *& - STUDENT : always sees only his / her OWN fee rows. The register
8  *&           number is taken from the login, never from the screen,
9  *&           so one student can never see another student's data.
10 *& - ACCOUNTANT / ADMIN : may enter any register number.
11 *& - Status per row is derived: PAID / PARTIAL / UNPAID / OVERDUE.
12 *&-----*
13 REPORT zstu_fee_view_c60.
14
15 TYPES: BEGIN OF ty_out,
16         light          TYPE c LENGTH 1,          " traffic light icon column
17         fee_type       TYPE zfee_details_c60-fee_type,
18         academic_year  TYPE zfee_details_c60-academic_year,
19         total_fee      TYPE zfee_details_c60-total_fee,
20         concession     TYPE zfee_details_c60-concession,
21         late_fee       TYPE zfee_details_c60-late_fee,
22         paid_amount    TYPE zfee_details_c60-paid_amount,
23         balance        TYPE zfee_details_c60-balance,
24         currency       TYPE zfee_details_c60-currency,
25         due_date       TYPE zfee_details_c60-due_date,
26         status        TYPE c LENGTH 10,
27     END OF ty_out.
28
29 DATA: gt_out  TYPE STANDARD TABLE OF ty_out,
30        gv_role TYPE zusers_c60-role,
31        gv_regno TYPE zstudent_cs60-reg_no,
32        gv_header TYPE string.
33
34 *-----*
35 * Selection screen
36 *-----*
37 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE TEXT-001.   " Login
38     PARAMETERS: p_user TYPE zusers_c60-user_id OBLIGATORY,
39                 p_pass TYPE c LENGTH 30 LOWER CASE OBLIGATORY.
40 SELECTION-SCREEN END OF BLOCK b1.
41
42 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE TEXT-002.   " Staff only
43     PARAMETERS p_reg TYPE zstudent_cs60-reg_no.
44 SELECTION-SCREEN END OF BLOCK b2.
45
46 AT SELECTION-SCREEN OUTPUT.
47     LOOP AT SCREEN.
48         IF screen-name = 'P_PASS'.
49             screen-invisible = 1.                " mask the password
50             MODIFY SCREEN.
51         ENDIF.
52     ENDLOOP.
53
54 *-----*
55 START-OF-SELECTION.
56 *-----*
57
58 "1. Log in
59 DATA(ls_login) = zcl_auth_cs60=>login( iv_user_id = p_user
60                                         iv_password = CONV string( p_pass ) ).
61 CLEAR p_pass.
62 IF ls_login-ok = abap_false.
63     MESSAGE ls_login-message TYPE 'S' DISPLAY LIKE 'E'.
64     RETURN.
65 ENDIF.
66 gv_role = ls_login-role.
67
68 "2. Decide whose fees to show
69 CASE gv_role.
70     WHEN 'S'.
71         gv_regno = p_user.                " student: own register number only
72         IF p_reg IS NOT INITIAL AND p_reg <> p_user.
73             MESSAGE 'Students can view only their own fees' TYPE 'S' DISPLAY LIKE 'E'.
74             RETURN.
75         ENDIF.
76     WHEN 'A' OR 'X'.
77         gv_regno = p_reg.
78         IF gv_regno IS INITIAL.
79             MESSAGE 'Enter the register number to view' TYPE 'S' DISPLAY LIKE 'E'.

```

```

80     RETURN.
81     ENDIF.
82     WHEN OTHERS.
83         MESSAGE 'Role not allowed' TYPE 'S' DISPLAY LIKE 'E'.
84     RETURN.
85 ENDCASE.
86
87 "3. Student master record
88 SELECT SINGLE reg_no, name, course, branch, studyyear
89     FROM zstudent_cs60
90     WHERE reg_no = @gv_regno
91     INTO @DATA(ls_stu).
92 IF sy-subrc <> 0.
93     MESSAGE 'Register number not found in student records' TYPE 'S' DISPLAY LIKE 'E'.
94     RETURN.
95 ENDIF.
96
97 "4. Fee rows
98 SELECT fee_type, academic_year, total_fee, concession, late_fee,
99     paid_amount, balance, currency, due_date
100     FROM zfee_details_c60
101     WHERE reg_no = @gv_regno
102     ORDER BY academic_year, fee_type
103     INTO CORRESPONDING FIELDS OF TABLE @gt_out.
104
105 IF gt_out IS INITIAL.
106     MESSAGE 'No fee demand generated for this student yet' TYPE 'S' DISPLAY LIKE 'E'.
107     RETURN.
108 ENDIF.
109
110 "5. Derive status and traffic light per row
111 LOOP AT gt_out ASSIGNING FIELD-SYMBOL(<ls_out>).
112     IF <ls_out>-balance <= 0.
113         <ls_out>-status = 'PAID'.
114         <ls_out>-light   = '3'.
115         " green
116     ELSEIF <ls_out>-paid_amount > 0.
117         <ls_out>-status = 'PARTIAL'.
118         <ls_out>-light   = '2'.
119         " yellow
120     ELSE.
121         <ls_out>-status = 'UNPAID'.
122         <ls_out>-light   = '1'.
123         " red
124     ENDIF.
125     " Anything still owed after the due date is overdue
126     IF <ls_out>-balance > 0 AND <ls_out>-due_date IS NOT INITIAL
127         AND <ls_out>-due_date < sy-datum.
128         <ls_out>-status = 'OVERDUE'.
129         <ls_out>-light   = '1'.
130     ENDIF.
131 ENDLOOP.
132
133 "6. Header text with totals (one currency assumed: INR)
134 DATA(lv_total) = REDUCE zfee_details_c60-total_fee(
135     INIT s = CONV zfee_details_c60-total_fee( 0 )
136     FOR r IN gt_out NEXT s = s + r-total_fee ).
137 DATA(lv_paid) = REDUCE zfee_details_c60-paid_amount(
138     INIT s = CONV zfee_details_c60-paid_amount( 0 )
139     FOR r IN gt_out NEXT s = s + r-paid_amount ).
140 DATA(lv_balance) = REDUCE zfee_details_c60-balance(
141     INIT s = CONV zfee_details_c60-balance( 0 )
142     FOR r IN gt_out NEXT s = s + r-balance ).
143
144 gv_header = |{ ls_stu-reg_no } { ls_stu-name } - { ls_stu-course } { ls_stu-branch } | &&
145     |Year { ls_stu-studyyear } | &&
146     |Total { lv_total CURRENCY = gt_out[ 1 ]-currency } | &&
147     |Paid { lv_paid CURRENCY = gt_out[ 1 ]-currency } | &&
148     |Balance { lv_balance CURRENCY = gt_out[ 1 ]-currency }|.
149
150 "7. Show as ALV
151 TRY.
152     cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv)
153         CHANGING t_table = gt_out ).
154     CATCH cx_salv_msg.
155         MESSAGE 'Fee list could not be displayed' TYPE 'S' DISPLAY LIKE 'E'.
156         RETURN.
157     ENDTRY.
158
159 DATA(lo_cols) = lo_alv->get_columns( ).
160 lo_cols->set_optimize( abap_true ).
161 lo_cols->set_exception_column( 'LIGHT' ).
162 lo_alv->get_functions( )->set_all( abap_true ).
163 lo_alv->get_display_settings( )->set_stripped_pattern( abap_true ).
164 lo_alv->get_display_settings( )->set_list_header( CONV #( gv_header ) ).
165
166 TRY.

```

```

164     DATA(lt_cur_cols) = VALUE string_table( ( `TOTAL_FEE` ) ( `CONCESSION` )
165                                               ( `LATE_FEE` ) ( `PAID_AMOUNT` ) ( `BALANCE` ) ).
166     LOOP AT lt_cur_cols INTO DATA(lv_col).
167         CAST cl_salv_column_table( lo_cols->get_column( CONV #( lv_col ) )
168             )->set_currency_column( 'CURRENCY' ).
169     ENDLOOP.
170     lo_cols->get_column( 'CURRENCY' )->set_visible( abap_false ).
171     CATCH cx_salv_not_found cx_salv_data_error.
172     " Column names are fixed in TY_OUT, so this cannot normally happen
173     ENDTRY.
174
175     lo_alv->display( ).

```

B.5 ZSTU_PAYMENT_C60

Program: student payment submission and history · 219 lines

```

1  *&-----*
2  *& Report ZSTU_PAYMENT_C60
3  *&-----*
4  *& Step 12 - Student payment submission
5  *&
6  *& 1. The student logs in (User ID = register number).
7  *& 2. To pay: fill the payment block and press Execute (F8).
8  *&    The payment is saved as SUBMITTED. Nothing is paid yet - the
9  *&    accountant must verify it first (Step 14).
10 *& 3. To only look at your payments: leave the Amount empty.
11 *& 4. The list at the end shows all of the student's payments with
12 *&    their status (SUBMITTED / VERIFIED / REJECTED) and receipt number.
13 *&
14 *& The student can only ever use his or her OWN register number: it is
15 *& taken from the login, never typed on the screen.
16 *&-----*
17 REPORT zstu_payment_c60.
18
19 TYPES: BEGIN OF ty_hist,
20     light      TYPE c LENGTH 1,
21     payment_id TYPE zpayment_c60-payment_id,
22     payment_date TYPE zpayment_c60-payment_date,
23     fee_type    TYPE zpayment_c60-fee_type,
24     amount      TYPE zpayment_c60-amount,
25     currency    TYPE zpayment_c60-currency,
26     payment_mode TYPE zpayment_c60-payment_mode,
27     transaction_id TYPE zpayment_c60-transaction_id,
28     status      TYPE zpayment_c60-status,
29     receipt_no  TYPE zpayment_c60-receipt_no,
30     remark      TYPE zpayment_c60-remark,
31 END OF ty_hist.
32
33 DATA: gt_hist TYPE STANDARD TABLE OF ty_hist,
34     gv_msg TYPE string.
35
36 *-----*
37 * Selection screen (all labels are set in INITIALIZATION, so you do not
38 * need to maintain any text elements)
39 *-----*
40 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE t_b1.
41     SELECTION-SCREEN BEGIN OF LINE.
42         SELECTION-SCREEN COMMENT 1(22) c_user FOR FIELD p_user.
43         PARAMETERS p_user TYPE zusers_c60-user_id OBLIGATORY.
44     SELECTION-SCREEN END OF LINE.
45     SELECTION-SCREEN BEGIN OF LINE.
46         SELECTION-SCREEN COMMENT 1(22) c_pass FOR FIELD p_pass.
47         PARAMETERS p_pass TYPE c LENGTH 30 LOWER CASE OBLIGATORY.
48     SELECTION-SCREEN END OF LINE.
49 SELECTION-SCREEN END OF BLOCK b1.
50
51 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE t_b2.
52     SELECTION-SCREEN BEGIN OF LINE.
53         SELECTION-SCREEN COMMENT 1(22) c_fee FOR FIELD p_fee.
54         PARAMETERS p_fee TYPE zpayment_c60-fee_type LOWER CASE DEFAULT 'Tuition Fee'.
55     SELECTION-SCREEN END OF LINE.
56     SELECTION-SCREEN BEGIN OF LINE.
57         SELECTION-SCREEN COMMENT 1(22) c_acad FOR FIELD p_acad.
58         PARAMETERS p_acad TYPE zpayment_c60-academic_year DEFAULT '1'.
59     SELECTION-SCREEN END OF LINE.
60     SELECTION-SCREEN BEGIN OF LINE.
61         SELECTION-SCREEN COMMENT 1(22) c_amt FOR FIELD p_amt.
62         PARAMETERS p_amt TYPE zpayment_c60-amount.
63     SELECTION-SCREEN END OF LINE.
64     SELECTION-SCREEN BEGIN OF LINE.
65         SELECTION-SCREEN COMMENT 1(22) c_txn FOR FIELD p_txn.

```

```

66     PARAMETERS p_txn TYPE zpayment_c60-transaction_id.
67     SELECTION-SCREEN END OF LINE.
68     SELECTION-SCREEN BEGIN OF LINE.
69     SELECTION-SCREEN COMMENT 1(22) c_bank FOR FIELD p_bank.
70     PARAMETERS p_bank TYPE zpayment_c60-bank_name LOWER CASE.
71     SELECTION-SCREEN END OF LINE.
72     SELECTION-SCREEN END OF BLOCK b2.
73
74     SELECTION-SCREEN BEGIN OF BLOCK b3 WITH FRAME TITLE t_b3.
75     SELECTION-SCREEN BEGIN OF LINE.
76     PARAMETERS p_upi RADIOBUTTON GROUP m DEFAULT 'X'.
77     SELECTION-SCREEN COMMENT 4(20) c_upi FOR FIELD p_upi.
78     SELECTION-SCREEN END OF LINE.
79     SELECTION-SCREEN BEGIN OF LINE.
80     PARAMETERS p_neft RADIOBUTTON GROUP m.
81     SELECTION-SCREEN COMMENT 4(20) c_neft FOR FIELD p_neft.
82     SELECTION-SCREEN END OF LINE.
83     SELECTION-SCREEN BEGIN OF LINE.
84     PARAMETERS p_dd RADIOBUTTON GROUP m.
85     SELECTION-SCREEN COMMENT 4(20) c_dd FOR FIELD p_dd.
86     SELECTION-SCREEN END OF LINE.
87     SELECTION-SCREEN BEGIN OF LINE.
88     PARAMETERS p_chq RADIOBUTTON GROUP m.
89     SELECTION-SCREEN COMMENT 4(20) c_chq FOR FIELD p_chq.
90     SELECTION-SCREEN END OF LINE.
91     SELECTION-SCREEN END OF BLOCK b3.
92
93     INITIALIZATION.
94     t_b1 = 'Step 1 - Log in'.
95     t_b2 = 'Step 2 - Payment details (leave Amount empty to only view your payments)'.
96     t_b3 = 'Payment mode'.
97     c_user = 'User ID (Reg. No.)'.
98     c_pass = 'Password'.
99     c_fee = 'Fee type'.
100    c_acad = 'Year of study'.
101    c_amt = 'Amount paid'.
102    c_txn = 'UTR / Reference no.'.
103    c_bank = 'Bank name'.
104    c_upi = 'UPI'.
105    c_neft = 'NEFT / bank transfer'.
106    c_dd = 'Demand Draft (DD)'.
107    c_chq = 'Cheque'.
108
109    AT SELECTION-SCREEN OUTPUT.
110    LOOP AT SCREEN.
111    IF screen-name = 'P_PASS'.
112    screen-invisible = 1. " hide the password
113    MODIFY SCREEN.
114    ENDIF.
115    ENLOOP.
116
117    *-----*
118    START-OF-SELECTION.
119    *-----*
120
121    "1. Log in - students only
122    DATA(ls_login) = zcl_auth_cs60=>login( iv_user_id = p_user
123    iv_password = CONV string( p_pass ) ).
124
125    CLEAR p_pass.
126    IF ls_login-ok = abap_false.
127    MESSAGE ls_login-message TYPE 'S' DISPLAY LIKE 'E'.
128    RETURN.
129    ENDIF.
130    IF ls_login-role <> 'S'.
131    MESSAGE 'This screen is for students. Accountants use the counter entry screen.'
132    TYPE 'S' DISPLAY LIKE 'E'.
133    RETURN.
134    ENDIF.
135
136    " The student's register number is the login ID - never typed on screen.
137    " (Variables are typed exactly like the class parameters.)
138    DATA: gv_reg TYPE zpayment_c60-reg_no,
139    gv_by TYPE zpayment_c60-entered_by,
140    gv_like TYPE c LENGTH 1.
141    gv_reg = p_user.
142    gv_by = p_user.
143
144    "2. Submit a payment (only when an amount was typed)
145    IF p_amt <> 0.
146    DATA(gv_mode) = CONV zpayment_c60-payment_mode(
147    COND string( WHEN p_upi = abap_true THEN 'UPI'
148    WHEN p_neft = abap_true THEN 'NEFT'
149    WHEN p_dd = abap_true THEN 'DD'

```

```

150                                     ELSE                                     'CHEQUE' ) ).
151
152     IF p_txn IS INITIAL.
153         gv_msg = 'Enter the UTR / reference number of your payment'.
154     ELSEIF p_bank IS INITIAL.
155         gv_msg = 'Enter the bank name'.
156     ELSE.
157         DATA(ls_res) = zcl_payment_c60=>submit(
158             iv_reg_no      = gv_reg
159             iv_fee_type     = p_fee
160             iv_acad_year   = p_acad
161             iv_amount      = p_amt
162             iv_txn_id      = CONV zpayment_c60-transaction_id( to_upper( p_txn ) )
163             iv_mode        = gv_mode
164             iv_bank        = p_bank
165             iv_entered     = gv_by ).
166         gv_msg = ls_res-message.
167     ENDIF.
168
169     gv_like = COND #( WHEN ls_res-ok = abap_true THEN 'S' ELSE 'E' ).
170     MESSAGE gv_msg TYPE 'S' DISPLAY LIKE gv_like.
171 ENDIF.
172
173 "3. This student's payments, newest first
174 SELECT payment_id, payment_date, fee_type, amount, currency, payment_mode,
175        transaction_id, status, receipt_no, remark
176 FROM zpayment_c60
177 WHERE reg_no = @gv_reg
178 ORDER BY payment_id DESCENDING
179 INTO CORRESPONDING FIELDS OF TABLE @gt_hist.
180
181 IF gt_hist IS INITIAL.
182     IF gv_msg IS INITIAL.
183         MESSAGE 'You have not submitted any payment yet' TYPE 'S'.
184     ENDIF.
185     RETURN.
186 ENDIF.
187
188 LOOP AT gt_hist ASSIGNING FIELD-SYMBOL(<ls_h>).
189     <ls_h>-light = SWITCH #( <ls_h>-status WHEN 'VERIFIED' THEN '3'
190                             WHEN 'SUBMITTED' THEN '2'
191                             ELSE '1' ).
192 ENLOOP.
193
194 TRY.
195     cl_salv_table=>factory( IMPORTING r_salv_table = DATA(lo_alv)
196                           CHANGING t_table      = gt_hist ).
197     CATCH cx_salv_msg.
198         MESSAGE 'Payment list could not be displayed' TYPE 'S' DISPLAY LIKE 'E'.
199     RETURN.
200 ENDTRY.
201
202 DATA(lo_cols) = lo_alv->get_columns( ).
203 lo_cols->set_optimize( abap_true ).
204 lo_cols->set_exception_column( 'LIGHT' ).
205 lo_alv->get_functions( )->set_all( abap_true ).
206 lo_alv->get_display_settings( )->set_striped_pattern( abap_true ).
207 lo_alv->get_display_settings( )->set_list_header(
208     CONV #( COND string( WHEN gv_msg IS NOT INITIAL THEN gv_msg
209                          ELSE |My payments - { gv_reg }| ) ) ).
210
211 TRY.
212     CAST cl_salv_column_table( lo_cols->get_column( 'AMOUNT' )
213                             )->set_currency_column( 'CURRENCY' ).
214     lo_cols->get_column( 'CURRENCY' )->set_visible( abap_false ).
215     CATCH cx_salv_not_found cx_salv_data_error.
216         " Column names are fixed in TY_HIST, so this cannot normally happen
217 ENDTRY.
218
219 lo_alv->display( ).

```

B.6 ZACC_COUNTER_C60

Program: accountant counter entry · 216 lines

```

1  *&-----*
2  *& Report ZACC_COUNTER_C60
3  *&-----*
4  *& Step 13 - Accountant counter entry (cash, DD, cheque)
5  *&
6  *& A student pays at the college counter. The accountant:
7  *& 1. logs in,
8  *& 2. types the register number and the payment details,
9  *& 3. presses Execute (F8).
10 *& The payment is saved. With "Verify now" ticked (default) the
11 *& accountant has the money in hand, so it is approved at once and a
12 *& receipt number is issued. Un-tick it to send the payment to the
13 *& verification list (ZACC_VERIFY_C60) instead.
14 *&-----*
15 REPORT zacc_counter_c60.
16
17 DATA: gv_reg TYPE zpayment_c60-reg_no,
18        gv_by  TYPE zpayment_c60-entered_by,
19        gv_ver  TYPE zpayment_c60-verified_by,
20        gv_mode TYPE zpayment_c60-payment_mode,
21        gv_txn  TYPE zpayment_c60-transaction_id,
22        gv_like TYPE c LENGTH 1,
23        gv_msg  TYPE string.
24
25 *-----*
26 * Selection screen
27 *-----*
28 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE t_b1.
29   SELECTION-SCREEN BEGIN OF LINE.
30     SELECTION-SCREEN COMMENT 1(24) c_user FOR FIELD p_user.
31     PARAMETERS p_user TYPE zusers_c60-user_id OBLIGATORY.
32   SELECTION-SCREEN END OF LINE.
33   SELECTION-SCREEN BEGIN OF LINE.
34     SELECTION-SCREEN COMMENT 1(24) c_pass FOR FIELD p_pass.
35     PARAMETERS p_pass TYPE c LENGTH 30 LOWER CASE OBLIGATORY.
36   SELECTION-SCREEN END OF LINE.
37 SELECTION-SCREEN END OF BLOCK b1.
38
39 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE t_b2.
40   SELECTION-SCREEN BEGIN OF LINE.
41     SELECTION-SCREEN COMMENT 1(24) c_reg FOR FIELD p_reg.
42     PARAMETERS p_reg TYPE zpayment_c60-reg_no OBLIGATORY.
43   SELECTION-SCREEN END OF LINE.
44   SELECTION-SCREEN BEGIN OF LINE.
45     SELECTION-SCREEN COMMENT 1(24) c_fee FOR FIELD p_fee.
46     PARAMETERS p_fee TYPE zpayment_c60-fee_type LOWER CASE DEFAULT 'Tuition Fee'.
47   SELECTION-SCREEN END OF LINE.
48   SELECTION-SCREEN BEGIN OF LINE.
49     SELECTION-SCREEN COMMENT 1(24) c_acad FOR FIELD p_acad.
50     PARAMETERS p_acad TYPE zpayment_c60-academic_year DEFAULT '1'.
51   SELECTION-SCREEN END OF LINE.
52   SELECTION-SCREEN BEGIN OF LINE.
53     SELECTION-SCREEN COMMENT 1(24) c_amt FOR FIELD p_amt.
54     PARAMETERS p_amt TYPE zpayment_c60-amount OBLIGATORY.
55   SELECTION-SCREEN END OF LINE.
56   SELECTION-SCREEN BEGIN OF LINE.
57     SELECTION-SCREEN COMMENT 1(24) c_ref FOR FIELD p_ref.
58     PARAMETERS p_ref TYPE zpayment_c60-transaction_id.
59   SELECTION-SCREEN END OF LINE.
60   SELECTION-SCREEN BEGIN OF LINE.
61     SELECTION-SCREEN COMMENT 1(24) c_bank FOR FIELD p_bank.
62     PARAMETERS p_bank TYPE zpayment_c60-bank_name LOWER CASE.
63   SELECTION-SCREEN END OF LINE.
64 SELECTION-SCREEN END OF BLOCK b2.
65
66 SELECTION-SCREEN BEGIN OF BLOCK b3 WITH FRAME TITLE t_b3.
67   SELECTION-SCREEN BEGIN OF LINE.
68     PARAMETERS p_cash RADIOBUTTON GROUP m DEFAULT 'X'.
69     SELECTION-SCREEN COMMENT 4(30) c_cash FOR FIELD p_cash.
70   SELECTION-SCREEN END OF LINE.
71   SELECTION-SCREEN BEGIN OF LINE.
72     PARAMETERS p_dd RADIOBUTTON GROUP m.
73     SELECTION-SCREEN COMMENT 4(30) c_dd FOR FIELD p_dd.
74   SELECTION-SCREEN END OF LINE.
75   SELECTION-SCREEN BEGIN OF LINE.
76     PARAMETERS p_chq RADIOBUTTON GROUP m.
77     SELECTION-SCREEN COMMENT 4(30) c_chq FOR FIELD p_chq.
78   SELECTION-SCREEN END OF LINE.
79 SELECTION-SCREEN END OF BLOCK b3.

```

```

80
81 SELECTION-SCREEN BEGIN OF BLOCK b4 WITH FRAME TITLE t_b4.
82   SELECTION-SCREEN BEGIN OF LINE.
83     PARAMETERS p_ver AS CHECKBOX DEFAULT 'X'.
84     SELECTION-SCREEN COMMENT 4(60) c_ver FOR FIELD p_ver.
85   SELECTION-SCREEN END OF LINE.
86 SELECTION-SCREEN END OF BLOCK b4.
87
88 INITIALIZATION.
89   t_b1 = 'Accountant log in'.
90   t_b2 = 'Payment received at the counter'.
91   t_b3 = 'Payment mode'.
92   t_b4 = 'After saving'.
93   c_user = 'User ID (Employee ID)'.
94   c_pass = 'Password'.
95   c_reg = 'Student register no.'.
96   c_fee = 'Fee type'.
97   c_acad = 'Year of study'.
98   c_amt = 'Amount received'.
99   c_ref = 'Cheque / DD no. (optional for cash)'.
100  c_bank = 'Bank name (DD / cheque)'.
101  c_cash = 'Cash'.
102  c_dd = 'Demand Draft (DD)'.
103  c_chq = 'Cheque'.
104  c_ver = 'Verify now and issue the receipt'.
105
106 AT SELECTION-SCREEN OUTPUT.
107   LOOP AT SCREEN.
108     IF screen-name = 'P_PASS'.
109       screen-invisible = 1.
110     MODIFY SCREEN.
111   ENDIF.
112 ENDLOOP.
113
114 *-----*
115 START-OF-SELECTION.
116 *-----*
117
118 "1. Log in - accountants only
119 DATA(ls_login) = zcl_auth_cs60=>login( iv_user_id = p_user
120                                       iv_password = CONV string( p_pass ) ).
121 CLEAR p_pass.
122 IF ls_login-ok = abap_false.
123   MESSAGE ls_login-message TYPE 'S' DISPLAY LIKE 'E'.
124   RETURN.
125 ENDIF.
126 IF ls_login-role <> 'A'.
127   MESSAGE 'Only accountants can record counter payments' TYPE 'S' DISPLAY LIKE 'E'.
128   RETURN.
129 ENDIF.
130 gv_by = p_user.
131 gv_ver = p_user.
132 gv_reg = p_reg.
133
134 "2. The student must exist
135 SELECT SINGLE name FROM zstudent_cs60
136   WHERE reg_no = @gv_reg
137   INTO @DATA(lv_name).
138 IF sy-subrc <> 0.
139   MESSAGE 'Register number not found in student records' TYPE 'S' DISPLAY LIKE 'E'.
140   RETURN.
141 ENDIF.
142
143 "3. Mode and reference rules
144 IF p_cash = abap_true.
145   gv_mode = 'CASH'.
146 ELSEIF p_dd = abap_true.
147   gv_mode = 'DD'.
148 ELSE.
149   gv_mode = 'CHEQUE'.
150 ENDIF.
151
152 IF gv_mode <> 'CASH'.
153   IF p_ref IS INITIAL.
154     MESSAGE 'Enter the DD / cheque number' TYPE 'S' DISPLAY LIKE 'E'.
155     RETURN.
156   ENDIF.
157   IF p_bank IS INITIAL.
158     MESSAGE 'Enter the bank name' TYPE 'S' DISPLAY LIKE 'E'.
159     RETURN.
160   ENDIF.
161 ENDIF.
162 gv_txn = to_upper( p_ref ).
163

```

```

164 "4. Save the payment
165 DATA(ls_sub) = zcl_payment_c60=>submit(
166     iv_reg_no      = gv_reg
167     iv_fee_type    = p_fee
168     iv_acad_year   = p_acad
169     iv_amount      = p_amt
170     iv_txn_id      = gv_txn
171     iv_mode        = gv_mode
172     iv_bank        = p_bank
173     iv_entered     = gv_by ).
174 IF ls_sub-ok = abap_false.
175     MESSAGE ls_sub-message TYPE 'S' DISPLAY LIKE 'E'.
176     RETURN.
177 ENDIF.
178
179 "5. Verify at once (the money is in the accountant's hand)
180 DATA ls_ver TYPE zcl_payment_c60=>ty_result.
181 IF p_ver = abap_true.
182     ls_ver = zcl_payment_c60=>verify( iv_payment_id = ls_sub-payment_id
183                                     iv_verifier   = gv_ver ).
184 ENDIF.
185
186 "6. Result
187 WRITE: / 'COUNTER PAYMENT RECORDED'.
188 ULINE.
189 WRITE: / 'Student      : ', gv_reg, lv_name,
190        / 'Fee type    : ', p_fee,
191        / 'Amount      : ', p_amt,
192        / 'Mode        : ', gv_mode,
193        / 'Reference    : ', gv_txn,
194        / 'Payment ID   : ', ls_sub-payment_id.
195 ULINE.
196 IF p_ver = abap_false.
197     WRITE: / 'Status      : SUBMITTED - approve it in the verification screen'.
198 ELSEIF ls_ver-ok = abap_true.
199     WRITE: / 'Status      : VERIFIED',
200           / 'Receipt no.  : ', ls_ver-receipt_no.
201 ELSE.
202     WRITE: / 'Status      : SUBMITTED - could not verify now:', ls_ver-message.
203     WRITE: / 'Approve it in the verification screen (ZACC_VERIFY_C60)'.
204 ENDIF.
205
206 IF p_ver = abap_true AND ls_ver-ok = abap_true.
207     gv_msg = |Counter payment verified. Receipt { ls_ver-receipt_no }|.
208     gv_like = 'S'.
209 ELSEIF p_ver = abap_false.
210     gv_msg = |Counter payment saved. ID { ls_sub-payment_id }|.
211     gv_like = 'S'.
212 ELSE.
213     gv_msg = |Saved but not verified: { ls_ver-message }|.
214     gv_like = 'E'.
215 ENDIF.
216 MESSAGE gv_msg TYPE 'S' DISPLAY LIKE gv_like.

```

B.7 ZACC_VERIFY_C60

Program: accountant verification (approve / reject) · 243 lines

```

1  *&-----*
2  *& Report ZACC_VERIFY_C60
3  *&-----*
4  *& Step 14 - Accountant verification screen
5  *&
6  *& 1. The accountant logs in (accountants only).
7  *& 2. The list shows every payment waiting for verification.
8  *& 3. DOUBLE-CLICK a row:
9  *&     Approve -> receipt number is created, the balance is updated,
10 *&        the audit log is written (all in one unit).
11 *&     Reject  -> a reason is asked, the payment is marked REJECTED.
12 *& 4. The list refreshes by itself after every action.
13 *&-----*
14 REPORT zacc_verify_c60.
15
16 TYPES: BEGIN OF ty_pay,
17     payment_id TYPE zpayment_c60-payment_id,
18     payment_date TYPE zpayment_c60-payment_date,
19     reg_no      TYPE zpayment_c60-reg_no,
20     name        TYPE zstudent_cs60-name,
21     fee_type    TYPE zpayment_c60-fee_type,
22     amount      TYPE zpayment_c60-amount,
23     currency    TYPE zpayment_c60-currency,
24     payment_mode TYPE zpayment_c60-payment_mode,

```

```

25         transaction_id TYPE zpayment_c60-transaction_id,
26         bank_name      TYPE zpayment_c60-bank_name,
27         entered_by     TYPE zpayment_c60-entered_by,
28         status         TYPE zpayment_c60-status,
29         receipt_no     TYPE zpayment_c60-receipt_no,
30         remark         TYPE zpayment_c60-remark,
31     END OF ty_pay.
32
33 DATA: gt_pay TYPE STANDARD TABLE OF ty_pay,
34       go_alv TYPE REF TO cl_salv_table,
35       gv_by  TYPE zpayment_c60-verified_by.
36
37 *-----*
38 * Selection screen
39 *-----*
40 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE t_b1.
41     SELECTION-SCREEN BEGIN OF LINE.
42         SELECTION-SCREEN COMMENT 1(22) c_user FOR FIELD p_user.
43         PARAMETERS p_user TYPE zusers_c60-user_id OBLIGATORY.
44     SELECTION-SCREEN END OF LINE.
45     SELECTION-SCREEN BEGIN OF LINE.
46         SELECTION-SCREEN COMMENT 1(22) c_pass FOR FIELD p_pass.
47         PARAMETERS p_pass TYPE c LENGTH 30 LOWER CASE OBLIGATORY.
48     SELECTION-SCREEN END OF LINE.
49 SELECTION-SCREEN END OF BLOCK b1.
50
51 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE t_b2.
52     SELECTION-SCREEN BEGIN OF LINE.
53         PARAMETERS p_all AS CHECKBOX.
54         SELECTION-SCREEN COMMENT 4(50) c_all FOR FIELD p_all.
55     SELECTION-SCREEN END OF LINE.
56 SELECTION-SCREEN END OF BLOCK b2.
57
58 INITIALIZATION.
59     t_b1 = 'Accountant log in'.
60     t_b2 = 'Which payments?'.
61     c_user = 'User ID (Employee ID)'.
62     c_pass = 'Password'.
63     c_all = 'Also show VERIFIED and REJECTED payments'.
64
65 AT SELECTION-SCREEN OUTPUT.
66     LOOP AT SCREEN.
67         IF screen-name = 'P_PASS'.
68             screen-invisible = 1.
69         MODIFY SCREEN.
70     ENDIF.
71 ENDLOOP.
72
73 *-----*
74 * Reads the payments (called at the start and after every action)
75 *-----*
76 FORM load_data.
77     DATA lr_status TYPE RANGE OF zpayment_c60-status.
78     IF p_all IS INITIAL.
79         lr_status = VALUE #( ( sign = 'I' option = 'EQ' low = 'SUBMITTED' ) ).
80     ENDIF.
81
82 CLEAR gt_pay.
83 SELECT p-payment_id, p-payment_date, p-reg_no, s-name, p-fee_type, p-amount,
84        p-currency, p-payment_mode, p-transaction_id, p-bank_name,
85        p-entered_by, p-status, p-receipt_no, p-remark
86     FROM zpayment_c60 AS p
87    LEFT OUTER JOIN zstudent_cs60 AS s ON s-reg_no = p-reg_no
88   WHERE p-status IN @lr_status
89   ORDER BY p-payment_id
90  INTO CORRESPONDING FIELDS OF TABLE @gt_pay.
91 ENDFORM.
92
93 *-----*
94 * Double-click on a row = Approve or Reject
95 *-----*
96 CLASS lcl_h DEFINITION.
97     PUBLIC SECTION.
98         CLASS-METHODS on_double_click
99             FOR EVENT double_click OF cl_salv_events_table
100             IMPORTING row column.
101 ENDClass.
102
103 CLASS lcl_h IMPLEMENTATION.
104     METHOD on_double_click.
105         DATA: lv_ans TYPE c LENGTH 1,
106               lv_rc  TYPE c LENGTH 1,
107               lv_text TYPE c LENGTH 200,
108               lv_remark TYPE zpayment_c60-remark,

```

```

109         lv_like    TYPE c LENGTH 1,
110         lt_sval     TYPE STANDARD TABLE OF sval,
111         ls_res      TYPE zcl_payment_c60=>ty_result.
112
113     READ TABLE gt_pay INTO DATA(ls_pay) INDEX row.
114     IF sy-subrc <> 0.
115         RETURN.
116     ENDIF.
117
118     IF ls_pay-status <> 'SUBMITTED'.
119         MESSAGE |Payment { ls_pay-payment_id } is { ls_pay-status }. Only SUBMITTED payments can be proces
sed.|
120         TYPE 'S' DISPLAY LIKE 'E'.
121         RETURN.
122     ENDIF.
123
124     lv_text = |{ ls_pay-reg_no } { ls_pay-name } - { ls_pay-fee_type } - | &&
125              |{ ls_pay-amount CURRENCY = ls_pay-currency } { ls_pay-currency } by { ls_pay-payment_mode
}| &&
126              |Ref: { ls_pay-transaction_id } Bank: { ls_pay-bank_name }. | &&
127              |Did you check this in the bank statement?|.
128
129     CALL FUNCTION 'POPUP_TO_CONFIRM'
130     EXPORTING
131         titlebar          = 'Verify payment'
132         text_question     = lv_text
133         text_button_1     = 'Approve'
134         text_button_2     = 'Reject'
135         default_button    = '1'
136         display_cancel_button = 'X'
137     IMPORTING
138         answer            = lv_ans
139     EXCEPTIONS
140         text_not_found    = 1
141         OTHERS            = 2.
142     IF sy-subrc <> 0.
143         RETURN.
144     ENDIF.
145
146     CASE lv_ans.
147         WHEN '1'.
148             " Approve
149             ls_res = zcl_payment_c60=>verify( iv_payment_id = ls_pay-payment_id
150                                             iv_verifier   = gv_by ).
151         WHEN '2'.
152             " Reject - ask for the reason
153             lt_sval = VALUE #( ( tabname   = 'ZPAYMENT_C60'
154                               fieldname = 'REMARK'
155                               fieldtext = 'Reason for rejection' ) ).
156             CALL FUNCTION 'POPUP_GET_VALUES'
157             EXPORTING
158                 popup_title = 'Reject payment'
159             IMPORTING
160                 returncode = lv_rc
161             TABLES
162                 fields     = lt_sval
163             EXCEPTIONS
164                 error_in_fields = 1
165                 OTHERS        = 2.
166             IF sy-subrc <> 0 OR lv_rc = 'A'.
167                 " cancelled
168                 RETURN.
169             ENDIF.
170             READ TABLE lt_sval INDEX 1 INTO DATA(ls_sval).
171             lv_remark = ls_sval-value.
172             ls_res = zcl_payment_c60=>reject( iv_payment_id = ls_pay-payment_id
173                                             iv_verifier   = gv_by
174                                             iv_remark    = lv_remark ).
175         WHEN OTHERS.
176             " Cancel
177             RETURN.
178     ENDCASE.
179
180     " Show the new situation and the result message
181     PERFORM load_data.
182     go_alv->refresh( ).
183
184     IF ls_res-ok = abap_true.
185         lv_like = 'S'.
186     ELSE.
187         lv_like = 'E'.
188     ENDIF.
189     MESSAGE ls_res-message TYPE 'S' DISPLAY LIKE lv_like.
190     ENDMETHOD.
191 ENDCLASS.

```

```

191 *-----*
192 START-OF-SELECTION.
193 *-----*
194
195 "1. Log in - accountants only
196 DATA(ls_login) = zcl_auth_cs60=>login( iv_user_id = p_user
197                                     iv_password = CONV string( p_pass ) ).
198 CLEAR p_pass.
199 IF ls_login-ok = abap_false.
200   MESSAGE ls_login-message TYPE 'S' DISPLAY LIKE 'E'.
201   RETURN.
202 ENDIF.
203 IF ls_login-role <> 'A'.
204   MESSAGE 'Only accountants can verify payments' TYPE 'S' DISPLAY LIKE 'E'.
205   RETURN.
206 ENDIF.
207 gv_by = p_user.
208
209 "2. Load and show the list
210 PERFORM load_data.
211 IF gt_pay IS INITIAL.
212   MESSAGE 'No payments are waiting for verification' TYPE 'S'.
213   RETURN.
214 ENDIF.
215
216 TRY.
217   cl_salv_table=>factory( IMPORTING r_salv_table = go_alv
218                         CHANGING t_table      = gt_pay ).
219   CATCH cx_salv_msg.
220     MESSAGE 'Payment list could not be displayed' TYPE 'S' DISPLAY LIKE 'E'.
221     RETURN.
222 ENDTRY.
223
224 DATA(lo_cols) = go_alv->get_columns( ).
225 lo_cols->set_optimize( abap_true ).
226 go_alv->get_functions( )->set_all( abap_true ).
227 go_alv->get_display_settings( )->set_striped_pattern( abap_true ).
228 go_alv->get_display_settings( )->set_list_header(
229   'Double-click a payment to Approve or Reject' ).
230 go_alv->get_selections( )->set_selection_mode( if_salv_c_selection_mode=>single ).
231
232 TRY.
233   CAST cl_salv_column_table( lo_cols->get_column( 'AMOUNT' )
234     )->set_currency_column( 'CURRENCY' ).
235   lo_cols->get_column( 'CURRENCY' )->set_visible( abap_false ).
236   CATCH cx_salv_not_found cx_salv_data_error.
237     " Column names are fixed in TY_PAY, so this cannot normally happen
238 ENDTRY.
239
240 DATA(lo_events) = go_alv->get_event( ).
241 SET HANDLER lcl_h=>on_double_click FOR lo_events.
242
243 go_alv->display( ).

```

B.8 ZDRIVER_FEE_RECEIPT_C60

Program: receipt driver (preview, PDF, print count, audit log) · 280 lines

```

1  *&-----*
2  *& Report ZDRIVER_FEE_RECEIPT_C60
3  *&-----*
4  *& Step 16 - Prints the fee receipt (Smart Form ZSF_RECEIPT_C60)
5  *&
6  *& 1. Type a receipt number (example FR/2026/000010).
7  *& 2. Press Execute (F8).
8  *&   - Default: the print preview opens.
9  *&   - Tick "Save as PDF file": a Save dialog opens and the receipt
10 *&    is saved as a PDF on your computer.
11 *& Every successful print or PDF adds 1 to PRINT_COUNT of the receipt
12 *& and writes a PRINT / PDF line in the audit log (Step 20).
13 *&-----*
14 REPORT zdriver_fee_receipt_c60.
15
16 PARAMETERS: p_rec TYPE zpayment_c60-receipt_no OBLIGATORY,
17             p_pdf AS CHECKBOX.
18
19 DATA: ls_payment TYPE zpayment_c60,
20       ls_student TYPE zstudent_cs60,
21       ls_fee     TYPE zfee_details_c60,
22       ls_receipt TYPE zreceipt_c60,
23       ls_smartform TYPE zst_fee_l03,
24       lv_year     TYPE zfee_details_c60-academic_year,

```

```

25     lv_fm_name   TYPE rs38l_fnam,
26     lv_ok        TYPE abap_bool.
27
28  INITIALIZATION.
29     %p_rec_%app%-text = 'Receipt No.'.
30     %p_pdf_%app%-text = 'Save as PDF file'.
31
32  START-OF-SELECTION.
33
34     "1. The payment of this receipt
35     SELECT SINGLE * FROM zpayment_c60
36         WHERE receipt_no = @p_rec
37         INTO @ls_payment.
38     IF sy-subrc <> 0.
39         MESSAGE 'Receipt number not found' TYPE 'S' DISPLAY LIKE 'E'.
40         RETURN.
41     ENDIF.
42
43     "2. The student
44     SELECT SINGLE * FROM zstudent_cs60
45         WHERE reg_no = @ls_payment-reg_no
46         INTO @ls_student.
47     IF sy-subrc <> 0.
48         MESSAGE 'Student details not found' TYPE 'S' DISPLAY LIKE 'E'.
49         RETURN.
50     ENDIF.
51
52     "3. The fee row of THIS payment (register no. + year + fee type)
53     lv_year = ls_payment-academic_year.
54     SELECT SINGLE * FROM zfee_details_c60
55         WHERE reg_no      = @ls_payment-reg_no
56         AND academic_year = @lv_year
57         AND fee_type      = @ls_payment-fee_type
58         INTO @ls_fee.
59     IF sy-subrc <> 0.
60         MESSAGE 'Fee details not found for this payment' TYPE 'S' DISPLAY LIKE 'E'.
61         RETURN.
62     ENDIF.
63
64     "4. The receipt row (issue date)
65     SELECT SINGLE * FROM zreceipt_c60
66         WHERE receipt_no = @p_rec
67         INTO @ls_receipt.
68
69     "5. Fill the form structure
70     CLEAR ls_smartform.
71     ls_smartform-receipt_no      = ls_payment-receipt_no.
72     ls_smartform-issued_on       = ls_receipt-issued_on.
73     ls_smartform-reg_no          = ls_student-reg_no.
74     ls_smartform-name            = ls_student-name.
75     ls_smartform-course          = ls_student-course.
76     ls_smartform-branch          = ls_student-branch.
77     ls_smartform-study_year      = ls_student-studyyear.
78     ls_smartform-academic_year   = ls_fee-academic_year.
79     ls_smartform-fee_type        = ls_fee-fee_type.
80     ls_smartform-currency        = ls_fee-currency.
81     ls_smartform-total_fee       = ls_fee-total_fee.
82     ls_smartform-concession      = ls_fee-concession.
83     ls_smartform-late_fee        = ls_fee-late_fee.
84     ls_smartform-paid_amount     = ls_fee-paid_amount.
85     ls_smartform-balance         = ls_fee-balance.
86     ls_smartform-amount          = ls_payment-amount.
87     ls_smartform-transaction_id  = ls_payment-transaction_id.
88     ls_smartform-payment_mode    = ls_payment-payment_mode.
89     ls_smartform-payment_date    = ls_payment-payment_date.
90     ls_smartform-bank_name       = ls_payment-bank_name.
91     ls_smartform-verified_by     = ls_payment-verified_by.
92     ls_smartform-status          = ls_payment-status.
93
94     "6. Find the function module of the form
95     CALL FUNCTION 'SSF_FUNCTION_MODULE_NAME'
96         EXPORTING
97             formname      = 'ZSF_RECEIPT_C60'
98         IMPORTING
99             fm_name       = lv_fm_name
100     EXCEPTIONS
101         no_form           = 1
102         no_function_module = 2
103         OTHERS            = 3.
104     IF sy-subrc <> 0.
105         MESSAGE 'Smart Form ZSF_RECEIPT_C60 not found or not active' TYPE 'S' DISPLAY LIKE 'E'.
106         RETURN.
107     ENDIF.
108

```

```

109 "7. Preview or PDF
110 IF p_pdf = abap_true.
111     PERFORM save_as_pdf CHANGING lv_ok.
112 ELSE.
113     PERFORM show_preview CHANGING lv_ok.
114 ENDIF.
115
116 "8. Count the print and write the audit log (one unit: both or none)
117 IF lv_ok = abap_true.
118     PERFORM count_and_log.
119 ENDIF.
120
121 *-----*
122 * print_count + 1 and one line in the audit log
123 *-----*
124 FORM count_and_log.
125     DATA: ls_log TYPE zaudit_log_c60,
126           lv_max TYPE zaudit_log_c60-log_id.
127
128     UPDATE zreceipt_c60
129         SET print_count = print_count + 1
130         WHERE receipt_no = @p_rec
131         AND print_count < 255.
132
133     SELECT MAX( log_id ) FROM zaudit_log_c60 INTO @lv_max.
134
135     ls_log-log_id      = lv_max + 1.
136     ls_log-user_id     = sy-uname.
137     IF p_pdf = abap_true.
138         ls_log-action  = 'PDF'.
139     ELSE.
140         ls_log-action  = 'PRINT'.
141     ENDIF.
142     ls_log-object_key = p_rec.
143     GET TIME STAMP FIELD ls_log-logged_at.
144     ls_log-remark     = |Receipt { p_rec } printed|.
145     INSERT zaudit_log_c60 FROM @ls_log.
146     IF sy-subrc <> 0.
147         ROLLBACK WORK.
148         MESSAGE 'Print was done, but the audit log could not be written' TYPE 'S' DISPLAY LIKE 'W'.
149     RETURN.
150 ENDIF.
151
152 COMMIT WORK.
153 ENDFORM.
154
155 *-----*
156 * Print preview on the screen
157 *-----*
158 FORM show_preview CHANGING cv_ok TYPE abap_bool.
159     cv_ok = abap_false.
160     CALL FUNCTION lv_fm_name
161         EXPORTING
162             is_fee      = ls_smartform
163         EXCEPTIONS
164             formatting_error = 1
165             internal_error  = 2
166             send_error      = 3
167             user_canceled   = 4
168             OTHERS          = 5.
169     IF sy-subrc = 0.
170         cv_ok = abap_true.
171     ELSE.
172         MESSAGE 'Error while generating the fee receipt' TYPE 'S' DISPLAY LIKE 'E'.
173     ENDIF.
174 ENDFORM.
175
176 *-----*
177 * Receipt saved as a PDF file on the user's computer
178 *-----*
179 FORM save_as_pdf CHANGING cv_ok TYPE abap_bool.
180     DATA: ls_ctrl TYPE ssfctrltop,
181           ls_out  TYPE ssfcompop,
182           ls_job  TYPE ssfcrescl,
183           lt_lines TYPE STANDARD TABLE OF tline,
184           lv_size TYPE i,
185           lv_name  TYPE string,
186           lv_filename TYPE string,
187           lv_path   TYPE string,
188           lv_fullpath TYPE string,
189           lv_action TYPE i.
190
191     cv_ok = abap_false.
192

```

```

193 "a. Make the form (no screen, give the result back as OTF)
194 ls_ctrl-no_dialog = abap_true.
195 ls_ctrl-getotf = abap_true.
196 ls_ctrl-preview = space.
197 ls_out-tddest = 'LOCL'.
198 ls_out-tdnoprev = abap_true.
199
200 CALL FUNCTION lv_fm_name
201   EXPORTING
202     control_parameters = ls_ctrl
203     output_options     = ls_out
204     user_settings      = space
205     is_fee             = ls_smartform
206   IMPORTING
207     job_output_info    = ls_job
208   EXCEPTIONS
209     formatting_error   = 1
210     internal_error     = 2
211     send_error        = 3
212     user_canceled     = 4
213     OTHERS            = 5.
214 IF sy-subrc <> 0.
215   MESSAGE 'Error while generating the fee receipt' TYPE 'S' DISPLAY LIKE 'E'.
216   RETURN.
217 ENDIF.
218
219 "b. OTF -> PDF
220 CALL FUNCTION 'CONVERT_OTF'
221   EXPORTING
222     format              = 'PDF'
223   IMPORTING
224     bin_filesize        = lv_size
225   TABLES
226     otf                = ls_job-otfdata
227     lines               = lt_lines
228   EXCEPTIONS
229     err_max_linewidth   = 1
230     err_format          = 2
231     err_conv_not_possible = 3
232     err_bad_otf        = 4
233     OTHERS             = 5.
234 IF sy-subrc <> 0.
235   MESSAGE 'PDF could not be created' TYPE 'S' DISPLAY LIKE 'E'.
236   RETURN.
237 ENDIF.
238
239 "c. Ask where to save (FR/2026/000010 -> FR_2026_000010.pdf)
240 lv_name = p_rec.
241 REPLACE ALL OCCURRENCES OF '/' IN lv_name WITH '_'.
242 lv_name = |{ lv_name }.pdf|.
243
244 cl_gui_frontend_services=>file_save_dialog(
245   EXPORTING
246     default_extension   = 'pdf'
247     default_file_name   = lv_name
248   CHANGING
249     filename            = lv_filename
250     path                = lv_path
251     fullpath            = lv_fullpath
252     user_action         = lv_action
253   EXCEPTIONS
254     cntl_error          = 1
255     error_no_gui        = 2
256     not_supported_by_gui = 3
257     OTHERS              = 4 ).
258 IF sy-subrc <> 0 OR lv_action <> cl_gui_frontend_services=>action_ok.
259   MESSAGE 'PDF was not saved' TYPE 'S' DISPLAY LIKE 'E'.
260   RETURN.
261 ENDIF.
262
263 "d. Write the file
264 CALL FUNCTION 'GUI_DOWNLOAD'
265   EXPORTING
266     bin_filesize = lv_size
267     filename     = lv_fullpath
268     filetype     = 'BIN'
269   TABLES
270     data_tab     = lt_lines
271   EXCEPTIONS
272     OTHERS       = 1.
273 IF sy-subrc <> 0.
274   MESSAGE 'PDF file could not be written' TYPE 'S' DISPLAY LIKE 'E'.
275   RETURN.
276 ENDIF.

```

```

277
278     cv_ok = abap_true.
279     MESSAGE |Receipt saved as { lv_fullpath }| TYPE 'S'.
280 ENDFORM.

```

B.9 ZCDS_FEE_PAY

CDS view with OData publication · 80 lines

```

1  @AbapCatalog.sqlViewName: 'ZCDS_FEE_C60'
2  @AbapCatalog.compiler.compareFilter: true
3  @AbapCatalog.preserveKey: true
4  @AccessControl.authorizationCheck: #NOT_REQUIRED
5  @EndUserText.label: 'Student Fee Payment'
6  @Metadata.ignorePropagatedAnnotations: true
7  @OData.publish: true
8
9  // Step 17 - one row per PAYMENT, with the student, the fee demand it
10 // belongs to and the receipt. Used by the ALV reports of Step 18
11 // and published as an OData service.
12 define view ZCDS_FEE_PAY
13 as select from zpayment_c60 as p
14     inner join zstudent_cs60 as s
15         on p.reg_no = s.reg_no
16     left outer join zfee_details_c60 as f
17         on f.reg_no = p.reg_no
18         and f.academic_year = substring(p.academic_year, 1, 1)
19         and f.fee_type = p.fee_type
20     left outer join zreceipt_c60 as r
21         on r.receipt_no = p.receipt_no
22 {
23     key p.payment_id,
24     key p.reg_no,
25
26     // student
27     s.name,
28     s.branch,
29     s.course,
30     s.studyyear,
31
32     // payment (old names kept)
33     p.transaction_id,
34
35     @Semantics.amount.currencyCode: 'Currency'
36     p.amount      as Amount,
37
38     @Semantics.currencyCode: true
39     p.currency     as Currency,
40
41     p.payment_mode,
42     p.payment_date,
43     p.status,
44
45     // payment (new)
46     p.fee_type,
47     p.academic_year,
48     p.bank_name,
49     p.entered_by,
50     p.verified_by,
51     p.verified_on,
52     p.remark,
53     p.receipt_no,
54
55     // receipt
56     r.issued_on,
57     r.print_count,
58     r.cancelled,
59
60     // fee demand of this payment
61     @Semantics.currencyCode: true
62     f.currency     as FeeCurrency,
63
64     @Semantics.amount.currencyCode: 'FeeCurrency'
65     f.total_fee    as TotalFee,
66
67     @Semantics.amount.currencyCode: 'FeeCurrency'
68     f.concession   as Concession,
69
70     @Semantics.amount.currencyCode: 'FeeCurrency'
71     f.late_fee     as LateFee,
72
73     @Semantics.amount.currencyCode: 'FeeCurrency'

```

```

74         f.paid_amount    as PaidAmount,
75
76         @Semantics.amount.currencyCode: 'FeeCurrency'
77         f.balance         as Balance,
78
79         f.due_date
80     }

```

B.10 ZRPT_PAY_REGISTER_C60

Report: payment register · 78 lines

```

1  *&-----*
2  *& Report ZRPT_PAY_REGISTER_C60
3  *& Step 18 - Report 1: PAYMENT REGISTER (ALV)
4  *& Shows every payment (SUBMITTED / VERIFIED / REJECTED) with filters
5  *&-----*
6  REPORT zrpt_pay_register_c60.
7
8  TABLES: zpayment_c60.
9
10 SELECT-OPTIONS: s_reg  FOR zpayment_c60-reg_no,
11                 s_date FOR zpayment_c60-payment_date,
12                 s_stat FOR zpayment_c60-status,
13                 s_type FOR zpayment_c60-fee_type,
14                 s_mode FOR zpayment_c60-payment_mode.
15
16 DATA: gt_pay TYPE STANDARD TABLE OF zcds_fee_pay,
17        go_alv TYPE REF TO cl_salv_table.
18
19 *-----*
20 * Proper names for the selection screen fields
21 *-----*
22 INITIALIZATION.
23   %_s_reg_%_app%-text = 'Register No.'.
24   %_s_date_%_app%-text = 'Payment Date'.
25   %_s_stat_%_app%-text = 'Status'.
26   %_s_type_%_app%-text = 'Fee Type'.
27   %_s_mode_%_app%-text = 'Payment Mode'.
28
29 START-OF-SELECTION.
30
31 SELECT * FROM zcds_fee_pay
32   WHERE reg_no      IN @s_reg
33   AND payment_date IN @s_date
34   AND status       IN @s_stat
35   AND fee_type     IN @s_type
36   AND payment_mode IN @s_mode
37   ORDER BY payment_date DESCENDING, payment_id DESCENDING
38   INTO TABLE @gt_pay.
39
40 IF gt_pay IS INITIAL.
41   MESSAGE 'No payments found for the selection' TYPE 'S' DISPLAY LIKE 'W'.
42   RETURN.
43 ENDIF.
44
45 TRY.
46   cl_salv_table=>factory(
47     IMPORTING r_salv_table = go_alv
48     CHANGING t_table      = gt_pay ).
49
50   go_alv->get_functions( )->set_all( abap_true ).
51   go_alv->get_display_settings( )->set_list_header( 'Payment Register' ).
52   go_alv->get_display_settings( )->set_striped_pattern( abap_true ).
53   go_alv->get_columns( )->set_optimize( abap_true ).
54
55   " hide the fee-demand columns - they belong to other reports
56   DATA(lo_cols) = go_alv->get_columns( ).
57   DATA(lt_hide) = VALUE string_table( ( `FEECURRENCY` ) ( `TOTALFEE` )
58     ( `CONCESSION` ) ( `LATEFEE` ) ( `PAIDAMOUNT` )
59     ( `BALANCE` ) ( `DUE_DATE` ) ( `CANCELLED` ) ).
60   LOOP AT lt_hide INTO DATA(lv_hide).
61     TRY.
62       lo_cols->get_column( CONV #( lv_hide ) )->set_visible( abap_false ).
63     CATCH cx_salv_not_found.
64   ENDTRY.
65   ENDLLOOP.
66
67   " total of Amount
68   TRY.
69     go_alv->get_aggregations( )->add_aggregation(
70       columnname = 'AMOUNT' ).

```

```

71      CATCH cx_salv_not_found cx_salv_data_error cx_salv_existing.
72      ENDTRY.
73
74      go_alv->display( ).
75
76      CATCH cx_salv_msg INTO DATA(lx_msg).
77      MESSAGE lx_msg->get_text( ) TYPE 'E'.
78      ENDTRY.

```

B.11 ZRPT_DEFAULTERS_C60

Report: fee defaulters · 142 lines

```

1  *&-----*
2  *& Report ZRPT_DEFAULTERS_C60
3  *& Step 18 - Report 2: DEFAULTERS (ALV)
4  *& Students whose balance is still unpaid after the due date
5  *&-----*
6  REPORT zrpt_defaulters_c60.
7
8  TABLES: zstudent_cs60.
9
10 TYPES: BEGIN OF ty_def,
11         reg_no      TYPE zfee_details_c60-reg_no,
12         name         TYPE zstudent_cs60-name,
13         branch       TYPE zstudent_cs60-branch,
14         studyyear    TYPE zstudent_cs60-studyyear,
15         phone        TYPE zstudent_cs60-phone,
16         email        TYPE zstudent_cs60-email,
17         fee_type     TYPE zfee_details_c60-fee_type,
18         currency     TYPE zfee_details_c60-currency,
19         total_fee    TYPE zfee_details_c60-total_fee,
20         concession   TYPE zfee_details_c60-concession,
21         late_fee     TYPE zfee_details_c60-late_fee,
22         paid_amount  TYPE zfee_details_c60-paid_amount,
23         balance      TYPE zfee_details_c60-balance,
24         due_date     TYPE zfee_details_c60-due_date,
25         days_late    TYPE i,
26     END OF ty_def.
27
28 SELECT-OPTIONS: s_reg FOR zstudent_cs60-reg_no,
29                 s_br  FOR zstudent_cs60-branch,
30                 s_yr  FOR zstudent_cs60-studyyear.
31 PARAMETERS:    p_asof TYPE d DEFAULT sy-datum,
32                 p_all  AS CHECKBOX. " also show balances not yet due
33
34 DATA: gt_def TYPE STANDARD TABLE OF ty_def,
35        go_alv TYPE REF TO cl_salv_table.
36
37 *-----*
38 * Proper names for the selection screen fields
39 *-----*
40 INITIALIZATION.
41   %s_reg_%app%-text = 'Register No.'.
42   %s_br_%app%-text = 'Branch'.
43   %s_yr_%app%-text = 'Year of Study'.
44   %p_asof_%app%-text = 'Overdue as on date'.
45   %p_all_%app%-text = 'Show not-yet-due balances'.
46
47 START-OF-SELECTION.
48
49   SELECT f~reg_no, s~name, s~branch, s~studyyear, s~phone, s~email,
50          f~fee_type, f~currency, f~total_fee, f~concession, f~late_fee,
51          f~paid_amount, f~balance, f~due_date
52     FROM zfee_details_c60 AS f
53    INNER JOIN zstudent_cs60 AS s ON s~reg_no = f~reg_no
54   WHERE f~reg_no IN @s_reg
55         AND s~branch IN @s_br
56         AND s~studyyear IN @s_yr
57         AND f~balance > 0
58   INTO CORRESPONDING FIELDS OF TABLE @gt_def.
59
60   " days overdue; drop rows not yet due (unless the checkbox is ticked)
61   LOOP AT gt_def ASSIGNING FIELD-SYMBOL(<ls_def>).
62     IF <ls_def>-due_date IS NOT INITIAL AND <ls_def>-due_date < p_asof.
63       <ls_def>-days_late = p_asof - <ls_def>-due_date.
64     ENDIF.
65   ENDLOOP.
66
67   IF p_all IS INITIAL.
68     DELETE gt_def WHERE days_late <= 0.
69   ENDIF.

```

```

70
71 IF gt_def IS INITIAL.
72   MESSAGE 'No defaulters found - nobody is overdue' TYPE 'S'.
73   RETURN.
74 ENDIF.
75
76 SORT gt_def BY days_late DESCENDING reg_no.
77
78 TRY.
79   cl_salv_table=>factory(
80     IMPORTING r_salv_table = go_alv
81     CHANGING  t_table      = gt_def ).
82
83   go_alv->get_functions( )->set_all( abap_true ).
84   go_alv->get_display_settings( )->set_list_header( 'Fee Defaulters' ).
85   go_alv->get_display_settings( )->set_striped_pattern( abap_true ).
86
87   DATA(lo_cols) = go_alv->get_columns( ).
88   lo_cols->set_optimize( abap_true ).
89
90   PERFORM head USING lo_cols 'REG_NO'      'Reg. No.'.
91   PERFORM head USING lo_cols 'NAME'        'Student Name'.
92   PERFORM head USING lo_cols 'BRANCH'      'Branch'.
93   PERFORM head USING lo_cols 'STUDYYEAR'   'Year'.
94   PERFORM head USING lo_cols 'PHONE'       'Phone'.
95   PERFORM head USING lo_cols 'EMAIL'       'E-mail'.
96   PERFORM head USING lo_cols 'FEE_TYPE'    'Fee Type'.
97   PERFORM head USING lo_cols 'CURRENCY'    'Currency'.
98   PERFORM head USING lo_cols 'TOTAL_FEE'   'Total Fee'.
99   PERFORM head USING lo_cols 'CONCESSION'  'Concession'.
100  PERFORM head USING lo_cols 'LATE_FEE'     'Late Fee'.
101  PERFORM head USING lo_cols 'PAID_AMOUNT'  'Paid'.
102  PERFORM head USING lo_cols 'BALANCE'      'Balance Due'.
103  PERFORM head USING lo_cols 'DUE_DATE'     'Due Date'.
104  PERFORM head USING lo_cols 'DAYS_LATE'    'Days Overdue'.
105
106  " total of the balance
107  TRY.
108    go_alv->get_aggregations( )->add_aggregation( columnname = 'BALANCE' ).
109    CATCH cx_salv_not_found cx_salv_data_error cx_salv_existing.
110  ENDTRY.
111
112  go_alv->display( ).
113
114  CATCH cx_salv_msg INTO DATA(lx_msg).
115  MESSAGE lx_msg->get_text( ) TYPE 'E'.
116 ENDTRY.
117
118 *-----*
119 * heading text + currency reference for one column
120 *-----*
121 FORM head USING io_cols TYPE REF TO cl_salv_columns_table
122   iv_name TYPE lvc_fname
123   iv_text TYPE csequence.
124   DATA lv_l TYPE scrtext_l.
125   DATA lv_m TYPE scrtext_m.
126   DATA lv_s TYPE scrtext_s.
127   lv_l = iv_text.
128   lv_m = iv_text.
129   lv_s = iv_text.
130   TRY.
131     DATA(lo_col) = io_cols->get_column( iv_name ).
132     lo_col->set_long_text( lv_l ).
133     lo_col->set_medium_text( lv_m ).
134     lo_col->set_short_text( lv_s ).
135     CASE iv_name.
136       WHEN 'TOTAL_FEE' OR 'CONCESSION' OR 'LATE_FEE'
137         OR 'PAID_AMOUNT' OR 'BALANCE'.
138         CAST cl_salv_column_table( lo_col )->set_currency_column( 'CURRENCY' ).
139     ENDCASE.
140     CATCH cx_salv_not_found cx_salv_data_error.
141   ENDTRY.
142 ENDFORM.

```

B.12 ZRPT_DAILY_COLL_C60

Report: daily collection · 108 lines

```

1  *&-----*
2  *& Report ZRPT_DAILY_COLL_C60
3  *& Step 18 - Report 3: DAILY COLLECTION (ALV)
4  *& VERIFIED payments only, per day and per payment mode
5  *&-----*
6  REPORT zrpt_daily_coll_c60.
7
8  TABLES: zpayment_c60.
9
10 TYPES: BEGIN OF ty_day,
11         payment_date TYPE zpayment_c60-payment_date,
12         payment_mode TYPE zpayment_c60-payment_mode,
13         currency      TYPE zpayment_c60-currency,
14         cnt           TYPE i,
15         total         TYPE zpayment_c60-amount,
16     END OF ty_day.
17
18 SELECT-OPTIONS: s_date FOR zpayment_c60-payment_date,
19                 s_mode FOR zpayment_c60-payment_mode.
20
21 DATA: gt_day TYPE STANDARD TABLE OF ty_day,
22        go_alv TYPE REF TO cl_salv_table.
23
24 *-----*
25 * Proper names for the selection screen fields
26 *-----*
27 INITIALIZATION.
28     %s_date_%app%-text = 'Payment Date'.
29     %s_mode_%app%-text = 'Payment Mode'.
30
31 START-OF-SELECTION.
32
33     SELECT payment_date, payment_mode, currency,
34            COUNT( * ) AS cnt,
35            SUM( amount ) AS total
36     FROM   zpayment_c60
37     WHERE  status = 'VERIFIED'
38            AND payment_date IN @s_date
39            AND payment_mode IN @s_mode
40     GROUP BY payment_date, payment_mode, currency
41     ORDER BY payment_date DESCENDING, payment_mode
42     INTO CORRESPONDING FIELDS OF TABLE @gt_day.
43
44 IF gt_day IS INITIAL.
45     MESSAGE 'No verified payments found for the selection' TYPE 'S' DISPLAY LIKE 'W'.
46     RETURN.
47 ENDIF.
48
49 TRY.
50     cl_salv_table=>factory(
51         IMPORTING r_salv_table = go_alv
52         CHANGING  t_table      = gt_day ).
53
54     go_alv->get_functions( )->set_all( abap_true ).
55     go_alv->get_display_settings( )->set_list_header( 'Daily Fee Collection' ).
56     go_alv->get_display_settings( )->set_striped_pattern( abap_true ).
57
58     DATA(lo_cols) = go_alv->get_columns( ).
59     lo_cols->set_optimize( abap_true ).
60
61     PERFORM head USING lo_cols 'PAYMENT_DATE' 'Date'.
62     PERFORM head USING lo_cols 'PAYMENT_MODE' 'Payment Mode'.
63     PERFORM head USING lo_cols 'CURRENCY' 'Currency'.
64     PERFORM head USING lo_cols 'CNT' 'No. of Payments'.
65     PERFORM head USING lo_cols 'TOTAL' 'Amount Collected'.
66
67     " grand totals
68     DATA(lo_aggr) = go_alv->get_aggregations( ).
69     TRY.
70         lo_aggr->add_aggregation( columnname = 'CNT' ).
71         lo_aggr->add_aggregation( columnname = 'TOTAL' ).
72         CATCH cx_salv_not_found cx_salv_data_error cx_salv_existing.
73     ENDTRY.
74
75     " sub-total for every date
76     TRY.
77         go_alv->get_sorts( )->add_sort(
78             columnname = 'PAYMENT_DATE'
79             subtotal   = abap_true ).

```

```

80      CATCH cx_salv_not_found cx_salv_existing cx_salv_data_error.
81      ENDTRY.
82
83      go_alv->display( ).
84
85      CATCH cx_salv_msg INTO DATA(lx_msg).
86      MESSAGE lx_msg->get_text( ) TYPE 'E'.
87      ENDTRY.
88
89      FORM head USING io_cols TYPE REF TO cl_salv_columns_table
90          iv_name TYPE lvc_fname
91          iv_text TYPE csequence.
92      DATA lv_l TYPE scrtext_l.
93      DATA lv_m TYPE scrtext_m.
94      DATA lv_s TYPE scrtext_s.
95      lv_l = iv_text.
96      lv_m = iv_text.
97      lv_s = iv_text.
98      TRY.
99          DATA(lo_col) = io_cols->get_column( iv_name ).
100         lo_col->set_long_text( lv_l ).
101         lo_col->set_medium_text( lv_m ).
102         lo_col->set_short_text( lv_s ).
103         IF iv_name = 'TOTAL'.
104             CAST cl_salv_column_table( lo_col )->set_currency_column( 'CURRENCY' ).
105         ENDIF.
106         CATCH cx_salv_not_found cx_salv_data_error.
107     ENDTRY.
108 ENDFORM.

```

B.13 ZRPT_FEETYPE_COLL_C60

Report: fee-type-wise collection · 133 lines

```

1  *&-----*
2  *& Report ZRPT_FEETYPE_COLL_C60
3  *& Step 18 - Report 4: FEE-TYPE-WISE COLLECTION (ALV)
4  *& Per fee type and branch: demand, collected, balance, % collected
5  *&-----*
6  REPORT zrpt_feetype_coll_c60.
7
8  TABLES: zfee_details_c60, zstudent_cs60.
9
10 TYPES: BEGIN OF ty_ft,
11     fee_type      TYPE zfee_details_c60-fee_type,
12     branch        TYPE zstudent_cs60-branch,
13     currency      TYPE zfee_details_c60-currency,
14     students      TYPE i,
15     total_fee     TYPE zfee_details_c60-total_fee,
16     concession    TYPE zfee_details_c60-concession,
17     paid_amount   TYPE zfee_details_c60-paid_amount,
18     balance       TYPE zfee_details_c60-balance,
19     pct           TYPE p LENGTH 5 DECIMALS 2,
20 END OF ty_ft.
21
22 SELECT-OPTIONS: s_type FOR zfee_details_c60-fee_type,
23                 s_br   FOR zstudent_cs60-branch,
24                 s_yr   FOR zstudent_cs60-studyyear.
25
26 DATA: gt_ft TYPE STANDARD TABLE OF ty_ft,
27        go_alv TYPE REF TO cl_salv_table.
28
29 *-----*
30 * Proper names for the selection screen fields
31 *-----*
32 INITIALIZATION.
33   %_s_type_%_app%-text = 'Fee Type'.
34   %_s_br_%_app%-text = 'Branch'.
35   %_s_yr_%_app%-text = 'Year of Study'.
36
37 START-OF-SELECTION.
38
39 SELECT f~fee_type, s~branch, f~currency,
40        COUNT( * ) AS students,
41        SUM( f~total_fee ) AS total_fee,
42        SUM( f~concession ) AS concession,
43        SUM( f~paid_amount ) AS paid_amount,
44        SUM( f~balance ) AS balance
45 FROM zfee_details_c60 AS f
46 INNER JOIN zstudent_cs60 AS s ON s~reg_no = f~reg_no
47 WHERE f~fee_type IN @s_type
48        AND s~branch IN @s_br

```

```

49     AND s~studyyear IN @s_yr
50     GROUP BY f~fee_type, s~branch, f~currency
51     ORDER BY f~fee_type, s~branch
52     INTO CORRESPONDING FIELDS OF TABLE @gt_ft.
53
54 IF gt_ft IS INITIAL.
55     MESSAGE 'No fee data found for the selection' TYPE 'S' DISPLAY LIKE 'W'.
56     RETURN.
57 ENDIF.
58
59 " percentage collected = paid / total fee
60 LOOP AT gt_ft ASSIGNING FIELD-SYMBOL(<ls_ft>).
61     IF <ls_ft>-total_fee > 0.
62         <ls_ft>-pct = <ls_ft>-paid_amount * 100 / <ls_ft>-total_fee.
63     ENDIF.
64 ENDLOOP.
65
66 TRY.
67     cl_salv_table=>factory(
68         IMPORTING r_salv_table = go_alv
69         CHANGING t_table      = gt_ft ).
70
71     go_alv->get_functions( )->set_all( abap_true ).
72     go_alv->get_display_settings( )->set_list_header( 'Fee-Type-wise Collection' ).
73     go_alv->get_display_settings( )->set_striped_pattern( abap_true ).
74
75     DATA(lo_cols) = go_alv->get_columns( ).
76     lo_cols->set_optimize( abap_true ).
77
78     PERFORM head USING lo_cols 'FEE_TYPE'      'Fee Type'.
79     PERFORM head USING lo_cols 'BRANCH'        'Branch'.
80     PERFORM head USING lo_cols 'CURRENCY'      'Currency'.
81     PERFORM head USING lo_cols 'STUDENTS'      'No. of Students'.
82     PERFORM head USING lo_cols 'TOTAL_FEE'     'Total Demand'.
83     PERFORM head USING lo_cols 'CONCESSION'    'Concession'.
84     PERFORM head USING lo_cols 'PAID_AMOUNT'   'Collected'.
85     PERFORM head USING lo_cols 'BALANCE'       'Balance'.
86     PERFORM head USING lo_cols 'PCT'          '% Collected'.
87
88     " grand totals (not for the percentage)
89     DATA(lo_aggr) = go_alv->get_aggregations( ).
90     TRY.
91         lo_aggr->add_aggregation( columnname = 'STUDENTS' ).
92         lo_aggr->add_aggregation( columnname = 'TOTAL_FEE' ).
93         lo_aggr->add_aggregation( columnname = 'CONCESSION' ).
94         lo_aggr->add_aggregation( columnname = 'PAID_AMOUNT' ).
95         lo_aggr->add_aggregation( columnname = 'BALANCE' ).
96     CATCH cx_salv_not_found cx_salv_data_error cx_salv_existing.
97     ENDTRY.
98
99     " sub-total for every fee type
100    TRY.
101        go_alv->get_sorts( )->add_sort(
102            columnname = 'FEE_TYPE'
103            subtotal   = abap_true ).
104    CATCH cx_salv_not_found cx_salv_existing cx_salv_data_error.
105    ENDTRY.
106
107    go_alv->display( ).
108
109    CATCH cx_salv_msg INTO DATA(lx_msg).
110    MESSAGE lx_msg->get_text( ) TYPE 'E'.
111    ENDTRY.
112
113 FORM head USING io_cols TYPE REF TO cl_salv_columns_table
114             iv_name TYPE lvc_fname
115             iv_text TYPE csequence.
116 DATA lv_l TYPE scrtext_l.
117 DATA lv_m TYPE scrtext_m.
118 DATA lv_s TYPE scrtext_s.
119 lv_l = iv_text.
120 lv_m = iv_text.
121 lv_s = iv_text.
122 TRY.
123     DATA(lo_col) = io_cols->get_column( iv_name ).
124     lo_col->set_long_text( lv_l ).
125     lo_col->set_medium_text( lv_m ).
126     lo_col->set_short_text( lv_s ).
127     CASE iv_name.
128         WHEN 'TOTAL_FEE' OR 'CONCESSION' OR 'PAID_AMOUNT' OR 'BALANCE'.
129             CAST cl_salv_column_table( lo_col )->set_currency_column( 'CURRENCY' ).
130     ENDCASE.
131     CATCH cx_salv_not_found cx_salv_data_error.
132     ENDTRY.

```

133 ENDFORM.

B.14 ZFEE_DASH_C60

Program: fee dashboard (transaction ZFEEDASH) · 135 lines

```

1  *&-----*
2  *& Report ZFEE_DASH_C60
3  *& Step 19 - FEE DASHBOARD: key numbers + buttons to the 4 reports
4  *&-----*
5  REPORT zfee_dash_c60.
6
7  TABLES: sscrfields.
8
9  *-----*
10 * Screen layout
11 *-----*
12 SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE t_b1.
13   SELECTION-SCREEN BEGIN OF LINE.
14     SELECTION-SCREEN COMMENT 1(32) l1.
15     SELECTION-SCREEN COMMENT 35(25) v1.
16   SELECTION-SCREEN END OF LINE.
17   SELECTION-SCREEN BEGIN OF LINE.
18     SELECTION-SCREEN COMMENT 1(32) l2.
19     SELECTION-SCREEN COMMENT 35(25) v2.
20   SELECTION-SCREEN END OF LINE.
21   SELECTION-SCREEN BEGIN OF LINE.
22     SELECTION-SCREEN COMMENT 1(32) l3.
23     SELECTION-SCREEN COMMENT 35(25) v3.
24   SELECTION-SCREEN END OF LINE.
25   SELECTION-SCREEN BEGIN OF LINE.
26     SELECTION-SCREEN COMMENT 1(32) l4.
27     SELECTION-SCREEN COMMENT 35(25) v4.
28   SELECTION-SCREEN END OF LINE.
29   SELECTION-SCREEN BEGIN OF LINE.
30     SELECTION-SCREEN COMMENT 1(32) l5.
31     SELECTION-SCREEN COMMENT 35(25) v5.
32   SELECTION-SCREEN END OF LINE.
33   SELECTION-SCREEN BEGIN OF LINE.
34     SELECTION-SCREEN COMMENT 1(32) l6.
35     SELECTION-SCREEN COMMENT 35(25) v6.
36   SELECTION-SCREEN END OF LINE.
37   SELECTION-SCREEN BEGIN OF LINE.
38     SELECTION-SCREEN COMMENT 1(32) l7.
39     SELECTION-SCREEN COMMENT 35(25) v7.
40   SELECTION-SCREEN END OF LINE.
41 SELECTION-SCREEN END OF BLOCK b1.
42
43 SELECTION-SCREEN BEGIN OF BLOCK b2 WITH FRAME TITLE t_b2.
44   SELECTION-SCREEN PUSHBUTTON /1(40) b_reg  USER-COMMAND reg.
45   SELECTION-SCREEN PUSHBUTTON /1(40) b_def  USER-COMMAND def.
46   SELECTION-SCREEN PUSHBUTTON /1(40) b_day  USER-COMMAND day.
47   SELECTION-SCREEN PUSHBUTTON /1(40) b_type USER-COMMAND typ.
48 SELECTION-SCREEN END OF BLOCK b2.
49
50 *-----*
51 * Texts of the screen, set once
52 *-----*
53 INITIALIZATION.
54   t_b1 = 'Fee Collection Summary'.
55   t_b2 = 'Reports'.
56   l1   = 'Total fee demand'.
57   l2   = 'Total collected'.
58   l3   = 'Total balance'.
59   l4   = '% collected'.
60   l5   = 'Collected today (verified)'.
61   l6   = 'Awaiting verification'.
62   l7   = 'Overdue fee records'.
63   b_reg = 'Payment Register'.
64   b_def = 'Defaulters'.
65   b_day = 'Daily Collection'.
66   b_type = 'Fee-Type-wise Collection'.
67
68 *-----*
69 * Numbers are re-read every time the screen is shown, so they are
70 * fresh again after coming back from a report
71 *-----*
72 AT SELECTION-SCREEN OUTPUT.
73   PERFORM load_numbers.
74
75 *-----*
76 * Button clicks

```

```

77 *-----*
78 AT SELECTION-SCREEN.
79   CASE sscrfields-ucomm.
80     WHEN 'REG'.
81       SUBMIT zrpt_pay_register_c60 VIA SELECTION-SCREEN AND RETURN.
82     WHEN 'DEF'.
83       SUBMIT zrpt_defaulter_c60 VIA SELECTION-SCREEN AND RETURN.
84     WHEN 'DAY'.
85       SUBMIT zrpt_daily_coll_c60 VIA SELECTION-SCREEN AND RETURN.
86     WHEN 'TYP'.
87       SUBMIT zrpt_feetype_coll_c60 VIA SELECTION-SCREEN AND RETURN.
88   ENDCASE.
89
90 START-OF-SELECTION.
91   " nothing to do - everything happens on the screen
92
93 *-----*
94 FORM load_numbers.
95   DATA: lv_demand TYPE p LENGTH 15 DECIMALS 2,
96         lv_paid    TYPE p LENGTH 15 DECIMALS 2,
97         lv_bal     TYPE p LENGTH 15 DECIMALS 2,
98         lv_today   TYPE p LENGTH 15 DECIMALS 2,
99         lv_pct     TYPE p LENGTH 7  DECIMALS 2,
100        lv_pend    TYPE i,
101        lv_over     TYPE i.
102
103   SELECT SUM( total_fee ), SUM( paid_amount ), SUM( balance )
104     FROM zfee_details_c60
105     INTO ( @lv_demand, @lv_paid, @lv_bal ).
106
107   SELECT SUM( amount )
108     FROM zpayment_c60
109     WHERE status = 'VERIFIED' AND payment_date = @sy-datum
110     INTO @lv_today.
111
112   SELECT COUNT( * )
113     FROM zpayment_c60
114     WHERE status = 'SUBMITTED'
115     INTO @lv_pend.
116
117   SELECT COUNT( * )
118     FROM zfee_details_c60
119     WHERE balance > 0
120           AND due_date <> '00000000'
121           AND due_date < @sy-datum
122     INTO @lv_over.
123
124   IF lv_demand > 0.
125     lv_pct = lv_paid * 100 / lv_demand.
126   ENDIF.
127
128   v1 = |{ lv_demand CURRENCY = 'INR' } INR|.
129   v2 = |{ lv_paid    CURRENCY = 'INR' } INR|.
130   v3 = |{ lv_bal     CURRENCY = 'INR' } INR|.
131   v4 = |{ lv_pct     DECIMALS = 2 } %|.
132   v5 = |{ lv_today   CURRENCY = 'INR' } INR|.
133   v6 = |{ lv_pend }|.
134   v7 = |{ lv_over }|.
135 ENDFORM.

```

B.15 ZRPT_AUDIT_C60

Report: audit log viewer · 114 lines

```

1  *&-----*
2  *& Report ZRPT_AUDIT_C60
3  *& Step 20 - AUDIT LOG viewer (ALV)
4  *& Who verified / rejected which payment, and when
5  *&-----*
6  REPORT zrpt_audit_c60.
7
8  TABLES: zaudit_log_c60.
9
10 TYPES: BEGIN OF ty_log,
11         log_id      TYPE zaudit_log_c60-log_id,
12         log_date    TYPE d,
13         log_time     TYPE t,
14         user_id     TYPE zaudit_log_c60-user_id,
15         action      TYPE zaudit_log_c60-action,
16         object_key  TYPE zaudit_log_c60-object_key,
17         remark      TYPE zaudit_log_c60-remark,
18     END OF ty_log.

```

```

19
20 SELECT-OPTIONS: s_user FOR zaudit_log_c60-user_id,
21                 s_act  FOR zaudit_log_c60-action,
22                 s_date FOR sy-datum.
23
24 DATA: gt_log TYPE STANDARD TABLE OF ty_log,
25        go_alv TYPE REF TO cl_salv_table.
26
27 *-----*
28 * Proper names for the selection screen fields
29 *-----*
30 INITIALIZATION.
31   %_s_user_%_app%-text = 'User ID'.
32   %_s_act_%_app%-text  = 'Action (VERIFY / REJECT)'.
33   %_s_date_%_app%-text = 'Date'.
34
35 START-OF-SELECTION.
36
37 DATA: ls_log TYPE ty_log,
38        lv_tz  TYPE tznzone.
39
40 IF sy-zonlo IS INITIAL.
41   lv_tz = 'INDIA'.
42 ELSE.
43   lv_tz = sy-zonlo.
44 ENDIF.
45
46 SELECT log_id, logged_at, user_id, action, object_key, remark
47 FROM   zaudit_log_c60
48 WHERE  user_id IN @s_user
49        AND action IN @s_act
50 ORDER BY log_id DESCENDING
51 INTO TABLE @DATA(lt_raw).
52
53 LOOP AT lt_raw INTO DATA(ls_raw).
54   CLEAR ls_log.
55   ls_log-log_id      = ls_raw-log_id.
56   ls_log-user_id     = ls_raw-user_id.
57   ls_log-action      = ls_raw-action.
58   ls_log-object_key  = ls_raw-object_key.
59   ls_log-remark      = ls_raw-remark.
60   CONVERT TIME STAMP ls_raw-logged_at TIME ZONE lv_tz
61   INTO DATE ls_log-log_date TIME ls_log-log_time.
62   IF ls_log-log_date IN s_date.
63     APPEND ls_log TO gt_log.
64   ENDIF.
65 ENDLOOP.
66
67 IF gt_log IS INITIAL.
68   MESSAGE 'No audit log entries found for the selection' TYPE 'S' DISPLAY LIKE 'W'.
69   RETURN.
70 ENDIF.
71
72 TRY.
73   cl_salv_table=>factory(
74     IMPORTING r_salv_table = go_alv
75     CHANGING t_table      = gt_log ).
76
77   go_alv->get_functions( )->set_all( abap_true ).
78   go_alv->get_display_settings( )->set_list_header( 'Audit Log' ).
79   go_alv->get_display_settings( )->set_striped_pattern( abap_true ).
80
81   DATA(lo_cols) = go_alv->get_columns( ).
82   lo_cols->set_optimize( abap_true ).
83
84   PERFORM head USING lo_cols 'LOG_ID'      'Log No.'.
85   PERFORM head USING lo_cols 'LOG_DATE'    'Date'.
86   PERFORM head USING lo_cols 'LOG_TIME'    'Time'.
87   PERFORM head USING lo_cols 'USER_ID'     'Done By'.
88   PERFORM head USING lo_cols 'ACTION'      'Action'.
89   PERFORM head USING lo_cols 'OBJECT_KEY'  'Receipt / Payment No.'.
90   PERFORM head USING lo_cols 'REMARK'      'Remark'.
91
92   go_alv->display( ).
93
94   CATCH cx_salv_msg INTO DATA(lx_msg).
95     MESSAGE lx_msg->get_text( ) TYPE 'E'.
96 ENDTRY.
97
98 FORM head USING io_cols TYPE REF TO cl_salv_columns_table
99               iv_name TYPE lvc_fname
100               iv_text TYPE csequence.
101 DATA lv_l TYPE scrtext_l.
102 DATA lv_m TYPE scrtext_m.

```

```

103 DATA lv_s TYPE scrtext_s.
104 lv_l = iv_text.
105 lv_m = iv_text.
106 lv_s = iv_text.
107 TRY.
108   DATA(lo_col) = io_cols->get_column( iv_name ).
109   lo_col->set_long_text( lv_l ).
110   lo_col->set_medium_text( lv_m ).
111   lo_col->set_short_text( lv_s ).
112 CATCH cx_salv_not_found.
113   ENDTRY.
114 ENDFORM.

```

B.16 ZTEST_PAYMENT_C60

Test program: 16 checks on ZCL_PAYMENT_C60 · 277 lines

```

1  *&-----*
2  *& Report ZTEST_PAYMENT_C60
3  *&-----*
4  *& Tests ZCL_PAYMENT_C60 (validate / submit / verify / reject).
5  *&
6  *& It uses ONE student fee row (default Y26ECE025 / Tuition Fee).
7  *& All test payments carry the transaction ID ZTEST-xxx.
8  *& With "Clean up" ticked, the report removes its own test payments and
9  *& receipts at the end and puts the fee row back to unpaid.
10 *& (Receipt numbers that were used stay used - that is how number
11 *& ranges work, so there will be gaps. Audit log lines are kept.)
12 *&-----*
13 REPORT ztest_payment_c60.
14
15 TYPES: BEGIN OF ty_fee,
16         total_fee TYPE zfee_details_c60-total_fee,
17         concession TYPE zfee_details_c60-concession,
18         late_fee TYPE zfee_details_c60-late_fee,
19         paid_amount TYPE zfee_details_c60-paid_amount,
20         balance TYPE zfee_details_c60-balance,
21         currency TYPE zfee_details_c60-currency,
22     END OF ty_fee.
23
24 PARAMETERS: p_reg TYPE zstudent_cs60-reg_no DEFAULT 'Y26ECE025',
25             p_fee TYPE zfee_details_c60-fee_type DEFAULT 'Tuition Fee' LOWER CASE,
26             p_acad TYPE zfee_details_c60-academic_year DEFAULT '1',
27             p_clean AS CHECKBOX DEFAULT 'X'.
28
29 DATA: gv_pass TYPE i,
30       gv_fail TYPE i,
31       gv_no TYPE i.
32
33 *-----*
34 CLASS lcl_t DEFINITION.
35     PUBLIC SECTION.
36     CLASS-METHODS check IMPORTING iv_name TYPE csequence
37                         iv_ok TYPE abap_bool
38                         iv_info TYPE csequence.
39 ENDCLASS.
40
41 CLASS lcl_t IMPLEMENTATION.
42     METHOD check.
43         gv_no = gv_no + 1.
44         IF iv_ok = abap_true.
45             gv_pass = gv_pass + 1.
46             FORMAT COLOR COL_POSITIVE.
47             WRITE: / 'PASS', gv_no LEFT-JUSTIFIED, iv_name.
48         ELSE.
49             gv_fail = gv_fail + 1.
50             FORMAT COLOR COL_NEGATIVE.
51             WRITE: / 'FAIL', gv_no LEFT-JUSTIFIED, iv_name.
52         ENDIF.
53         FORMAT COLOR OFF.
54         IF iv_info IS NOT INITIAL.
55             WRITE: / '      ->', iv_info.
56         ENDIF.
57     ENDMETHOD.
58 ENDCLASS.
59
60 FORM read_fee CHANGING cs_fee TYPE ty_fee.
61     CLEAR cs_fee.
62     SELECT SINGLE total_fee, concession, late_fee, paid_amount, balance, currency
63     FROM zfee_details_c60
64     WHERE reg_no = @p_reg
65           AND academic_year = @p_acad

```

```

66         AND fee_type      = @p_fee
67         INTO CORRESPONDING FIELDS OF @cs_fee.
68 ENDFORM.
69
70 FORM read_status USING iv_pay_id TYPE zpayment_c60-payment_id
71                   CHANGING cv_status TYPE zpayment_c60-status.
72 CLEAR cv_status.
73 SELECT SINGLE status FROM zpayment_c60
74     WHERE payment_id = @iv_pay_id
75     INTO @cv_status.
76 ENDFORM.
77
78 * Removes every ZTEST-xxx payment (and its receipt) and resets the fee row
79 FORM cleanup.
80 SELECT payment_id FROM zpayment_c60
81     WHERE reg_no = @p_reg
82           AND transaction_id LIKE 'ZTEST-%'
83     INTO TABLE @DATA(lt_ids).
84 LOOP AT lt_ids INTO DATA(ls_id).
85     DELETE FROM zreceipt_c60 WHERE pay_id = @ls_id-payment_id.
86     DELETE FROM zpayment_c60 WHERE payment_id = @ls_id-payment_id.
87 ENDLOOP.
88
89 DATA ls_fee TYPE ty_fee.
90 PERFORM read_fee CHANGING ls_fee.
91 DATA(lv_full) = ls_fee-total_fee - ls_fee-concession + ls_fee-late_fee.
92 UPDATE zfee_details_c60
93     SET paid_amount = 0,
94         balance      = @lv_full
95     WHERE reg_no     = @p_reg
96           AND academic_year = @p_acad
97           AND fee_type   = @p_fee.
98 COMMIT WORK.
99 ENDFORM.
100
101 *-----*
102 START-OF-SELECTION.
103 *-----*
104 DATA: gs_fee0 TYPE ty_fee,
105        gs_fee  TYPE ty_fee,
106        gs_r    TYPE zcl_payment_c60=>ty_result,
107        gv_id1  TYPE zpayment_c60-payment_id,
108        gv_id2  TYPE zpayment_c60-payment_id,
109        gv_rc1  TYPE zpayment_c60-receipt_no,
110        gv_status TYPE zpayment_c60-status,
111        gv_amt   TYPE zpayment_c60-amount,
112        gv_expect TYPE zfee_details_c60-balance,
113        gv_acad  TYPE zpayment_c60-academic_year,
114        gv_user  TYPE zpayment_c60-entered_by.
115
116 gv_acad = p_acad.
117 gv_user = p_reg.
118
119 " Leftovers from an earlier run that stopped half way
120 SELECT SINGLE payment_id FROM zpayment_c60
121     WHERE reg_no = @p_reg AND transaction_id LIKE 'ZTEST-%'
122     INTO @DATA(lv_left).
123 IF sy-subrc = 0.
124     PERFORM cleanup.
125 ENDIF.
126
127 " The fee row must exist and be completely unpaid before we start
128 PERFORM read_fee CHANGING gs_fee0.
129 IF gs_fee0-total_fee IS INITIAL.
130     WRITE: / 'Fee row not found for', p_reg, p_fee.
131     WRITE: / 'Run ZFEE_DEMAND_GEN_C60 first, or choose another student.'.
132     RETURN.
133 ENDIF.
134 IF gs_fee0-paid_amount <> 0.
135     WRITE: / 'This fee row already has real payments on it.'.
136     WRITE: / 'Choose a student with no payments (for example Y26ECE025)'.
137     RETURN.
138 ENDIF.
139
140 WRITE: / 'Testing on', p_reg, p_fee, 'Balance before:', gs_fee0-balance CURRENCY gs_fee0-currency.
141 ULINE.
142
143 "---- 1. Valid submission -----
144 gs_r = zcl_payment_c60=>submit(
145     iv_reg_no = p_reg iv_fee_type = p_fee iv_acad_year = gv_acad
146     iv_amount = '10000.00' iv_txn_id = 'ZTEST-001'
147     iv_mode = 'UPI' iv_bank = 'Test Bank' iv_entered = gv_user ).
148 gv_id1 = gs_r-payment_id.
149 PERFORM read_status USING gv_id1 CHANGING gv_status.

```

```

150 PERFORM read_fee CHANGING gs_fee.
151 lcl_t=>check( iv_name = 'Valid submission is accepted and saved as SUBMITTED'
152             iv_ok  = xsdbool( gs_r-ok = abap_true AND gv_status = 'SUBMITTED' )
153             iv_info = gs_r-message ).
154
155 "---- 2. Balance must NOT change at submission -----
156 lcl_t=>check( iv_name = 'Balance is unchanged after submission'
157             iv_ok  = xsdbool( gs_fee-balance = gs_fee0-balance AND gs_fee-paid_amount = 0 )
158             iv_info = |Balance now { gs_fee-balance CURRENCY = gs_fee-currency }| ).
159
160 "---- 3. Duplicate transaction ID -----
161 gs_r = zcl_payment_c60=>submit(
162     iv_reg_no = p_reg iv_fee_type = p_fee iv_acad_year = gv_acad
163     iv_amount = '1000.00' iv_txn_id = 'ZTEST-001'
164     iv_mode = 'UPI' iv_bank = 'Test Bank' iv_entered = gv_user ).
165 lcl_t=>check( iv_name = 'Duplicate transaction ID is blocked'
166             iv_ok  = xsdbool( gs_r-ok = abap_false )
167             iv_info = gs_r-message ).
168
169 "---- 4. Amount above the balance -----
170 gv_amt = gs_fee0-balance + 1.
171 gs_r = zcl_payment_c60=>validate(
172     iv_reg_no = p_reg iv_fee_type = p_fee iv_acad_year = gv_acad
173     iv_amount = gv_amt iv_txn_id = 'ZTEST-OVER' ).
174 lcl_t=>check( iv_name = 'Amount above the balance is blocked'
175             iv_ok  = xsdbool( gs_r-ok = abap_false )
176             iv_info = gs_r-message ).
177
178 "---- 5. Zero amount -----
179 gs_r = zcl_payment_c60=>validate(
180     iv_reg_no = p_reg iv_fee_type = p_fee iv_acad_year = gv_acad
181     iv_amount = '0.00' iv_txn_id = 'ZTEST-ZERO' ).
182 lcl_t=>check( iv_name = 'Zero amount is blocked'
183             iv_ok  = xsdbool( gs_r-ok = abap_false )
184             iv_info = gs_r-message ).
185
186 "---- 6. Verify: receipt number and balance -----
187 gs_r = zcl_payment_c60=>verify( iv_payment_id = gv_id1 iv_verifier = 'TESTACC' ).
188 gv_rc1 = gs_r-receipt_no.
189 PERFORM read_fee CHANGING gs_fee.
190 gv_expect = gs_fee0-balance - '10000.00'.
191 lcl_t=>check( iv_name = 'Verify creates a receipt number like FR/2026/000125'
192             iv_ok  = xsdbool( gs_r-ok = abap_true AND gv_rc1(3) = 'FR/' AND strlen( condense( gv_rc
193 1 ) ) = 14 )
194             iv_info = |Receipt: { gv_rc1 } { gs_r-message }| ).
195 lcl_t=>check( iv_name = 'Verify reduces the balance by the amount paid'
196             iv_ok  = xsdbool( gs_fee-balance = gv_expect AND gs_fee-paid_amount = '10000.00' )
197             iv_info = |Paid { gs_fee-paid_amount CURRENCY = gs_fee-currency }, balance { gs_fee-bala
198 nce CURRENCY = gs_fee-currency }| ).
199
200 "---- 7. Receipt row exists and points to the payment -----
201 SELECT SINGLE pay_id FROM zreceipt_c60
202     WHERE receipt_no = @gv_rc1
203     INTO @DATA(lv_pay_in_rcpt).
204 lcl_t=>check( iv_name = 'Receipt row is saved in ZRECEIPT_C60 and linked to the payment'
205             iv_ok  = xsdbool( sy-subrc = 0 AND lv_pay_in_rcpt = gv_id1 )
206             iv_info = '' ).
207
208 "---- 8. Verifying the same payment twice -----
209 gs_r = zcl_payment_c60=>verify( iv_payment_id = gv_id1 iv_verifier = 'TESTACC' ).
210 PERFORM read_fee CHANGING gs_fee.
211 lcl_t=>check( iv_name = 'Verifying an already VERIFIED payment is blocked'
212             iv_ok  = xsdbool( gs_r-ok = abap_false AND gs_fee-balance = gv_expect )
213             iv_info = gs_r-message ).
214
215 "---- 9. Reject needs a remark -----
216 gs_r = zcl_payment_c60=>submit(
217     iv_reg_no = p_reg iv_fee_type = p_fee iv_acad_year = gv_acad
218     iv_amount = '6000.00' iv_txn_id = 'ZTEST-002'
219     iv_mode = 'NEFT' iv_bank = 'Test Bank' iv_entered = gv_user ).
220 gv_id2 = gs_r-payment_id.
221 gs_r = zcl_payment_c60=>reject( iv_payment_id = gv_id2 iv_verifier = 'TESTACC' iv_remark = '' ).
222 lcl_t=>check( iv_name = 'Reject without a remark is blocked'
223             iv_ok  = xsdbool( gs_r-ok = abap_false )
224             iv_info = gs_r-message ).
225
226 "---- 10. Reject with a remark -----
227 gs_r = zcl_payment_c60=>reject( iv_payment_id = gv_id2 iv_verifier = 'TESTACC'
228                             iv_remark = 'UTR not found in bank statement' ).
229 PERFORM read_status USING gv_id2 CHANGING gv_status.
230 PERFORM read_fee CHANGING gs_fee.
231 lcl_t=>check( iv_name = 'Reject with a remark works and the balance stays the same'
232             iv_ok  = xsdbool( gs_r-ok = abap_true AND gv_status = 'REJECTED' AND gs_fee-balance = g
233 v_expect )

```

```

231         iv_info = gs_r-message ).
232
233 "---- 11. A rejected payment cannot be verified -----
234 gs_r = zcl_payment_c60=>verify( iv_payment_id = gv_id2 iv_verifier = 'TESTACC' ).
235 lcl_t=>check( iv_name = 'A REJECTED payment cannot be verified'
236             iv_ok   = xsdbool( gs_r-ok = abap_false )
237             iv_info = gs_r-message ).
238
239 "---- 12. Resubmit the same UTR after the rejection -----
240 gs_r = zcl_payment_c60=>submit(
241     iv_reg_no = p_reg iv_fee_type = p_fee iv_acad_year = gv_acad
242     iv_amount = '6000.00' iv_txn_id = 'ZTEST-002'
243     iv_mode = 'NEFT' iv_bank = 'Test Bank' iv_entered = gv_user ).
244 lcl_t=>check( iv_name = 'A rejected payment can be resubmitted with the same reference'
245             iv_ok   = xsdbool( gs_r-ok = abap_true )
246             iv_info = gs_r-message ).
247 gv_id2 = gs_r-payment_id.
248
249 "---- 13. Second verification: new receipt number and partial balance --
250 gs_r = zcl_payment_c60=>verify( iv_payment_id = gv_id2 iv_verifier = 'TESTACC' ).
251 PERFORM read_fee CHANGING gs_fee.
252 gv_expect = gs_fee0-balance - '16000.00'.
253 lcl_t=>check( iv_name = 'Second receipt number is different (unique)'
254             iv_ok   = xsdbool( gs_r-ok = abap_true AND gs_r-receipt_no <> gv_rc1 )
255             iv_info = |Receipt: { gs_r-receipt_no }| ).
256 lcl_t=>check( iv_name = 'Partial payments add up (10,000 + 6,000 paid, rest still due)'
257             iv_ok   = xsdbool( gs_fee-balance = gv_expect AND gs_fee-paid_amount = '16000.00' )
258             iv_info = |Paid { gs_fee-paid_amount CURRENCY = gs_fee-currency }, balance { gs_fee-bala
nce CURRENCY = gs_fee-currency }| ).
259
260 "---- 14. Audit log has the actions -----
261 SELECT COUNT(*) FROM zaudit_log_c60
262     WHERE user_id = 'TESTACC'
263     INTO @DATA(lv_logs).
264 lcl_t=>check( iv_name = 'Audit log has VERIFY and REJECT entries'
265             iv_ok   = xsdbool( lv_logs >= 3 )
266             iv_info = |{ lv_logs } log lines for TESTACC| ).
267
268 "---- Result -----
269 ULINE.
270 WRITE: / 'Passed:', gv_pass, '   Failed:', gv_fail.
271
272 IF p_clean = abap_true.
273     PERFORM cleanup.
274     WRITE: / 'Clean-up done: test payments and receipts removed, fee row reset to unpaid.'.
275 ELSE.
276     WRITE: / 'Clean-up skipped: test payments are still in the tables.'.
277 ENDIF.

```